

# OpenSeek Closest Integers 技术报告

## 1. 任务概述

本任务对应 OpenSeek 系列中的 **Closest Integers** 问题，核心目标是：给定一个整数列表，计算列表中任意两个整数之间的最小绝对差，并将结果以字符串形式输出。

输入数据通常表现为字符串格式的整数列表，例如：

```
1 "[59, 26, -96, -30]"
```

程序需要将该字符串解析为整数列表，然后在所有整数对中寻找绝对差最小的一组。对于上述示例，排序后列表为：

```
1 [-96, -30, 26, 59]
```

相邻元素差值分别为：

```
1 66, 56, 33
```

因此最小绝对差为 33，最终输出：

```
1 "33"
```

本任务虽然表面上可以通过两层循环暴力枚举所有整数对完成，但考虑到测试集规模、运行效率以及提交稳定性，代码采用了排序后比较相邻元素的方式完成计算。该方法利用了一个基本性质：对于一组数字，任意两个数之间的最小绝对差一定出现在排序后的相邻元素之间。因此，只需要对列表排序，然后线性扫描相邻差值即可。

## 2. 数据集结构与输入输出格式

数据文件为 JSON 格式，代码通过 `task1_data_loader` 函数读取数据。数据文件中主要包含以下字段：

```
1 task_id
2 task_name
3 Definition
4 examples
5 test_samples
```

其中：

`task_id` 表示当前任务编号，用于标识任务来源。

`task_name` 表示任务名称，本任务为 Closest Integers。

`Definition` 保存任务自然语言描述，主要说明输入、输出以及任务目标。

`examples` 表示训练样例或参考样例，每条样例一般包含输入和标准输出。代码会使用这些样例对模型生成的 `solve` 函数进行预验证。

`test_samples` 表示最终需要预测的测试样例。程序不会读取测试样例的标准答案，而是调用生成或回退的求解器计算预测结果。

每条测试样例的输入字段为字符串形式的整数列表。最终程序输出 JSONL 文件，每一行对应一个测试样例，格式如下：

```
1 {"test_sample_id": "xxx", "prediction": "33"}
```

其中 `test_sample_id` 来自原测试样例的 `id` 字段，`prediction` 是程序计算得到的最小绝对差结果，统一保存为字符串。

### 3. 总体设计思路

本代码不是直接手写单一求解函数后处理测试集，而是采用了“模型生成求解器 + 自动清洗 + 安全验证 + 样例验证 + 回退求解器 + 批量预测”的整体流程。

整体流程如下：

```
1  读取 JSON 数据
2      ↓
3  构造代码生成提示词
4      ↓
5  调用 Qwen3-4B 生成 solve 函数
6      ↓
7  提取并清洗生成代码
8      ↓
9  安全检查与受限环境编译
10     ↓
11 使用 examples 前 200 条验证准确率
12     ↓
13 若生成函数可用，则用于测试集预测
14 若生成函数不可用，则回退到内置 fallback_solve
15     ↓
16 多线程处理 test_samples
17     ↓
18 生成 JSONL 提交文件
```

该设计兼顾了两点：

第一，符合任务希望利用大模型生成代码的思路。程序会根据任务定义和若干样例，自动构造 prompt，并请求 Qwen3-4B 生成一个符合接口要求的 `solve(input_text: str) -> str` 函数。

第二，保证提交结果的稳定性。由于大模型生成代码可能出现格式错误、解释性文字、非法 import、函数名不符合要求、运行异常等问题，程序额外设计了代码清洗、安全检查、编译验证和 fallback 回退机制。这样即使模型生成失败，也可以使用内置确定性算法继续完成测试集预测。

## 4. 核心算法原理

本任务要求计算整数列表中任意两个整数的最小绝对差。直接做法是枚举所有二元组：

```
1 for i in range(n):
2     for j in range(i + 1, n):
3         ans = min(ans, abs(nums[i] - nums[j]))
```

该方法时间复杂度为  $O(n^2)$ ，当列表较长时效率较低。

本代码采用排序优化方法：

1. 将输入字符串解析为整数列表；
2. 将整数列表按升序排序；
3. 扫描排序后相邻元素；
4. 计算相邻元素差值；
5. 取所有相邻差值中的最小值；
6. 返回字符串结果。

其原因在于：排序后，如果两个元素之间不是相邻关系，那么它们中间至少存在一个其他元素。中间元素与两端某个元素之间的距离一定不会大于两端元素之间的距离。因此，最小绝对差只可能出现在排序后的相邻元素之间。

例如：

```
1 原列表: [10, -2, 3, 8]
2 排序后: [-2, 3, 8, 10]
3 相邻差值: 5, 5, 2
4 最小绝对差: 2
```

因此，算法复杂度为：

```
1 排序:  $O(n \log n)$ 
2 扫描:  $O(n)$ 
3 总复杂度:  $O(n \log n)$ 
```

空间复杂度主要取决于排序过程和列表存储，通常为  $O(n)$  或  $O(1)$  到  $O(n)$ ，具体取决于 Python 排序实现和输入转换过程。

## 5. 代码模块设计

### 5.1 数据加载模块

代码中的 `task1_data_loader(file_path)` 负责读取任务 JSON 文件。该函数使用 `json.load` 解析输入文件，并返回任务 ID、任务名称、任务定义、训练样例和测试样例。

其主要作用是将原始 JSON 数据转换为后续流程可直接使用的 Python 对象。返回内容包括：

```
1 task_id
2 task_name
3 definition_list
4 examples
5 test_samples
```

该模块是整个流程的入口。后续 prompt 构造、样例验证和测试集预测都依赖它提供的数据。

## 5.2 模型调用模块

代码使用 OpenAI SDK 初始化了一个兼容 OpenAI 接口格式的客户端，然后通过 `qwen_api` 函数调用 Qwen3-4B 模型。

该函数的输入是 `messages`，即聊天模型所需的消息列表。函数内部设置了：

```
1 model="/Qwen3-4B/Qwen/Qwen3-4B"
2 temperature=0.0
3 retries=3
```

其中，`temperature=0.0` 表示模型输出尽可能确定，减少随机性。由于本任务目标是生成可执行代码，而不是开放式文本创作，因此低温度设置更适合保证输出稳定。

`retries=3` 表示如果模型调用失败，程序最多尝试三次。每次失败后会等待 2 秒再重试。如果最终仍失败，则返回空字符串，并在控制台打印 API 调用失败信息。

该模块解决的是模型输出来源问题，为后续自动生成 `solve` 函数提供基础。

## 5.3 代码生成提示词设计

代码中包含两个层次的 prompt。

第一部分是系统提示词 `CODEGEN_SYSTEM_PROMPT`。

该提示词明确要求模型扮演严格的 Python 算法代码生成器，并规定：

```
1 只输出可执行 Python 代码
2 不要解释
3 不要 markdown
4 不要代码块标记
5 只定义一个 solve(input_text: str) -> str 函数
6 不要 import
7 不要 main
8 不要示例
9 不要调用 open、eval、exec、compile、__import__
10 输入是字符串格式的整数列表
11 输出必须是字符串
```

这部分约束非常重要。因为模型默认可能会输出解释文字、Markdown 代码块，或者添加 `import`、测试样例、主函数等内容。这些内容都会影响后续自动编译和调用。因此，系统提示词通过强约束将输出范围限制为一个单独的 `solve` 函数。

第二部分是用户提示词，由 `build_codegen_user_prompt` 函数动态构造。

该函数会把任务 ID、任务名称、任务定义和最多 8 条样例拼接成 prompt。例如：

```
1 任务ID: ...
2 任务名: ...
3
4 任务定义:
5 - ...
6
7 下面是若干样例:
8 输入: ...
9 输出: ...
10
11 请直接输出 solve(input_text: str) -> str 的完整 Python 代码。
12 再次强调:
13 - 不要 import
14 - 不要解释
15 - 只输出代码
```

这种设计的目的在于给模型提供足够的任务上下文，同时避免 prompt 过长。由于本任务逻辑明确，使用少量样例即可帮助模型理解输入输出格式。

## 5.4 生成代码提取模块

模型输出不一定完全符合要求。即使 prompt 中要求“不输出 markdown”，模型仍可能返回如下格式：

```
1 ```python
2 def solve(input_text: str) -> str:
3     ...
4 ```
```

因此，代码设计了 `extract_code(text)` 函数，用于从模型输出中提取真正的 Python 代码。

该函数首先检查文本中是否存在 Markdown 代码块。如果存在，则通过正则表达式提取代码块内部内容。如果不存在，则直接返回原文本。

这一步的作用是提高模型输出的容错性，使程序能够处理模型偶尔添加代码块标记的情况。

## 5.5 生成代码后处理模块

仅提取代码还不够，因为模型输出中可能仍然包含多余内容。例如：

```
1 下面是代码：
2  def solve(input_text: str) -> str:
3      ...
```

或者：

```
1  import ast
2
3  def solve(input_text: str) -> str:
4      ...
```

因此，代码进一步使用 `postprocess_generated_code(code)` 对生成结果进行清洗。该函数主要完成三类处理：

第一，删除 Markdown 残留标记，例如：

```
1  ```python
2  ```
```

第二，删除所有 `import` 和 `from ... import ...` 行。因为系统要求生成代码不要 `import`，而且后续执行环境已经预先提供了 `ast` 对象。如果允许模型自行 `import`，会增加不确定性和安全风险。

第三，如果模型在函数前输出了解释文字，则通过正则表达式定位：

```
1  def solve(input_text: str) -> str:
```

并只保留从该函数定义开始的内容。

通过这一步处理，程序可以将大模型输出转换为更规范、更容易编译的 Python 函数代码。

---

## 5.6 安全检查模块

由于模型生成的代码会被程序动态执行，因此必须进行安全检查。代码中的 `is_code_safe(code)` 函数用于检测生成代码中是否包含高风险内容。

禁止内容包括：

```
1  open(
2  eval(
3  exec(
4  compile(
5  __import__
6  os
7  sys
8  subprocess
9  pathlib
10 shutil
11 pickle
```

这些内容可能带来文件访问、系统调用、动态执行、模块导入或反序列化风险。虽然本任务只需要解析整数列表并计算最小差值，但模型生成代码存在不可控性，因此安全检查是必要环节。

如果检测到禁止模式，函数会返回：

```
1 | False, "生成代码包含不允许内容：..."
```

随后 `compile_solver` 会抛出异常，本次生成结果会被判定为不可用。

需要注意的是，这种检查属于基础安全过滤，主要用于本任务场景下的风险控制。它并不是完整的 Python 沙箱，但已经能拦截与本任务明显无关的高风险操作。

## 5.7 动态编译与受限执行环境

通过安全检查后，代码会进入 `compile_solver(code)` 函数进行动态编译。

该函数首先构造了一个受限的 `safe_builtins` 字典，只允许生成代码访问少量基础函数，例如：

```
1 | len
2 | min
3 | max
4 | sum
5 | abs
6 | str
7 | int
8 | float
9 | list
10 | dict
11 | set
12 | tuple
13 | sorted
14 | range
15 | enumerate
16 | zip
17 | map
18 | filter
19 | any
20 | all
```

同时，执行环境中提供了 `ast` 对象，允许生成代码使用：

```
1 | ast.literal_eval
```

解析输入字符串。

然后程序使用：

```
1 | exec(code, exec_globals, exec_locals)
```

执行生成代码，并从局部或全局命名空间中提取 `solve` 函数。

如果生成代码中没有定义 `solve` 函数，程序会抛出：

```
1 | 生成代码中未找到 solve 函数
```

这一模块的作用是将模型生成的文本代码转换为可调用的 Python 函数对象。后续无论是样例验证还是测试集预测，都会通过该函数对象完成。

## 5.8 内置回退求解器

考虑到模型生成代码可能失败，程序额外定义了 `fallback_solve(input_text: str) -> str` 作为保底方案。

其逻辑如下：

```
1 | nums = ast.literal_eval(input_text.strip())
2 | nums = sorted(int(x) for x in nums)
3 |
4 | if len(nums) < 2:
5 |     return "0"
6 |
7 | best = min(nums[i] - nums[i - 1] for i in range(1, len(nums)))
8 | return str(best)
```

该函数不依赖模型生成结果，完全按照本任务的确定性算法实现。它先使用 `ast.literal_eval` 安全解析输入字符串，再将列表元素转换为整数并排序。若列表长度小于 2，则返回 `"0"` 作为边界处理。否则，扫描排序后相邻元素，计算最小差值并返回字符串。

回退机制的意义在于保证程序稳定性。即使模型输出代码为空、格式错误、编译失败、验证不通过，程序仍然能够完成测试集预测，而不会因为生成式模型的不稳定性导致整个流程中断。

## 5.9 样例验证模块

程序使用 `validate_solver(solve_func, examples, limit=200)` 对生成的 `solve` 函数进行验证。

该函数最多取前 200 条 examples，逐条调用生成的 `solve_func`，并将预测结果与标准答案比较。

验证过程中，如果某条样例运行出错，程序会将预测值记录为：

```
1 | [ERROR] 错误信息
```

如果预测值与标准答案不一致，则将该样例加入 `error_cases`，保存内容包括：

```
1 | 样例 ID
2 | 输入
3 | 标准答案
4 | 预测答案
```



最终函数返回：

```
1 | acc, error_cases
```

其中 `acc` 是验证准确率，`error_cases` 是错误样例列表。

这一环节是代码可靠性的关键。因为大模型生成的代码即使能成功编译，也不一定逻辑正确。通过训练样例验证，可以在正式处理测试集之前筛掉错误求解器。

## 5.10 多次生成与最优选择机制

程序中的 `generate_valid_solver` 函数负责完整的求解器生成流程。

该函数最多尝试生成 `max_try=3` 次。每次流程为：

```
1 | 构造 messages
2 | 调用 qwen_api
3 | 提取代码
4 | 清洗代码
5 | 编译 solve 函数
6 | 使用 examples 验证
7 | 记录准确率
```

如果某一次生成代码编译失败，程序会打印：

```
1 | 第 i 次代码编译失败：...
```

然后继续下一次尝试。

如果某一次生成代码验证准确率达到 100%，程序会立即返回该版本代码和函数对象。

如果三次都没有达到 100%，但至少有一个版本能成功编译和验证，则返回准确率最高的版本。

如果所有模型生成结果都失败，则返回内置 `fallback_solve`，并记录：

```
1 | 模型生成代码均未通过验证，已回退
```

这一机制体现了“生成式方法 + 验证筛选”的思想。程序不是盲目信任模型第一次生成的代码，而是通过多次尝试和样例验证选择最可靠的函数。

## 6. 测试集预测流程

当求解器确定后，程序进入测试集预测阶段。

首先，主程序读取：

```
1 | eval_data = test_samples
```

然后使用 `functools.partial` 固定 `solve_func` 和 `task_id` 参数，得到可直接处理单条样例的函数：

```
1 process_func = partial(process_single_sample, solve_func=solve_func, task_id=task_id)
```

单条样例处理函数 `process_single_sample` 的逻辑为：

1. 读取样例 ID；
2. 读取输入字符串；
3. 调用 `solve_func(input_text)` 计算预测值；
4. 如果运行异常，则输出错误信息，并将预测值设为 `"0"`；
5. 返回包含任务 ID、样例 ID 和预测值的字典。

其返回结构为：

```
1 {  
2     "task_id": task_id,  
3     "sample_id": sample_id,  
4     "prediction": pred  
5 }
```

虽然最终提交文件中不会保留 `task_id` 字段，但该字段在内部结果列表中有助于保留任务上下文。

## 7. 多线程加速设计

代码使用 `ThreadPoolExecutor` 对测试集进行并行处理：

```
1 max_workers = 200
```

然后通过：

```
1 executor.map(process_func, eval_data)
```

并发处理所有测试样例。

本任务中每条样例之间完全独立，不存在共享状态修改，因此适合并行化处理。多线程可以减少大量测试样例逐条处理时的总耗时，尤其是在测试集规模较大时，能够明显提升输出效率。

同时，程序使用 `tqdm` 显示预测进度：

```
1 tqdm(..., total=total_cnt, desc="Predicting")
```

这样运行时可以直观看到处理进度，方便判断程序是否正常运行。

需要说明的是，本任务单条样例计算本身非常快，因此多线程主要起到批量调度和进度管理作用。在测试样例特别少的情况下，多线程收益可能不明显；但在大规模测试集场景中，该设计可以增强处理效率。

## 8. 输出文件生成

预测完成后，程序会将内部结果转换为最终提交格式：

```
1 output_data = [  
2     {  
3         "test_sample_id": item["sample_id"],  
4         "prediction": item["prediction"]  
5     }  
6     for item in results_list  
7 ]
```

最终每一行只保留两个字段：

```
1 test_sample_id  
2 prediction
```

然后程序使用：

```
1 os.makedirs(os.path.dirname(out_path), exist_ok=True)
```

确保输出目录存在，再逐行写入 JSONL 文件：

```
1 with open(out_path, 'w', encoding='utf-8') as f:  
2     for item in output_data:  
3         line = json.dumps(item, ensure_ascii=False)  
4         f.write(line + '\n')
```

使用 JSONL 格式的好处是每条预测结果独占一行，便于评测系统逐条读取，也便于后续调试和增量检查。

## 9. 异常处理与稳定性设计

本代码在多个关键位置加入了异常处理机制。

第一，API 调用阶段设置重试机制。模型调用失败不会立即终止程序，而是等待后重试。若最终失败，则返回空字符串。

第二，代码编译阶段捕获异常。如果模型生成代码不符合语法、安全检查失败或没有定义 `solve` 函数，程序会跳过该次生成，继续尝试下一次。

第三，样例验证阶段捕获运行异常。如果某条样例执行失败，程序会记录错误，而不是中断整体验证流程。

第四，测试集预测阶段也捕获异常。如果某个测试样例处理失败，程序会打印错误信息，并返回默认预测 `"0"`，保证最终 JSONL 文件仍可完整生成。

第五，整体求解器层面提供 fallback。若模型生成求解器不可用，程序会切换到内置确定性求解器，避免生成模型失败导致提交失败。

这些设计共同提高了程序在真实运行环境中的鲁棒性。



# OpenSeek-2 Count Nouns/Verbs 技术报告

## 1. 任务概述

本任务对应 OpenSeek-2 的 **Count Nouns/Verbs** 子任务，任务目标是在给定英文句子中统计名词或动词的数量。与普通语法题不同，本任务并不是简单按照传统英语语法规则进行名词、动词识别，而是要求尽可能匹配数据集自身的标注规则。

任务的输入通常是一条自然语言指令加一句英文图像描述，例如：

```
1 Count the number of nouns in this sentence: 'A man holding a baseball bat'
```

或者：

```
1 Count the number of verbs in this sentence: 'A man is using a cell phone to photograph a barn'
```

模型需要判断当前样例要求统计的是 nouns 还是 verbs，然后根据对应规则识别句子中应被计数的词项，最后输出数量。例如，在名词统计任务中：

```
1 Sentence: 'A man holding a baseball bat'
```

按照数据集规则，`man`、`baseball`、`bat` 均可能被计入名词单位，因此预测数量为 `3`。而在动词统计任务中：

```
1 Sentence: 'A man is using a cell phone to photograph a barn'
```

其中 `is` 作为助动词不计入，`using` 和 `photograph` 表示动作，应计入动词，因此预测数量为 `2`。

本代码的核心思路是：不直接让模型只输出一个数字，而是要求模型先进行逐词分析，输出被计入的名词或动词列表，再由程序统计该列表长度作为最终答案。这样可以提高预测过程的可解释性，也便于后续调试和错误分析。

## 2. 数据集结构与输入输出格式

代码通过 `task2_data_loader(file_path)` 读取 JSON 格式数据文件。数据文件主要包含以下字段：

```
1 task_id
2 task_name
3 Definition
4 examples
5 test_samples
```

各字段含义如下：

| 字段                        | 含义                                 |
|---------------------------|------------------------------------|
| <code>task_id</code>      | 当前任务 ID，例如 <code>openseek-2</code> |
| <code>task_name</code>    | 当前任务名称，例如 Count Nouns/Verbs        |
| <code>Definition</code>   | 任务定义说明                             |
| <code>examples</code>     | 训练样例，包含输入句子和标准答案                   |
| <code>test_samples</code> | 测试样例，只包含输入，需要生成预测结果                |

其中，`examples` 用于可选的训练集验证。每条样例通常包含：

```
1 {
2   "id": "sample_id",
3   "input": "Count the number of nouns in this sentence: '...'",
4   "output": ["5"]
5 }
```

`test_samples` 用于最终提交文件生成，每条测试样例通常包含：

```
1 {
2   "id": "test_sample_id",
3   "input": "Count the number of verbs in this sentence: '...'"
4 }
```

最终输出文件为 JSONL 格式，每行对应一个测试样例，格式为：

```
1 {"test_sample_id": "openseek-2-xxxx", "prediction": "5"}
```

其中：

- `test_sample_id` 来自测试样例中的 `id`；
- `prediction` 是程序预测出的名词或动词数量；
- 预测结果统一转换为字符串，符合提交格式要求。

### 3. 总体技术路线

本代码采用“模型逐词分析 + JSON 结构化解析 + 列表长度计数 + JSONL 输出”的整体流程。

完整流程如下：

```
1 读取任务 JSON 文件
2      ↓
3 获取 task_id / task_name / examples / test_samples
4      ↓
5 判断输入样例是 nouns 任务还是 verbs 任务
```

```
6      ↓
7  选择 NOUN_SYSTEM_PROMPT 或 VERB_SYSTEM_PROMPT
8      ↓
9  调用 Qwen3-4B 生成逐词分析 JSON
10     ↓
11 解析模型输出，提取 output 列表
12     ↓
13 用 output 列表长度作为预测数量
14     ↓
15 可选：在 examples 上验证准确率
16     ↓
17 对 test_samples 并行预测
18     ↓
19 保存 JSONL 提交文件
```

该流程的关键设计点在于：模型输出的不是最终数字，而是一个包含 `reason` 和 `output` 两个字段的 JSON 对象。例如：

```
1  {
2    "reason": "1. A->冠词，不计入；2. man->名词，表示可见人物，计入；3. holding->动词，现在分词表示动作，不按名词计入；4. a->冠词，不计入；5. baseball->名词，在 baseball bat 中按复合名词前项计入；6. bat->名词，表示可见物体，计入",
3    "output": ["man", "baseball", "bat"]
4  }
```

程序不会直接相信模型输出的数字，而是统计 `output` 数组长度。因此，只要模型能够正确列出应计入的名词或动词，程序就可以稳定生成最终数字。

## 4. 数据加载模块

代码中的数据加载函数如下：

```
1  def task2_data_loader(file_path):
2      with open(file_path, 'r', encoding='utf-8') as f:
3          data = json.load(f)
4      return (
5          data.get("task_id"),
6          data.get("task_name"),
7          data.get("Definition", []),
8          data.get("examples", []),
9          data.get("test_samples", [])
10     )
```

该函数主要完成以下工作：

1. 使用 `open` 打开本地 JSON 数据文件；
2. 使用 `json.load` 将文件内容解析为 Python 字典；
3. 分别读取 `task_id`、`task_name`、`Definition`、`examples` 和 `test_samples`；

4. 对缺失字段提供默认值，避免字段不存在时报错。

该模块是整个程序的入口。后续无论是训练集验证还是测试集预测，都依赖该函数提供的数据。

## 5. 模型调用模块

代码使用 OpenAI SDK 初始化了一个兼容 OpenAI API 格式的客户端：

```
1 client = OpenAI(  
2     api_key="dummy",  
3     base_url="https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1"  
4 )
```

模型调用函数为：

```
1 def qwen_api(messages, model="/Qwen3-4B/Qwen/Qwen3-4B", retries=3):  
2     for attempt in range(retries):  
3         try:  
4             res = client.chat.completions.create(  
5                 model=model,  
6                 messages=messages,  
7                 temperature=0.0,  
8             )  
9             return res.choices[0].message.content  
10        except Exception as e:  
11            if attempt == retries - 1:  
12                print(f"\nAPI 调用失败: {e}")  
13                return ""  
14            time.sleep(5)
```

该函数具有以下特点：

### 5.1 使用确定性生成参数

代码设置：

```
1 temperature=0.0
```

这表示模型输出尽可能稳定，减少随机性。对于分类、抽取和计数任务而言，稳定性比创造性更重要，因此低温度设置是合理的。

### 5.2 支持失败重试

函数设置：

```
1 retries=3
```



如果 API 调用失败，程序不会立刻终止，而是最多重试 3 次。每次失败后等待 5 秒再继续尝试。如果最后一次仍然失败，则返回空字符串。

## 5.3 输入采用 messages 格式

每次调用模型时，程序传入：

```
1 messages = [  
2     {"role": "system", "content": system_prompt},  
3     {"role": "user", "content": input_text}  
4 ]
```

其中 `system_prompt` 用于规定任务规则和输出格式，`input_text` 是当前样例的原始输入。

## 6. 系统提示词设计

本任务的核心难点在于，数据集的名词和动词计数规则并不完全等同于标准语法规则。因此，代码分别设计了两个较长的系统提示词：

```
1 NOUN_SYSTEM_PROMPT  
2 VERB_SYSTEM_PROMPT
```

程序会根据输入内容自动选择对应提示词。

## 7. 名词提示词设计

`NOUN_SYSTEM_PROMPT` 用于处理名词计数任务。它要求模型扮演一个 “dataset-aligned noun extractor”，也就是数据集对齐的名词抽取器。

该提示词明确说明：目标不是按照标准语法定义统计所有名词，而是按照图像描述数据集的标注习惯，统计句子中可见实体、物体、人物、动物、地点、场景部件等可数单位。

### 7.1 名词计数范围

提示词中将以下内容视为可计入名词：

| 类型      | 示例                                                       |
|---------|----------------------------------------------------------|
| 人物      | man, woman, boy, girl, child, people, person, player     |
| 动物      | dog, horse, elephant, bird, giraffe, zebra               |
| 物体      | bat, phone, chair, plate, suitcase, umbrella             |
| 地点/场景部件 | room, street, court, kitchen, beach, field, wall, window |

这种定义更接近图像标注任务中的“可见对象”或“场景元素”，而不是传统语法中的所有名词。

## 7.2 不计入名词的情况

提示词明确要求不统计以下类型：

| 类型         | 示例                                             |
|------------|------------------------------------------------|
| 代词或占位词     | it, he, she, they, this, that, something       |
| 抽象词        | idea, purpose, appearance, memory, life        |
| 图像描述元词     | image, picture, photo, view, scene, background |
| 纯形容词/颜色/状态 | small, large, red, white, empty, open          |
| 纯方向或位置词    | front, back, side, top, bottom, middle         |
| 表示动作的动词形式  | walking, standing, holding, riding, playing    |

例如：

```
1 | A person holding an umbrella in front of a building
```

其中：

- person 计入；
- holding 是动作，不作为名词计入；
- umbrella 计入；
- front 在 in front of 中是方位结构，不计入；
- building 计入。

最终名词列表为：

```
1 | ["person", "umbrella", "building"]
```

## 7.3 复合名词规则

提示词对复合名词进行了专门说明。部分复合名词会被拆分计数，例如：

| 复合结构         | 输出                  |
|--------------|---------------------|
| tennis court | ["tennis", "court"] |
| baseball bat | ["baseball", "bat"] |
| cell phone   | ["cell", "phone"]   |
| living room  | ["living", "room"]  |
| parking lot  | ["parking", "lot"]  |
| fire hydrant | ["fire", "hydrant"] |
| teddy bear   | ["teddy", "bear"]   |

但也有一些结构通常不拆分，例如：

| 结构               | 输出        |
|------------------|-----------|
| motor bike       | ["bike"]  |
| ski slope        | ["slope"] |
| swiss army knife | ["knife"] |
| soft drink       | ["drink"] |

这说明本任务的标注规则具有明显的数据集特定性，不能简单使用词性标注器或语法规则完成。

## 7.4 “of” 结构规则

提示词还规定了若干 of 结构通常需要同时计入前后两部分，例如：

| 短语               | 输出                   |
|------------------|----------------------|
| group of people  | ["group", "people"]  |
| bunch of bananas | ["bunch", "bananas"] |
| couple of cars   | ["couple", "cars"]   |
| pair of zebras   | ["pair", "zebras"]   |
| piece of cake    | ["piece", "cake"]    |
| slice of pizza   | ["slice", "pizza"]   |

这类规则同样体现出数据集对“可见单位”和“数量单位”的特殊处理方式。

## 8. 动词提示词设计

`VERB_SYSTEM_PROMPT` 用于处理动词计数任务。它要求模型扮演一个 “dataset-aligned verb extractor”，即数据集对齐的动词抽取器。

该提示词强调：动词统计也不是标准语法中的所有动词，而是统计描述动作、事件、过程或结果状态变化的词。

## 8.1 动词计数范围

提示词中将以下内容视为可计入动词：

| 类型          | 示例                                                                    |
|-------------|-----------------------------------------------------------------------|
| 动作动词        | walk, hold, ride, sit, play, eat, catch, make, take                   |
| 事件性 -ing 形式 | walking, holding, riding, sitting, standing, grazing                  |
| 结果/事件分词     | painted, decorated, displayed, canned, parked, filled, shown, chopped |

例如：

```
1 | Jars of food are being canned in boiling water
```

其中：

- `are` 是助动词，不计入；
- `being` 是助动词成分，不计入；
- `canned` 表示被处理的结果事件，计入；
- `boiling` 在此处修饰 `water`，不作为主要动词计入。

因此动词列表为：

```
1 | [ "canned" ]
```

## 8.2 不计入动词的情况

提示词明确要求不统计以下内容：

| 类型        | 示例                                                  |
|-----------|-----------------------------------------------------|
| 助动词       | is, are, was, were, am, be, been, being             |
| 介词/小品词    | on, in, at, with, near, from, to, into, over, under |
| 名词性词语     | jump, trick, game, rail, fun, shot                  |
| 普通形容词/状态词 | full, open, ready, bright, barefoot                 |

例如：

```
1 | A skateboarder hitting a trick on a ramp
```

其中：

- `hitting` 是动作动词，计入；
- `trick` 是名词宾语，不计入动词；
- `on` 是介词，不计入。

最终动词列表为：

```
1 | ["hitting"]
```

## 8.3 上下文判断规则

动词提示词强调同一个词在不同上下文中可能有不同词性。例如：

```
1 | doing a jump
```

其中：

- `doing` 是动词，计入；
- `jump` 在这里是名词宾语，不计入动词。

而在其他语境中，`jump` 也可能作为动词。因此模型必须根据句子上下文逐词判断，而不能只依据词表匹配。

## 9. 输出格式约束

两个系统提示词都要求模型只输出 JSON 对象，且必须包含两个字段：

```
1 | {  
2 |   "reason": "逐词分析过程",  
3 |   "output": ["word1", "word2"]  
4 | }
```

其中：

- `reason` 是中文逐词分析过程；
- `output` 是被计入的名词或动词列表。

提示词还规定：

1. 只输出合法 JSON；
2. 不允许输出 Markdown；
3. 不允许输出 JSON 之外的解释；
4. `reason` 必须逐词分析；
5. `output` 必须是字符串数组；

6. 不能跳过句子中的词；
7. 不能只写简短总结。

这种格式约束有两个作用：

第一，提升输出可解析性。程序后续可以直接读取 `output` 字段并计算长度。

第二，提升模型推理稳定性。逐词分析要求模型显式判断每个词是否计入，减少漏判和误判。

---

## 10. 模型输出解析模块

由于大模型输出不一定完全稳定，代码设计了 `parse_json_output(raw_output)` 函数对模型输出进行多层解析。

该函数的解析策略分为三步。

### 10.1 直接 JSON 解析

程序首先尝试：

```
1 data = json.loads(raw_output)
```

如果模型输出本身就是合法 JSON，并且包含 `reason` 和 `output` 字段，则直接提取结果。

其中，`output` 字段会被清洗为字符串列表：

```
1 cleaned = [str(x).strip() for x in output if str(x).strip()]
```

这样可以去掉空字符串或格式异常的元素。

### 10.2 从文本中提取 JSON 块

如果直接 JSON 解析失败，程序会尝试用正则表达式从文本中抓取 JSON 块：

```
1 match = re.search(r'\{.*\}', raw_output, flags=re.S)
```

这种情况适用于模型输出了 JSON 之外的少量额外文字，例如：

```
1 下面是结果：
2 {"reason": "...", "output": ["man", "bat"]}
```

程序会提取 `{...}` 部分并再次尝试 `json.loads`。

### 10.3 兼容只返回列表的情况

如果前两步都失败，程序最后尝试：

```
1 items = ast.literal_eval(raw_output)
```

这用于兼容模型偶尔只返回列表的情况，例如：

```
1 ["man", "baseball", "bat"]
```

如果解析结果是列表，则程序仍然可以把它当成 `output` 使用，只是 `reason` 字段为空。

## 10.4 完全失败时返回空结果

如果所有解析方式都失败，函数返回：

```
1 {"reason": "", "output": []}
```

此时后续预测数量会变成 0。这种处理方式可以保证单条样例解析失败时不会中断整个测试集预测流程。

## 11. 训练样例处理模块

训练样例处理函数为：

```
1 def process_single_example(sample, noun_system_prompt, verb_system_prompt, task_id,
2   model_name):
3     sample_id = sample.get("id", "")
4     input_text = sample["input"]
5     gt = str(sample["output"][0]).strip()
6
7     lowered = input_text.lower()
8     if "count the number of nouns" in lowered:
9         system_prompt = noun_system_prompt
10        task_type = "nouns"
11    elif "count the number of verbs" in lowered:
12        system_prompt = verb_system_prompt
13        task_type = "verbs"
14    else:
15        system_prompt = noun_system_prompt
16        task_type = "unknown_default_nouns"
17
18    messages = [
19        {"role": "system", "content": system_prompt},
20        {"role": "user", "content": input_text}
21    ]
22
23    raw_output = qwen_api(messages, model=model_name)
24    parsed_items, parsed_reason = parse_output_items(raw_output)
25    pred = str(len(parsed_items))
26    is_correct = (pred == gt)
```

该函数主要完成以下步骤：

1. 读取样例 ID；

2. 读取输入句子；
3. 读取标准答案；
4. 判断当前任务是 nouns 还是 verbs；
5. 选择对应系统提示词；
6. 调用模型；
7. 解析模型输出；
8. 统计 `output` 列表长度作为预测值；
9. 与标准答案比较，得到是否正确。

返回结果中包含：

```
1 {
2     "task_id": task_id,
3     "sample_id": sample_id,
4     "input": input_text,
5     "task_type": task_type,
6     "gt": gt,
7     "model_output": pred,
8     "parsed_items": parsed_items,
9     "parsed_reason": parsed_reason,
10    "raw_output": raw_output,
11    "is_correct": is_correct
12 }
```

该结构非常适合调试，因为它不仅保存最终预测结果，还保存了模型原始输出、解析后的词列表和逐词分析理由。

## 12. 测试样例处理模块

测试样例处理函数为：

```
1 def process_single_test(sample, noun_system_prompt, verb_system_prompt, task_id,
2   model_name):
3     sample_id = sample.get("id", "")
4     input_text = sample["input"]
5
6     lowered = input_text.lower()
7     if "count the number of nouns" in lowered:
8         system_prompt = noun_system_prompt
9         task_type = "nouns"
10    elif "count the number of verbs" in lowered:
11        system_prompt = verb_system_prompt
12        task_type = "verbs"
13    else:
14        system_prompt = noun_system_prompt
15        task_type = "unknown_default_nouns"
```



```

16     messages = [
17         {"role": "system", "content": system_prompt},
18         {"role": "user", "content": input_text}
19     ]
20
21     raw_output = qwen_api(messages, model=model_name)
22     items, parsed_reason = parse_output_items(raw_output)
23     pred = len(items)

```

该函数与训练样例处理函数类似，但测试样例没有标准答案，因此不会计算 `is_correct`。它返回：

```

1  {
2      "task_id": task_id,
3      "sample_id": sample_id,
4      "input": input_text,
5      "task_type": task_type,
6      "model_output": pred,
7      "parsed_items": items,
8      "parsed_reason": parsed_reason,
9      "raw_output": raw_output
10 }

```

其中 `model_output` 是整数形式的预测数量，最终写入 JSONL 文件时会转换为字符串。

## 13. 任务类型判断机制

代码通过输入文本中的关键词判断当前任务类型：

```

1  lowered = input_text.lower()
2  if "count the number of nouns" in lowered:
3      system_prompt = noun_system_prompt
4      task_type = "nouns"
5  elif "count the number of verbs" in lowered:
6      system_prompt = verb_system_prompt
7      task_type = "verbs"
8  else:
9      system_prompt = noun_system_prompt
10     task_type = "unknown_default_nouns"

```

该设计简单直接，适用于当前数据集格式较固定的情况。

如果输入中包含：

```

1  count the number of nouns

```

则使用名词提示词。

如果输入中包含：

```
1 | count the number of verbs
```

则使用动词提示词。

如果两者都不包含，则默认按照名词任务处理，并标记为：

```
1 | unknown_default_nouns
```

该默认策略可以避免程序报错，但也存在一定风险。如果数据集中出现不规范输入，默认名词处理可能导致预测错误。因此后续可以考虑增加更严格的异常提示或任务类型检测逻辑。

## 14. 并行处理设计

代码使用 `ThreadPoolExecutor` 对 `examples` 或 `test_samples` 进行并行处理。主程序中设置：

```
1 | max_workers = 200
```

测试集预测阶段使用：

```
1 | with ThreadPoolExecutor(max_workers=max_workers) as executor:
2 |     for result in tqdm(executor.map(test_func, test_samples), total=test_total,
   |     desc="Predicting test"):
3 |         test_results.append(result)
```

并行处理的优势在于：

1. 每条样例之间相互独立；
2. 每条样例都需要调用模型接口；
3. API 调用通常存在等待时间；
4. 多线程可以提高整体吞吐量；
5. `tqdm` 可以实时显示处理进度。

对于本任务来说，单条样例的处理主要耗时来自模型 API 调用，而不是本地计算。因此多线程可以有效提高测试集预测效率。

不过，`max_workers=200` 也可能带来潜在问题。例如：

- 并发过高可能触发接口限流；
- 本地线程调度开销增加；
- 服务端负载过大时可能导致失败率提高。

因此，在实际运行中，可以根据接口稳定性将其调整为更保守的数值，例如 32、64 或 100。

## 15. 训练集验证流程

代码中保留了训练集验证逻辑，但当前被关闭：

```
1 | if False:
2 |     ...
```

如果将其改为：

```
1 | if True:
```

则程序会对 `examples` 中的所有训练样例进行并行预测，并统计准确率。

训练集验证流程如下：

```
1 | 读取 examples
2 |     ↓
3 | 并行调用 process_single_example
4 |     ↓
5 | 统计 correct_cnt
6 |     ↓
7 | 计算 accuracy
8 |     ↓
9 | 保存错误样本 wrong_cases
10 |    ↓
11 | 保存完整验证结果 JSON
```

验证结果中包含：

```
1 | eval_output_data = {
2 |     "task_id": task_id,
3 |     "task_name": task_name,
4 |     "evaluate_time": ...,
5 |     "model_name": model_name,
6 |     "prompt": NOUN_SYSTEM_PROMPT + VERB_SYSTEM_PROMPT,
7 |     "examples_total": total_cnt,
8 |     "examples_correct": correct_cnt,
9 |     "examples_accuracy": accuracy,
10 |    "wrong_cases": wrong_cases,
11 |    "all_results": eval_results
12 | }
```

该结构能够支持后续详细分析：

- 哪些样例预测错误；
- 错误样例属于名词任务还是动词任务；
- 模型输出了哪些词；
- 模型逐词分析理由是什么；
- 是模型判断错了，还是解析失败。

这部分虽然当前没有启用，但对于 prompt 调优非常重要。通过分析 `wrong_cases`，可以不断补充系统提示词中的规则和 few-shot 示例，从而提高最终测试集表现。

## 16. 测试集预测流程

当前代码中测试集预测逻辑处于开启状态：

```
1 if True:
2     ...
```

测试集预测流程如下：

```
1 读取 test_samples
2      ↓
3 构造 test_func
4      ↓
5 多线程调用 process_single_test
6      ↓
7 得到 test_results
8      ↓
9 转换为提交格式 output_data
10     ↓
11 写入 JSONL 文件
```

程序将每条测试结果转换为：

```
1 {
2     "test_sample_id": item["sample_id"],
3     "prediction": str(item["model_output"])
4 }
```

然后逐行写入目标文件：

```
1 with open(test_jsonl_path, 'w', encoding='utf-8') as f:
2     for item in output_data:
3         line = json.dumps(item, ensure_ascii=False)
4         f.write(line + '\n')
```

最终生成的 JSONL 示例为：

```
1 {"test_sample_id": "openseek-2-f907968b5d6340e8affa8452ea2b6f94", "prediction": "5"}
```

该格式符合常见比赛提交要求，也便于评测脚本逐行读取。

## 17. 异常处理与鲁棒性设计

本代码在多个位置加入了容错机制。

## 17.1 API 调用容错

`qwen_api` 支持最多 3 次重试。如果接口暂时失败，程序会等待 5 秒后再次尝试。最终失败时返回空字符串，而不是直接抛出异常终止程序。

## 17.2 JSON 解析容错

`parse_json_output` 支持三种解析方式：

1. 直接 JSON 解析；
2. 正则提取 JSON 块后解析；
3. 使用 `ast.literal_eval` 兼容列表输出。

这使得程序可以处理模型输出中的轻微格式偏差。

## 17.3 空输出容错

如果模型没有返回有效内容，解析函数返回：

```
1 | {"reason": "", "output": []}
```

此时预测数量为 0。虽然这不一定正确，但可以保证整个测试集预测流程不中断。

## 17.4 未知任务类型默认处理

当输入中无法识别 nouns 或 verbs 时，程序默认使用名词提示词：

```
1 | task_type = "unknown_default_nouns"
```

这可以避免异常，但也应在后续日志中重点检查。

# OpenSeek-3 Collatz Conjecture 技术报告

## 1. 任务概述

本任务对应 OpenSeek-3 的 **Collatz Conjecture** 子任务。任务要求对输入整数列表中的每一个整数执行一次 Collatz 变换，并输出变换后的整数列表。

Collatz 变换规则如下：

- 如果当前整数是偶数，则将其除以 2；
- 如果当前整数是奇数，则将其乘以 3 后再加 1。

也就是说，对于任意整数  $x$ ，其转换结果为：

```
1 | if x % 2 == 0:  
2 |     new_x = x // 2  
3 | else:  
4 |     new_x = 3 * x + 1
```

例如，输入为：

```
1 | [72, 29, 49]
```

逐个元素执行转换：

```
1 | 72 是偶数，因此  $72 / 2 = 36$   
2 | 29 是奇数，因此  $29 * 3 + 1 = 88$   
3 | 49 是奇数，因此  $49 * 3 + 1 = 148$ 
```

最终输出为：

```
1 | [36, 88, 148]
```

需要注意的是，本任务并不是要求持续执行 Collatz 序列直到到达 1，而是只对列表中的每个元素执行 **一步 Collatz 变换**。代码实现中也是按照这一点进行设计的。

## 2. 数据集结构与输入输出格式

本任务的数据文件为 JSON 格式。代码通过 `load_task(file_path)` 函数读取数据，并从 JSON 文件中解析出任务信息、训练样例和测试样例。

数据文件主要包含以下字段：

|   |              |
|---|--------------|
| 1 | task_id      |
| 2 | task_name    |
| 3 | Definition   |
| 4 | examples     |
| 5 | test_samples |

各字段含义如下：

| 字段           | 说明                                        |
|--------------|-------------------------------------------|
| task_id      | 当前任务编号，例如 <code>openseek-3</code>         |
| task_name    | 当前任务名称，例如 <code>Collatz Conjecture</code> |
| Definition   | 任务自然语言定义，说明 Collatz 转换规则                  |
| examples     | 训练样例，包含输入和标准输出                            |
| test_samples | 测试样例，只包含输入，需要生成预测结果                       |

训练样例通常具有如下结构：

|   |                           |
|---|---------------------------|
| 1 | {                         |
| 2 | " id": "sample_id",       |
| 3 | "input": "[72, 29, 49]",  |
| 4 | "output": "[36, 88, 148]" |
| 5 | }                         |

测试样例通常具有如下结构：

|   |                          |
|---|--------------------------|
| 1 | {                        |
| 2 | " id": "test_sample_id", |
| 3 | "input": "[72, 29, 49]"  |
| 4 | }                        |

程序最终会生成 JSONL 格式的提交文件，每一行对应一个测试样例：

|   |                                                                        |
|---|------------------------------------------------------------------------|
| 1 | { "test_sample_id": "openseek-3-xxxx", "prediction": "[36, 88, 148]" } |
|---|------------------------------------------------------------------------|

其中：

- `test_sample_id` 来自测试样例中的 `id` 字段；
- `prediction` 是程序计算出的转换后列表；
- `prediction` 需要以字符串形式保存。

### 3. 总体设计思路

本代码采用“模型生成求解器 + 自动验证 + 失败回退 + 批量预测 + 结果保存”的整体流程。

与直接手写固定求解函数不同，本代码首先根据任务定义和样例构造提示词，让 Qwen3-4B 模型生成一个符合要求的 Python `solve(input_text: str) -> str` 函数。之后程序会对生成函数进行编译和验证。如果模型生成的函数在训练样例上能够达到要求，则使用该函数处理测试集；如果模型生成失败或验证效果不理想，则回退到内置的保底求解器。

整体流程如下：

```
1  读取任务 JSON 文件
2      ↓
3  获取 task_id / task_name / Definition / examples / test_samples
4      ↓
5  构造代码生成 Prompt
6      ↓
7  调用 Qwen3-4B 生成 solve 函数代码
8      ↓
9  提取 Python 代码块
10     ↓
11 使用 exec 编译生成代码
12     ↓
13 在 examples 上验证生成 solver 的准确率
14     ↓
15 若准确率为 100%，使用生成 solver
16 若多次尝试仍失败，则回退到 fallback solver
17     ↓
18 对训练集进行完整评估
19     ↓
20 对测试集进行预测
21     ↓
22 保存完整 JSON 结果文件
23     ↓
24 保存比赛提交 JSONL 文件
```

该设计的核心目的是提高自动化程度。模型负责根据任务描述生成求解代码，程序负责对生成代码进行验证、筛选和执行，避免直接信任模型输出。

## 4. 核心算法原理

本任务的核心算法非常明确：对输入列表中的每个整数执行一次 Collatz 转换。

假设输入列表为：

```
1  nums = [x1, x2, x3, ..., xn]
```

输出列表为：

```
1  result = []
```



对每个元素 `x` 执行：

```
1 if x % 2 == 0:
2     result.append(x // 2)
3 else:
4     result.append(x * 3 + 1)
```

最终返回：

```
1 str(result)
```

## 4.1 偶数处理

当 `x` 是偶数时，使用整除：

```
1 x // 2
```

例如：

```
1 72 -> 36
2 10 -> 5
3 4 -> 2
```

这里使用 `//` 而不是 `/`，是因为输出需要保持整数。如果使用 `/`，Python 会返回浮点数，例如 `72 / 2 = 36.0`，这会导致输出格式和标准答案不一致。

## 4.2 奇数处理

当 `x` 是奇数时，执行：

```
1 x * 3 + 1
```

例如：

```
1 29 -> 88
2 49 -> 148
3 7 -> 22
```

## 4.3 复杂度分析

设输入列表长度为 `n`。

程序只需要遍历列表一次，因此时间复杂度为：

```
1 O(n)
```

程序需要保存输出列表，因此空间复杂度为：

该算法是本任务的最优解之一，因为必须至少读取每个输入元素一次。

## 5. 数据加载模块

代码中的数据加载函数如下：

```
1 def load_task(file_path: str) -> Tuple[str, str, List[str], List[Dict[str, Any]],  
  List[Dict[str, Any]]]:  
2     with open(file_path, 'r', encoding='utf-8') as f:  
3         data = json.load(f)  
4     return (  
5         data.get("task_id", ""),  
6         data.get("task_name", ""),  
7         data.get("Definition", []),  
8         data.get("examples", []),  
9         data.get("test_samples", []),  
10    )
```

该函数完成以下工作：

1. 打开本地 JSON 数据文件；
2. 使用 `json.load` 解析 JSON 内容；
3. 读取任务 ID；
4. 读取任务名称；
5. 读取任务定义；
6. 读取训练样例；
7. 读取测试样例；
8. 对缺失字段提供默认值。

该函数返回五个对象：

```
1 task_id, task_name, definition, examples, test_samples
```

主程序后续的所有流程都依赖这些数据，包括 prompt 构造、solver 生成、solver 验证、测试集预测和结果保存。

## 6. 模型 API 调用模块

代码使用 OpenAI SDK 创建了一个兼容 OpenAI 接口格式的客户端：

```

1 client = OpenAI(
2     api_key="dummy",
3     base_url="https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1",
4 )

```

模型调用函数为：

```

1 def qwen_api(messages: List[Dict[str, str]], model: str = "/Qwen3-4B/Qwen/Qwen3-4B",
2   retries: int = 3) -> str:
3     for attempt in range(retries):
4         try:
5             res = client.chat.completions.create(
6                 model=model,
7                 messages=messages,
8                 temperature=0.0,
9             )
10            return res.choices[0].message.content or ""
11        except Exception as e:
12            if attempt == retries - 1:
13                print(f"\nAPI 调用失败: {e}")
14                return ""
15            time.sleep(2)
16    return ""

```

该函数具有以下特点。

## 6.1 使用 Qwen3-4B 模型

代码默认调用：

```

1 /Qwen3-4B/Qwen/Qwen3-4B

```

该模型用于根据任务说明和样例自动生成 Python 求解函数。

## 6.2 使用低温度生成

代码设置：

```

1 temperature=0.0

```

这表示尽量减少模型输出随机性。对于代码生成任务而言，稳定性比创造性更重要，因此 `temperature=0.0` 是合理设置。

## 6.3 支持重试机制

函数设置：

```

1 retries=3

```

如果模型接口调用失败，程序最多尝试三次。前两次失败后等待 2 秒继续重试；如果最后一次仍失败，则打印错误信息并返回空字符串。

这种设计可以应对临时网络波动或服务端响应失败，增强程序运行稳定性。

## 7. Python 代码块提取模块

模型生成代码时，有时会直接返回纯代码，有时会返回 Markdown 代码块，例如：

```
1  ```python
2  def solve(input_text: str) -> str:
3      ...
4  ```
```

为了兼容这两种情况，代码定义了正则表达式：

```
1  CODE_BLOCK_PATTERN = re.compile(
2      r"```python\s*(.*?)```|```\s*(.*?)```",
3      re.DOTALL | re.IGNORECASE
4  )
```

并定义代码提取函数：

```
1  def extract_code(text: str) -> str:
2      if not text:
3          return ""
4      match = CODE_BLOCK_PATTERN.search(text)
5      if match:
6          return (match.group(1) or match.group(2) or "").strip()
7      return text.strip()
```

该函数逻辑如下：

1. 如果模型输出为空，直接返回空字符串；
2. 如果模型输出中包含 ```python 代码块，则提取代码块内容；
3. 如果模型输出中包含普通 ``` 代码块，也提取其中内容；
4. 如果没有代码块，则将完整文本作为代码返回。

这一步可以提高模型输出解析的容错性，使程序不依赖模型完全按照纯代码格式输出。

## 8. 保底求解器设计

代码定义了 `fallback_solver_code()` 函数，用于在模型生成代码失败时提供内置求解器。

保底代码如下：

```

1 def fallback_solver_code() -> str:
2     return ''
3 def solve(input_text: str) -> str:
4     nums = ast.literal_eval(input_text.strip())
5     result = []
6     for x in nums:
7         if x % 2 == 0:
8             result.append(x // 2)
9         else:
10            result.append(x * 3 + 1)
11    return str(result)
12    ''.strip()

```

该保底求解器的逻辑非常直接：

1. 使用 `ast.literal_eval` 将输入字符串解析为 Python 列表；
2. 创建空列表 `result`；
3. 遍历输入列表中的每个整数；
4. 如果整数为偶数，则加入 `x // 2`；
5. 如果整数为奇数，则加入 `x * 3 + 1`；
6. 返回 `str(result)`。

## 8.1 fallback 的意义

模型生成代码存在一定不稳定性，可能出现以下问题：

- 生成结果不是完整 Python 代码；
- 没有定义 `solve` 函数；
- 输出格式不符合要求；
- 逻辑错误；
- 编译失败；
- 运行时报错；
- 在训练样例上准确率不足。

因此，fallback 求解器可以作为最后保障，确保即使模型生成失败，程序仍然可以继续完成测试集预测。

## 8.2 当前代码中的潜在问题

当前 `fallback_solver_code()` 中使用了：

```
1 ast.literal_eval(input_text.strip())
```

但是 `compile_solver` 函数执行代码时使用的是：

```
1 namespace: Dict[str, Any] = {}
2 exec(code, namespace)
```

如果 fallback 代码本身没有写入 `import ast`，并且 `namespace` 中也没有提前放入 `ast`，那么 fallback 的 `solve` 函数在实际调用时可能会出现：

```
1 NameError: name 'ast' is not defined
```

因此，更稳妥的做法有两种。

第一种是在 fallback 代码中加入：

```
1 import ast
```

第二种是在 `compile_solver` 中提供 `ast`：

```
1 def compile_solver(code: str) -> Callable[[str], str]:
2     namespace: Dict[str, Any] = {"ast": ast}
3     exec(code, namespace)
4     if "solve" not in namespace:
5         raise ValueError("生成代码中未定义 solve")
6     return namespace["solve"]
```

从代码可靠性角度看，第二种方式更统一，因为模型生成代码和 fallback 代码都可以共享外部注入的 `ast` 对象。

## 9. Prompt 构造模块

代码通过 `build_code_generation_prompt` 函数构造用于代码生成的用户提示词：

```
1 def build_code_generation_prompt(task_name: str, definition: List[str], examples:
2   List[Dict[str, Any]]) -> str:
3     demo_examples = examples[:10]
4     examples_text = "\n".join(f"输入: {ex['input']}\n输出: {ex['output']}" for ex
5   in demo_examples)
6     return f'''
7     你是 Python 算法工程师。
8     请根据任务定义和样例生成可执行 Python 代码。
9
10    任务名: {task_name}
11    任务定义:
12    {chr(10).join(definition)}
13
14    样例:
15    {examples_text}
16
17    严格要求:
18    1. 定义函数 solve(input_text: str) -> str
```

```
17 2. input_text 是完整字符串，例如 "[1,2,3]"
18 3. 可以使用 import 或标准库
19 4. 用 ast.literal_eval 解析输入
20 5. 返回值必须是字符串
21 6. 生成代码必须能被 exec 直接执行
22 '''
```

该提示词包含以下信息：

1. 模型角色：Python 算法工程师；
2. 任务名称；
3. 任务定义；
4. 前 10 条训练样例；
5. 严格输出要求。

## 9.1 使用样例进行上下文学习

代码选择：

```
1 demo_examples = examples[:10]
```

也就是说，使用前 10 条 examples 作为示例，帮助模型理解输入输出格式和转换规则。

对于本任务来说，规则较简单，10 条样例通常足以让模型归纳出：输入是整数列表字符串，输出是执行一次 Collatz 变换后的列表字符串。

## 9.2 明确 solve 函数接口

提示词要求生成代码必须定义：

```
1 solve(input_text: str) -> str
```

这一接口是后续自动编译、验证和批量预测的基础。如果模型没有生成该函数，程序就无法统一调用。

## 9.3 允许使用标准库

提示词允许：

```
1 可以使用 import 或标准库
```

这使模型可以生成 `import ast` 等语句，从而正确解析输入字符串。但与此同时，这也意味着安全限制较弱，具体问题会在后续“安全性与风险分析”中说明。

# 10. 编译 solver 模块

代码通过 `compile_solver` 函数将模型生成的代码字符串转换为可调用的 Python 函数：

```

1 def compile_solver(code: str) -> Callable[[str], str]:
2     namespace: Dict[str, Any] = {}
3     exec(code, namespace)
4     if "solve" not in namespace:
5         raise ValueError("生成代码中未定义 solve")
6     return namespace["solve"]

```

该函数执行流程如下：

1. 创建空命名空间 `namespace`；
2. 使用 `exec(code, namespace)` 执行生成代码；
3. 检查命名空间中是否存在 `solve`；
4. 如果不存在，则抛出异常；
5. 如果存在，则返回 `solve` 函数对象。

## 10.1 动态执行的作用

动态执行使程序可以将模型生成的文本代码变成真正可调用的函数。这样后续验证和预测阶段就可以统一调用：

```

1 solve_func(sample["input"])

```

## 10.2 动态执行的风险

当前代码中写明“取消安全限制”，并直接执行模型生成代码：

```

1 exec(code, namespace)

```

这在受控比赛环境中可能可以接受，但从工程安全角度看存在风险。如果模型生成了文件操作、系统命令、网络请求等代码，就可能造成安全问题。

更安全的做法包括：

- 禁止危险 import；
- 限制 `__builtins__`；
- 使用 AST 分析检查代码结构；
- 只允许定义 `solve` 函数；
- 禁止 `open`、`exec`、`eval`、`compile`、`os`、`subprocess` 等危险操作；
- 在隔离环境中执行生成代码。

## 11. solver 验证模块

代码使用 `validate_solver` 函数验证生成的求解器是否正确：



```

1 def validate_solver(solve_func: Callable[[str], str], examples: List[Dict[str, Any]],
2   limit: int = 100) -> Tuple[float, List[Dict[str, Any]]]:
3     total = min(limit, len(examples))
4     errors: List[Dict[str, Any]] = []
5     correct = 0
6     for sample in examples[:total]:
7         try:
8             pred = str(solve_func(sample["input"])).strip()
9         except Exception as e:
10            pred = f"<runtime_error: {e}>"
11        gt = str(sample["output"][0]).strip()
12        if pred == gt:
13            correct += 1
14        else:
15            errors.append({"id": sample.get("id", ""), "input": sample["input"], "gt":
16                          gt, "pred": pred})
17    return correct / total if total else 0.0, errors

```

该函数的主要功能是：

1. 从 examples 中取前 `limit` 条样例；
2. 调用 `solve_func` 生成预测；
3. 与标准答案进行字符串比较；
4. 统计正确数量；
5. 收集错误样例；
6. 返回准确率和错误列表。

## 11.1 字符串级比较

代码使用：

```

1 pred = str(solve_func(sample["input"])).strip()
2 gt = str(sample["output"][0]).strip()

```

然后判断：

```

1 pred == gt

```

这意味着输出格式必须严格一致。例如：

```

1 [36, 88, 148]

```

和：

```

1 [36,88,148]

```

在字符串层面并不相等。虽然两者表达的列表内容相同，但如果标准答案包含空格，而预测结果没有空格，就可能被判为错误。

当前 fallback 使用 `str(result)`，Python 列表默认格式会包含逗号和空格，通常与数据集输出格式一致。因此该做法在本任务中是合理的。

## 11.2 运行时异常处理

如果 `solve_func` 在某条样例上运行出错，程序不会中断，而是将预测记录为：

```
1 <runtime_error: ...>
```

并将该样例加入错误列表。这有助于后续分析错误原因。

## 12. 自动生成 solver 模块

代码中的核心生成函数为 `generate_valid_solver`：

```
1 def generate_valid_solver(task_name: str, definition: List[str], examples:
  List[Dict[str, Any]], max_attempts: int = 3):
2     last_code = ""
3     last_errors = []
4     for attempt in range(1, max_attempts + 1):
5         ...
```

该函数最多尝试 3 次生成 solver。

### 12.1 第一次生成

第一次尝试时，程序调用：

```
1 prompt = build_code_generation_prompt(task_name, definition, examples)
```

该 prompt 包含任务定义和前 10 条样例。模型根据这些信息生成完整 Python 代码。

消息格式如下：

```
1 messages = [
2     {"role": "system", "content": "你是严格输出可执行 Python 代码的助手。"},
3     {"role": "user", "content": prompt}
4 ]
```

之后程序调用：

```
1 raw_code = qwen_api(messages, model="/Qwen3-4B/Qwen/Qwen3-4B")
2 code = extract_code(raw_code)
```

提取出 Python 代码。

## 12.2 编译与验证

生成代码提取后，程序尝试编译：

```
1 solve_func = compile_solver(code)
```

然后在训练样例上验证：

```
1 acc, errors = validate_solver(solve_func, examples, limit=min(200, len(examples)))
```

如果准确率达到 100%，立即返回：

```
1 return code, solve_func, acc, errors
```

## 12.3 基于错误样例的再次生成

如果第一次生成的函数没有达到 100% 准确率，程序会记录错误样例，并在下一次 prompt 中加入错误反馈：

```
1 error_text = "\n".join(  
2     f"输入: {e['input']} | 正确输出: {e['gt']} | 你的输出: {e['pred']}"  
3     for e in last_errors[:10]  
4 )
```

新的 prompt 会告诉模型：

```
1 你上一次生成的 solve 函数在以下样例上出错：  
2 ...  
3 请重新生成完整 Python 代码，并修正逻辑。
```

这种设计相当于一个简单的自我修正流程。模型不是每次从零开始生成，而是根据上一轮错误案例修复逻辑。

## 12.4 回退机制

如果多次尝试后仍然失败，程序输出：

```
1 回退到保底求解器。
```

然后使用：

```
1 last_code = fallback_solver_code()  
2 solve_func = compile_solver(last_code)
```

最后再次验证 fallback solver，并返回结果。

---

## 13. 预测模块

代码使用 `run_predictions` 函数对训练集或测试集执行预测：

```
1 def run_predictions(solve_func: Callable[[str], str], samples: List[Dict[str, Any]],
2   has_label: bool=False):
3     results = []
4     correct = 0
5     for s in tqdm(samples, desc="Running solver"):
6         try:
7             pred = str(solve_func(s["input"])).strip()
8         except Exception as e:
9             pred = f"<runtime_error: {e}>"
10            item = {"test_sample_id": s.get("id", ""), "prediction": pred}
11            if has_label:
12                gt = str(s["output"][0]).strip()
13                item["gt"] = gt
14                item["is_correct"] = pred == gt
15                if item["is_correct"]:
16                    correct += 1
17            results.append(item)
18    output = {"total": len(samples), "details": results}
19    if has_label:
20        output["correct"] = correct
21        output["accuracy"] = correct / len(samples) if samples else 0.0
22    return output
```

该函数可以同时用于训练集和测试集。

## 13.1 用于训练集

当 `has_label=True` 时，函数会读取标准答案，并计算：

```
1 item["is_correct"] = pred == gt
```

最终返回：

```
1 {
2     "total": 样例总数,
3     "details": 每条样例预测详情,
4     "correct": 正确数量,
5     "accuracy": 准确率
6 }
```

## 13.2 用于测试集

当 `has_label=False` 时，函数只保存：

```

1  {
2      "test_sample_id": 样例 ID,
3      "prediction": 预测结果
4  }

```

不会计算准确率。

## 13.3 使用 tqdm 显示进度

函数使用：

```

1  tqdm(samples, desc="Running solver")

```

显示处理进度，方便运行时观察程序是否正常推进。

## 14. 主流程分析

主程序位于：

```

1  if __name__ == "__main__":

```

其中定义了三个路径：

```

1  file_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
    Annotation\data\openseek-3_collatz_conjecture.json"
2  out_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
    Annotation\rgs_q3\experiment\openseek3_collatz_results.json"
3  test_jsonl_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
    Annotation\experiment\openseek-3-v1.jsonl"

```

三个路径分别表示：

| 路径变量                         | 作用                   |
|------------------------------|----------------------|
| <code>file_path</code>       | 输入数据集 JSON 文件路径      |
| <code>out_path</code>        | 保存完整评估结果的 JSON 文件路径  |
| <code>test_jsonl_path</code> | 保存最终比赛提交文件的 JSONL 路径 |

### 14.1 加载数据

主流程首先执行：

```

1  task_id, task_name, definition, examples, test_samples = load_task(file_path)

```

然后打印任务信息：

```
1 print(f"任务: {task_id} / {task_name}")
2 print(f"训练样例数: {len(examples)}, 测试样例数: {len(test_samples)}")
```

## 14.2 生成 solver

随后调用：

```
1 generated_code, solve_func, preview_acc, preview_errors = generate_valid_solver(
2     task_name, definition, examples, max_attempts=3
3 )
```

该步骤会自动完成代码生成、编译、验证和 fallback。

## 14.3 训练集完整评估

程序对 examples 进行完整评估：

```
1 example_eval = run_predictions(solve_func, examples, has_label=True)
```

这一步可以得到完整训练集准确率。

## 14.4 测试集预测

程序对 test\_samples 进行预测：

```
1 test_eval = run_predictions(solve_func, test_samples, has_label=False)
```

得到每条测试样例的预测结果。

## 14.5 保存完整 JSON 结果

程序将生成代码、预验证准确率、错误样例、训练集评估结果和测试集预测结果保存到 `out_path`：

```
1 output_data = {
2     "task_id": task_id,
3     "task_name": task_name,
4     "evaluate_time": time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()),
5     "generated_code": generated_code,
6     "preview_accuracy_on_examples": preview_acc,
7     "preview_errors": preview_errors[:20],
8     "examples_eval": example_eval,
9     "test_eval": test_eval
10 }
```

该 JSON 文件适合用于后续复盘和调试。

## 14.6 保存 JSONL 提交文件

最后，程序将测试集预测结果保存为 JSONL：

```
1 os.makedirs(os.path.dirname(test_jsonl_path), exist_ok=True)
2 with open(test_jsonl_path, 'w', encoding='utf-8') as f:
3     for item in test_eval["details"]:
4         line = json.dumps(item, ensure_ascii=False)
5         f.write(line + '\n')
```

该文件可直接用于比赛提交。

## 15. 输出文件说明

本代码会生成两个主要输出文件。

### 15.1 完整结果 JSON 文件

路径由 `out_path` 指定。该文件包含完整运行信息，例如：

```
1 {
2   "task_id": "openseek-3",
3   "task_name": "Collatz Conjecture",
4   "evaluate_time": "2026-xx-xx xx:xx:xx",
5   "generated_code": "...",
6   "preview_accuracy_on_examples": 1.0,
7   "preview_errors": [],
8   "examples_eval": {
9     "total": 100,
10    "correct": 100,
11    "accuracy": 1.0,
12    "details": []
13  },
14  "test_eval": {
15    "total": 100,
16    "details": []
17  }
18 }
```

该文件主要用于实验记录和调试。

### 15.2 JSONL 提交文件

路径由 `test_jsonl_path` 指定。每一行是一个测试样例预测结果，例如：

```
1 {"test_sample_id": "openseek-3-xxxx", "prediction": "[36, 88, 148]"}
```

该文件是最终提交给评测系统的结果文件。





# OpenSeek-4 Conala Concat Strings 详细技术报告

## 1. 项目背景与任务概述

本技术报告基于现有的《OpenSeek-4 Conala Concat Strings 技术报告》和当前提交代码 `submit.py` 重新整理、扩展而成。原始报告已经给出了任务概述、数据集描述、核心函数说明和实验结果，但整体偏简略，缺少完整的数据流分析、模块职责划分、异常路径说明、输出文件结构、可复现流程，以及代码与报告之间的一致性检查。因此，本报告在原有内容基础上补充更细粒度的实现解释，并对当前代码中存在的命名不一致和回退求解器逻辑错误进行专门说明。

本任务对应 OpenSeek 系列中的第 4 个任务：**Conala Concat Strings**。任务目标是对输入中的字符串列表进行顺序拼接，并返回拼接后的字符串。输入通常是一个可以被 Python 字面量解析的字符串列表，例如：

```
1 ['p', 'that.', 'o']
```

对应输出为：

```
1 pthat.o
```

从算法角度看，该任务本身并不复杂，核心操作可以概括为：

```
1 strings = ast.literal_eval(input_text.strip())
2 return ''.join(strings)
```

但是当前代码并不是简单手写一个固定 solver，而是实现了一个“大模型代码生成 + 本地样例验证 + 错误反馈重试 + 回退求解器 + 测试集预测输出”的自动化流程。因此，本项目的重点不只是字符串拼接本身，还包括如何让模型根据任务定义和样例自动生成可执行 Python 代码，并通过训练样例验证其正确性，最后批量生成测试集提交文件。

## 2. 任务输入输出规范

### 2.1 输入格式

输入 `input_text` 是完整字符串形式的数据，内容本质上是 Python 列表字面量。列表元素均为字符串，列表长度不固定。典型输入如下：

```
1 ['hello', 'world']
2 ['p', 'that.', 'o']
3 ['fk', 'buttoned', 'earef', 'IasW']
```

由于输入是字符串形式，因此不能直接当作列表使用，需要先解析。代码中要求使用：

```
1 ast.literal_eval(input_text.strip())
```

该方法比 `eval` 更安全，只允许解析 Python 字面量结构，例如字符串、列表、字典、数字、布尔值等，适合处理此类任务输入。

## 2.2 输出格式

输出为一个字符串，表示将输入列表中的所有字符串元素按原顺序连接后的结果。输出中不额外添加空格、逗号、换行或列表符号。

例如：

| 输入                               | 输出                      |
|----------------------------------|-------------------------|
| <code>['p', 'that.', 'o']</code> | <code>pthat.o</code>    |
| <code>['hello', 'world']</code>  | <code>helloworld</code> |
| <code>['a', '', 'b']</code>      | <code>ab</code>         |
| <code>[]</code>                  | 空字符串                    |

## 2.3 样例与测试集的区别

数据文件中包含两类样本：

- 1. 训练/示例样本 **examples**：包含输入和标准输出，可用于提示词构造、代码验证和准确率计算。
- 2. 测试样本 **test\_samples**：通常只包含输入，需要程序生成预测结果，并保存为 JSONL 提交文件。

当前代码会先在 `examples` 上验证生成的 `solver`，再对 `test_samples` 进行预测。

## 3. 数据集组织方式

当前代码通过 `load_task(file_path)` 读取 JSON 格式任务文件。该文件预计包含如下字段：

```
1 {
2   "task_id": "openseek-4",
3   "task_name": "Conala Concat Strings",
4   "Definition": [
5     "Given a list of strings, concatenate them in order."
6   ],
7   "examples": [
8     {
9       "id": "example-id",
10      "input": ["'p'", "'that.', 'o'"],
11      "output": ["pthat.o"]
12    }
13  ],
14  "test_samples": [
15    {
16      "id": "test-id",
17      "input": ["'fk'", "'buttoned', 'earef', 'IasW']"
```

```
18     }
19   ]
20 }
```

`load_task` 返回五项内容：

```
1 task_id, task_name, definition, examples, test_samples
```

各字段含义如下：

| 字段                        | 类型                      | 作用                                         |
|---------------------------|-------------------------|--------------------------------------------|
| <code>task_id</code>      | <code>str</code>        | 任务编号，例如 <code>openseek-4</code>            |
| <code>task_name</code>    | <code>str</code>        | 任务名称，例如 <code>Conala Concat Strings</code> |
| <code>Definition</code>   | <code>List[str]</code>  | 自然语言任务定义，用于构造提示词                           |
| <code>examples</code>     | <code>List[Dict]</code> | 带标签样例，用于 few-shot 提示和验证                    |
| <code>test_samples</code> | <code>List[Dict]</code> | 无标签测试样本，用于最终预测                             |

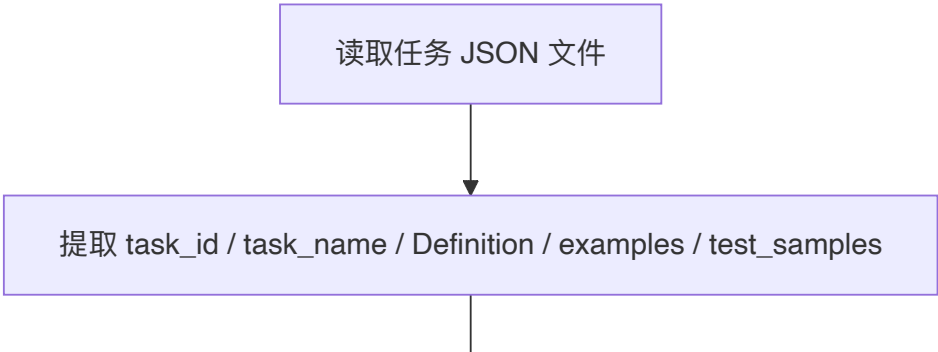
这种结构将任务元信息、样例数据和测试数据统一放入一个 JSON 文件中，使得主流程可以不依赖硬编码样例，而是根据数据文件动态构建提示词和评测流程。

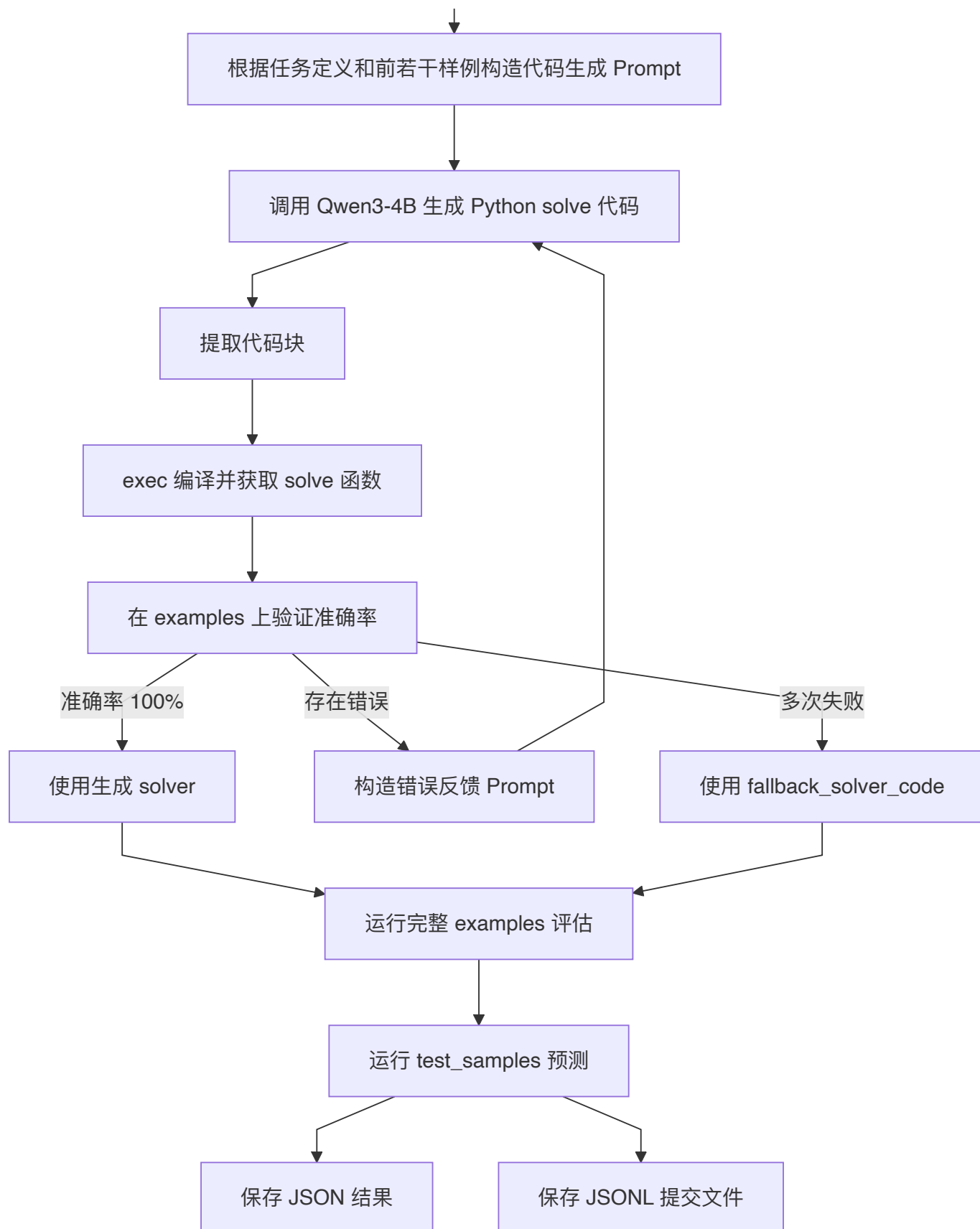
## 4. 代码整体架构

当前代码可以划分为七个核心模块：

- 1. **数据加载模块**：读取任务 JSON 文件，提取任务信息、样例和测试集。
- 2. **模型调用模块**：通过 OpenAI 兼容接口调用 Qwen3-4B 生成 Python 代码。
- 3. **代码提取模块**：从模型输出中提取 Markdown 代码块或纯代码文本。
- 4. **回退求解器模块**：当模型生成失败或验证不通过时使用保底 solver。
- 5. **代码编译模块**：使用 `exec` 动态执行生成代码，并提取 `solve` 函数。
- 6. **验证模块**：在 `examples` 上运行 solver，计算准确率并记录错误样例。
- 7. **预测与保存模块**：对 `examples` 和 `test_samples` 运行 solver，并保存 JSON 与 JSONL 文件。

整体流程如下：





该流程体现了一种“生成-验证-修正”的自动代码生成框架。模型生成的代码不会被直接用于测试集，而是先在带标签样例上进行本地验证；只有通过验证后，才会进入最终预测流程。

## 5. 核心模块详细分析

## 5.1 数据加载函数 `load_task`

函数定义如下：

```
1 def load_task(file_path: str) -> Tuple[str, str, List[str], List[Dict[str, Any]],  
  List[Dict[str, Any]]]:  
2     with open(file_path, 'r', encoding='utf-8') as f:  
3         data = json.load(f)  
4     return (  
5         data.get("task_id", ""),  
6         data.get("task_name", ""),  
7         data.get("Definition", []),  
8         data.get("examples", []),  
9         data.get("test_samples", []),  
10    )
```

该函数负责读取任务数据文件。实现上使用 `data.get(key, default)`，因此即使某个字段缺失，也不会直接抛出 `KeyError`，而是返回默认值。这种设计增强了程序对数据文件格式波动的容错能力。

不过，该函数目前只完成“读取”和“字段提取”，没有对字段类型和样本结构进行严格校验。例如：

- `examples` 是否为列表；
- 每个样例是否包含 `input`；
- 带标签样例是否包含 `output`；
- `output` 是否是列表，且至少包含一个标准答案；
- `test_samples` 是否为空。

如果数据文件格式不符合预期，错误可能会延迟到验证或预测阶段才暴露。为了提高工程健壮性，可以在该函数中增加数据校验逻辑。

建议增强版本如下：

```
1 def load_task(file_path: str):  
2     with open(file_path, 'r', encoding='utf-8') as f:  
3         data = json.load(f)  
4  
5     required_keys = ["task_id", "task_name", "Definition", "examples",  
6 "test_samples"]  
7     for key in required_keys:  
8         if key not in data:  
9             raise ValueError(f"任务文件缺少必要字段: {key}")  
10  
11     if not isinstance(data["examples"], list):  
12         raise TypeError("examples 必须是列表")  
13  
14     if not isinstance(data["test_samples"], list):  
15         raise TypeError("test_samples 必须是列表")  
16  
17     return (  
18         data["task_id"],
```

```

18         data["task_name"],
19         data["Definition"],
20         data["examples"],
21         data["test_samples"],
22     )

```

## 5.2 模型调用函数 `qwen_api`

代码中使用 OpenAI 兼容客户端调用 Qwen3-4B:

```

1 client = OpenAI(
2     api_key="dummy",
3     base_url=".../v1",
4 )

```

调用函数为:

```

1 def qwen_api(messages: List[Dict[str, str]], model: str = "/Qwen3-4B/Qwen/Qwen3-4B",
2   retries: int = 3) -> str:
3     for attempt in range(retries):
4         try:
5             res = client.chat.completions.create(
6                 model=model,
7                 messages=messages,
8                 temperature=0.0,
9             )
10            return res.choices[0].message.content or ""
11        except Exception as e:
12            if attempt == retries - 1:
13                print(f"\nAPI 调用失败: {e}")
14                return ""
15            time.sleep(2)
16    return ""

```

该函数的设计特点包括:

1. **固定低温度生成:** `temperature=0.0`，用于降低模型输出随机性，使生成代码更稳定。
2. **重试机制:** 默认最多重试 3 次，适合处理临时网络错误、服务端繁忙或接口异常。
3. **失败返回空字符串:** 最后一次失败后不抛出异常，而是返回空字符串，后续代码会尝试提取和编译，最终可能进入回退流程。

该设计适合比赛或批量运行场景，因为它不会因为一次 API 调用失败而立即中断整个程序。不过也存在不足:

- 没有记录每次失败的详细日志;
- 没有区分超时、限流、认证失败等错误类型;
- 没有设置 `max_tokens`，可能受到默认输出长度限制;
- API 地址硬编码在代码中，不利于迁移和提交。

更好的工程做法是通过环境变量或配置文件传入 API 参数:

```

1 client = OpenAI(
2     api_key=os.getenv("OPENAI_API_KEY", "dummy"),
3     base_url=os.getenv("OPENAI_BASE_URL"),
4 )

```

这样可以避免将服务地址或密钥相关信息写死在源码中。

## 5.3 代码块提取函数 `extract_code`

模型输出可能有两种形式：

第一种是 Markdown 代码块：

```

1 ```python
2 def solve(input_text: str) -> str:
3     ...
4 ```

```

第二种是纯代码文本：

```

1 def solve(input_text: str) -> str:
2     ...

```

因此代码使用正则表达式提取代码块：

```

1 CODE_BLOCK_PATTERN = re.compile(
2     r"```python\s*(.*?)```|```\s*(.*?)```",
3     re.DOTALL | re.IGNORECASE
4 )

```

该正则表达式支持两类代码块：

1. 显式标注 `python` 的代码块；
2. 未标注语言类型的普通代码块。

函数逻辑为：

```

1 def extract_code(text: str) -> str:
2     if not text:
3         return ""
4     match = CODE_BLOCK_PATTERN.search(text)
5     if match:
6         return (match.group(1) or match.group(2) or "").strip()
7     return text.strip()

```

如果匹配到代码块，则返回代码块内部内容；否则将整个模型输出作为代码返回。这种设计比较实用，因为不同模型在代码生成时的格式并不总是稳定。

但该方法也有局限：

- 如果模型输出多个代码块，只会取第一个；
- 如果代码块中包含解释文字和代码混排，可能提取不干净；
- 如果模型输出不是 Python 代码而是自然语言，后续 `exec` 会失败。

在当前流程中，这些问题会由 `compile_solver` 和 `validate_solver` 捕获，并触发下一轮修正或回退。

## 5.4 Prompt 构造函数 `build_code_generation_prompt`

当前代码通过任务名称、任务定义和前 10 个样例构造提示词：

```

1  def build_code_generation_prompt(task_name: str, definition: List[str], examples:
    List[Dict[str, Any]]) -> str:
2      demo_examples = examples[:10]
3      examples_text = "\n".join(f"输入: {ex['input']}\n输出: {ex['output']}" for ex
    in demo_examples)
4      return f"""
5      你是 Python 算法工程师。
6      请根据任务定义和样例生成可执行 Python 代码。
7
8      任务名: {task_name}
9      任务定义:
10     {chr(10).join(definition)}
11
12     样例:
13     {examples_text}
14
15     严格要求:
16     1. 定义函数 solve(input_text: str) -> str
17     2. input_text 是完整字符串, 例如 "[1,2,3]"
18     3. 可以使用 import 或标准库
19     4. 用 ast.literal_eval 解析输入
20     5. 返回值必须是字符串
21     6. 生成代码必须能被 exec 直接执行
22     """

```

该提示词包含五类信息：

| 信息类型   | 作用                      |
|--------|-------------------------|
| 角色设定   | 让模型以 Python 算法工程师身份生成代码 |
| 任务名称   | 告诉模型当前任务主题              |
| 任务定义   | 给出自然语言规则                |
| 输入输出样例 | 提供 few-shot 归纳依据        |
| 严格要求   | 限定函数签名、解析方式、返回类型和可执行性   |

对于 Concat Strings 任务，模型只需要从样例中归纳出“对字符串列表进行拼接”这一规则即可。前 10 个样例通常足够让模型生成正确代码。



不过，这里有一个细节问题：提示词第 2 条写的是：

```
1 input_text 是完整字符串，例如 "[1,2,3]"
```

这个例子来自数字列表任务，更适合 Collatz 或 Closest Integers 等任务。对于本任务，建议改成字符串列表示例：

```
1 input_text 是完整字符串，例如 "['p', 'that.', 'o']"
```

虽然 `ast.literal_eval` 同时适用于数字列表和字符串列表，但提示词示例与当前任务越一致，模型越不容易生成错误逻辑。

## 5.5 动态编译函数 `compile_solver`

代码通过 `exec` 执行模型生成的 Python 代码：

```
1 def compile_solver(code: str) -> Callable[[str], str]:
2     namespace: Dict[str, Any] = {}
3     exec(code, namespace)
4     if "solve" not in namespace:
5         raise ValueError("生成代码中未定义 solve")
6     return namespace["solve"]
```

该函数的作用是将字符串形式的 Python 代码转换成可调用函数对象。它要求生成代码中必须定义：

```
1 def solve(input_text: str) -> str:
2     ...
```

如果没有 `solve`，则直接抛出异常。该异常会被 `generate_valid_solver` 捕获，并作为错误反馈的一部分用于下一轮生成。

需要注意的是，当前实现中 `exec(code, namespace)` 没有限制可用内置函数、导入模块或文件系统访问，因此从安全角度看，它只适合在受控比赛环境或可信模型输出场景下使用。如果在开放环境中执行来自模型或用户的任意代码，应当使用沙箱、进程隔离、资源限制或白名单机制。

## 5.6 验证函数 `validate_solver`

验证函数用于在带标签样例上评估 solver：

```
1 def validate_solver(solve_func, examples, limit=100):
2     total = min(limit, len(examples))
3     errors = []
4     correct = 0
5     for sample in examples[:total]:
6         try:
7             pred = str(solve_func(sample["input"])).strip()
8         except Exception as e:
9             pred = f"<runtime_error: {e}>"
```

```

10     gt = str(sample["output"][0]).strip()
11     if pred == gt:
12         correct += 1
13     else:
14         errors.append({
15             "id": sample.get("id", ""),
16             "input": sample["input"],
17             "gt": gt,
18             "pred": pred
19         })
20     return correct / total if total else 0.0, errors

```

其评估标准是严格字符串匹配：

```

1 | pred == gt

```

该标准适合本任务，因为输出是一个确定字符串，不存在多个等价格式。对于 Concat Strings，输出中任何多余空格、引号、列表符号或换行都会导致判错。

函数还捕获运行时异常，并将异常信息写入预测结果：

```

1 | pred = f"<runtime_error: {e}>"

```

这样可以避免单个样例报错导致整个验证流程中断，也方便后续将错误样例反馈给模型进行修正。

## 5.7 自动生成与修正函数 `generate_valid_solver`

这是当前代码中最核心的自动化模块。其流程可以概括为：

1. 第一次调用模型时，使用任务定义和前 10 个样例构造初始 prompt；
2. 提取模型生成的代码；
3. 编译代码并获取 `solve` 函数；
4. 在 examples 的前若干条样例上验证准确率；
5. 如果准确率为 100%，直接返回该 solver；
6. 如果存在错误，将前 10 个错误样例构造成错误反馈 prompt；
7. 重新调用模型生成完整代码；
8. 如果多轮尝试后仍失败，则使用回退求解器。

核心代码逻辑如下：

```

1 | for attempt in range(1, max_attempts + 1):
2 |     if attempt == 1:
3 |         prompt = build_code_generation_prompt(task_name, definition, examples)
4 |     else:
5 |         error_text = "\n".join(
6 |             f"输入: {e['input']} | 正确输出: {e['gt']} | 你的输出: {e['pred']}"
7 |             for e in last_errors[:10]

```

```

8         )
9         prompt = f"""
10 你上一次生成的 solve 函数在以下样例上出错：
11 {error_text}
12 请重新生成完整 Python 代码，并修正逻辑。
13 严格要求：
14 - 定义 solve(input_text: str) -> str
15 - 可以使用 import 或标准库
16 - 输出字符串
17 """

```

这一设计体现了一个简单但有效的自修正机制：模型不只是一次性生成答案，而是根据本地验证结果迭代修复。对于复杂任务，这种机制可以显著提升最终 solver 的正确率。

对于本任务，由于规则很简单，通常第一轮即可生成正确代码。但保留多轮重试仍然有意义，因为模型可能输出解释文字、遗漏函数签名、返回列表而非字符串，或者错误地理解其他字符串操作任务。

## 5.8 预测函数 `run_predictions`

该函数用于批量运行 solver，并根据 `has_label` 参数决定是否计算准确率：

```

1 def run_predictions(solve_func, samples, has_label=False):
2     results = []
3     correct = 0
4     for s in tqdm(samples, desc="Running solver"):
5         try:
6             pred = str(solve_func(s["input"])).strip()
7         except Exception as e:
8             pred = f"<runtime_error: {e}>"
9         item = {"test_sample_id": s.get("id", ""), "prediction": pred}
10        if has_label:
11            gt = str(s["output"][0]).strip()
12            item["gt"] = gt
13            item["is_correct"] = pred == gt
14            if item["is_correct"]:
15                correct += 1
16            results.append(item)
17        output = {"total": len(samples), "details": results}
18        if has_label:
19            output["correct"] = correct
20            output["accuracy"] = correct / len(samples) if samples else 0.0
21        return output

```

当 `has_label=True` 时，输出包含：

```

1  {
2      "total": 100,
3      "correct": 100,
4      "accuracy": 1.0,
5      "details": [
6          {
7              "test_sample_id": "...",
8              "prediction": "...",
9              "gt": "...",
10             "is_correct": true
11         }
12     ]
13 }

```

当 `has_label=False` 时，输出只包含预测结果：

```

1  {
2      "total": 100,
3      "details": [
4          {
5              "test_sample_id": "...",
6              "prediction": "..."
7          }
8      ]
9  }

```

该函数同时用于 `examples` 和 `test_samples`，因此保证了训练集评估和测试集预测使用同一套运行逻辑。

## 6. 主流程说明

主流程位于：

```

1  if __name__ == "__main__":

```

执行步骤如下：

### 6.1 配置输入与输出路径

当前代码中路径为 Windows 本地绝对路径：

```

1  file_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
Annotation\data\openseek-4_conala_concat_strings.json"
2  out_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
Annotation\rgs_q4\experiment\openseek4_collatz_results.json"
3  test_jsonl_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
Annotation\experiment\openseek-4-v1.jsonl"

```

这里存在一个命名问题：`out_path` 文件名包含 `collatz`，但当前任务是 `Conala Concat Strings`。这不会影响代码运行，但会造成实验记录混乱。建议将其修改为：

```
1 out_path = r"...\\openseek4_concat_strings_results.json"
```

## 6.2 加载任务数据

```
1 task_id, task_name, definition, examples, test_samples = load_task(file_path)
```

随后打印任务信息和样例数量，便于确认数据是否正确读取。

## 6.3 生成有效 solver

```
1 generated_code, solve_func, preview_acc, preview_errors = generate_valid_solver(  
2     task_name, definition, examples, max_attempts=3  
3 )
```

该步骤会调用模型生成代码，并在 `examples` 的前最多 200 条样例上预验证。若准确率达到 100%，则进入后续预测；否则最多重试 3 次。

## 6.4 评估完整 examples

```
1 example_eval = run_predictions(solve_func, examples, has_label=True)
```

该步骤用于获得完整训练/示例集上的准确率。对于本任务，正确 solver 应达到 100%。

## 6.5 预测 test\_samples

```
1 test_eval = run_predictions(solve_func, test_samples, has_label=False)
```

该步骤生成测试集预测结果。由于测试集通常没有标准答案，因此不计算准确率。

## 6.6 保存完整 JSON 结果

```
1 output_data = {  
2     "task_id": task_id,  
3     "task_name": task_name,  
4     "evaluate_time": time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()),  
5     "generated_code": generated_code,  
6     "preview_accuracy_on_examples": preview_acc,  
7     "preview_errors": preview_errors[:20],  
8     "examples_eval": example_eval,  
9     "test_eval": test_eval  
10 }
```

该 JSON 文件用于保存完整实验记录，包括：

- 任务信息；
- 评估时间；
- 模型生成的 solver 代码；
- 预验证准确率；
- 预验证错误样例；
- examples 完整评估结果；
- test\_samples 预测结果。

这类文件适合用于调试、复盘和报告撰写。

## 6.7 保存 JSONL 提交文件

```
1 with open(test_jsonl_path, 'w', encoding='utf-8') as f:
2     for item in test_eval["details"]:
3         line = json.dumps(item, ensure_ascii=False)
4         f.write(line + '\n')
```

最终提交文件采用 JSONL 格式，即每一行是一个独立 JSON 对象。例如：

```
1 {"test_sample_id": "openseek-4-785d9cb1e1274411aafb19181ff05596", "prediction":
  "fkbuttonedearefIasW"}
```

JSONL 的优点是便于按行读取、增量写入和大规模评测。

## 7. 正确 solver 的逻辑分析

对于 Conala Concat Strings，正确解法应满足以下逻辑：

1. 去除输入字符串前后空白；
2. 使用 `ast.literal_eval` 将输入解析为 Python 列表；
3. 确认列表元素为字符串；
4. 使用 `''.join(strings)` 按原顺序拼接；
5. 返回拼接结果字符串。

最简正确实现如下：

```
1 import ast
2
3 def solve(input_text: str) -> str:
4     strings = ast.literal_eval(input_text.strip())
5     return ''.join(strings)
```

该实现的时间复杂度为：

```
1 | O(n + L)
```

其中：

- `n` 是列表元素数量；
- `L` 是所有字符串总长度。

空间复杂度为：

```
1 | O(L)
```

因为最终输出字符串需要存储所有拼接后的字符。

对于本任务，不需要排序、过滤、去重、插入分隔符或进行大小写转换。任何额外操作都可能导致答案错误。

---

## 8. 当前代码与原报告的一致性检查

现有 PDF 报告中将回退求解器描述为：

```
1 | def solve(input_text: str) -> str:
2 |     strings = ast.literal_eval(input_text.strip())
3 |     return ''.join(strings)
```

这个描述是符合 Conala Concat Strings 任务的。

但是，当前上传的 `submit.py` 中，`fallback_solver_code()` 实际内容为：

```
1 | def solve(input_text: str) -> str:
2 |     nums = ast.literal_eval(input_text.strip())
3 |     result = []
4 |     for x in nums:
5 |         if x % 2 == 0:
6 |             result.append(x // 2)
7 |         else:
8 |             result.append(x * 3 + 1)
9 |     return str(result)
```

该逻辑是 Collatz Conjecture 任务中的一步变换逻辑，并不是字符串拼接逻辑。对于本任务，如果模型生成代码失败、编译失败或验证多次不通过，程序会回退到这个错误 solver，导致最终预测完全不符合 Concat Strings 任务要求。

因此，当前代码存在一个明显问题：回退求解器未从前一个任务中正确替换为本任务逻辑。

建议修正为：

```

1 def fallback_solver_code() -> str:
2     return ''
3 import ast
4
5 def solve(input_text: str) -> str:
6     strings = ast.literal_eval(input_text.strip())
7     return ''.join(strings)
8     ''.strip()

```

如果希望进一步增强鲁棒性，可以加入类型转换：

```

1 def fallback_solver_code() -> str:
2     return ''
3 import ast
4
5 def solve(input_text: str) -> str:
6     items = ast.literal_eval(input_text.strip())
7     return ''.join(str(x) for x in items)
8     ''.strip()

```

不过，如果数据集明确保证元素均为字符串，则第一种实现更严格，也更符合任务定义。

## 9. 当前代码存在的问题与修改建议

### 9.1 回退求解器逻辑错误

问题：当前 `fallback_solver_code()` 是 Collatz 逻辑，不适用于 Concat Strings。

影响：当模型生成失败或验证失败时，程序会使用错误 solver，导致输出为数字列表变换结果，而不是字符串拼接结果。

修改建议：改为：

```

1 def fallback_solver_code() -> str:
2     return ''
3 import ast
4
5 def solve(input_text: str) -> str:
6     strings = ast.literal_eval(input_text.strip())
7     return ''.join(strings)
8     ''.strip()

```

### 9.2 输出文件命名不一致

问题：`out_path` 使用了 `openseek4_collatz_results.json`，但当前任务是 Concat Strings。

影响：不会影响程序运行，但会影响实验文件管理，容易和 OpenSeek-3 Collatz Conjecture 任务混淆。

修改建议：



```
1 out_path = r"...\\openseek4_concat_strings_results.json"
```

## 9.3 Prompt 示例与任务不完全一致

问题：提示词中写道：

```
1 input_text 是完整字符串，例如 "[1,2,3]"
```

这更像数字列表任务示例。对于本任务，更建议改成：

```
1 input_text 是完整字符串，例如 "['p', 'that.', 'o']"
```

影响：虽然不一定导致错误，但可能增加模型误解任务类型的概率。

## 9.4 API 配置硬编码

问题：API 地址和模型名称写在代码中，不利于跨机器运行，也不利于提交版本管理。

修改建议：使用环境变量：

```
1 client = OpenAI(  
2     api_key=os.getenv("OPENAI_API_KEY", "dummy"),  
3     base_url=os.getenv("OPENAI_BASE_URL"),  
4 )  
5  
6 MODEL_NAME = os.getenv("MODEL_NAME", "/Qwen3-4B/Qwen/Qwen3-4B")
```

## 9.5 `exec` 缺少安全限制

问题：`compile_solver` 直接执行模型生成代码，存在安全风险。

影响：在受控比赛环境中可以接受；在开放环境中不建议直接执行未知代码。

修改建议：

- 使用独立子进程运行生成代码；
- 设置超时；
- 限制文件系统访问；
- 限制可导入模块；
- 对 AST 做静态检查，只允许函数定义、导入标准库和简单表达式。

## 9.6 缺少运行超时控制

问题：如果模型生成了死循环代码，`validate_solver` 或 `run_predictions` 可能卡住。

修改建议：可以使用 `multiprocessing` 或 `signal` 添加单样例超时控制。对于本任务，正确逻辑非常简单，但作为通用自动代码生成框架，超时控制仍然重要。

## 10. 长上下文与提示词规模说明

当前代码的提示词由以下部分组成：

1. 任务角色说明；
2. 任务名称；
3. 任务定义；
4. examples 前 10 条样例；
5. 函数签名和输出格式要求。

对应代码为：

```
1 demo_examples = examples[:10]
```

因此，当前代码并没有构造超长 prompt，也没有显式保证 prompt 超过 20k tokens 或 20k 字符。如果比赛要求必须体现长上下文能力，当前版本需要进一步修改。例如：

```
1 def build_long_context_prompt(task_name, definition, examples, min_chars=20000):
2     blocks = []
3     blocks.append("你是 Python 算法工程师。请根据任务定义和大量样例生成 solve 函数。")
4     blocks.append(f"任务名: {task_name}")
5     blocks.append("任务定义: \n" + "\n".join(definition))
6     blocks.append("样例: ")
7
8     for ex in examples:
9         blocks.append(f"输入: {ex['input']}\n输出: {ex['output']}\n")
10        if len("\n".join(blocks)) >= min_chars:
11            break
12
13        blocks.append("""
14严格要求:
151. 定义 solve(input_text: str) -> str
162. input_text 是完整字符串, 例如 "['p', 'that.', 'o']"
173. 使用 ast.literal_eval 解析输入
184. 返回拼接后的字符串
195. 只输出完整 Python 代码
20""")
21        return "\n\n".join(blocks)
```

不过，对于 Concat Strings 本身，超长 prompt 并不是算法正确性的必要条件。由于规则简单，少量样例已经足够归纳出 `''.join(strings)`。如果只是追求解题效果，当前 few-shot prompt 更简洁；如果需要满足长上下文实验要求，则应当明确增加样例数量或加入任务分析知识块。

# OpenSeek-5 SemEval 2018 Task 1 Tweet Sadness Detection 技术报告

## 1. 项目概述

本项目面向 OpenSeek-5 任务，即 **SemEval 2018 Task 1 Tweet Sadness Detection**。任务目标是对输入推文进行二分类判断：识别推文作者是否表达了悲伤或广义负面情绪，并输出 `sad` 或 `not sad`。

与一般情感分类不同，本任务的难点不只是判断文本是否出现了 `sad`、`cry` 等显式词汇，而是需要结合推文中的短文本语境、表情符号、标签、幽默、讽刺、礼貌用语、成就语境等因素判断作者真实情绪。因此，本代码没有采用传统机器学习模型训练，也没有手写固定规则直接分类，而是采用 **大语言模型提示词推理 + 多提示词投票集成** 的方式完成预测。

当前实现的核心思路如下：

1. 从 JSON 数据集中读取任务 ID、训练样例和测试样例；
2. 对推文文本进行基础清洗，主要去除 `@username` 并规整空格；
3. 构建 6 个不同风格的情感判断提示词；
4. 对每条测试推文分别调用 Qwen3-4B 模型进行 6 次判断；
5. 将 6 次输出转换为 `sad` 或 `not sad`；
6. 使用多数投票得到最终预测；
7. 使用多线程并行处理测试集；
8. 将预测结果保存为 JSONL 格式，供平台提交。

与原有技术报告相比，本报告进一步补充了代码结构、模块职责、提示词设计逻辑、投票机制、并发推理、输出格式、可复现运行方式、代码中存在的问题以及后续优化方向。

## 2. 任务定义

### 2.1 任务名称

- 任务 ID: `openseek-5`
- 任务名称: `SemEval 2018 Task 1 Tweet Sadness Detection`
- 任务类型: 短文本情感二分类
- 输入: 一条英文推文文本
- 输出: `sad` 或 `not sad`

### 2.2 输入输出示例

输入示例：

```
1 | @user I am so upset 🙄
```

输出示例：

```
1 | sad
```

再如：

```
1 | Welcome and thanks for the greeting!
```

输出为：

```
1 | not sad
```

## 2.3 分类标准

本任务中的 `sad` 并不局限于狭义的“悲伤”。根据当前代码的提示词设计，`sad` 被扩展为一种广义负面情绪类别，包含以下情绪或表达：

- 悲伤、难过、哭泣、心碎；
- 失望、期待落空、不满；
- 沮丧、无力、痛苦；
- 愤怒、烦躁、厌恶；
- 明显抱怨或强烈负面评价；
- 第一人称表达出的负面状态；
- 明确的负面情绪标签。

而 `not sad` 不仅包括积极文本，也包括以下容易误判的情况：

- 纯积极、祝福、感谢、欢迎类表达；
- 礼貌性的 `sorry`；
- 成就完成后的哭泣表情；
- 纯客观事实陈述；
- 纯幽默或夸张表达；
- 宗教引用或固定文本；
- 只有标签但正文无真实情绪；
- 仅表达身体疲劳但无明显负面情绪。

## 3. 数据集结构

代码默认从如下 JSON 文件读取数据：

```
1 | file_path = r"D:\WorkSpace\python\flagOS赛题三\LongContext-ICL-Annotation\data\openseek-5_semeval_2018_task1_tweet_sadness_detection.json"
```

数据加载函数为：

```
1 def task5_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数返回三个对象：

| 字段           | 含义                                |
|--------------|-----------------------------------|
| task_id      | 当前任务编号，例如 <code>openseek-5</code> |
| examples     | 带标签训练样例，可用于提示词设计、调试或本地验证          |
| test_samples | 无标签测试样例，需要生成最终预测                  |

每条样例一般具有如下结构：

```
1 {
2     "id": "openseek-5-xxxx",
3     "input": "tweet text here",
4     "output": ["sad"]
5 }
```

测试集样例通常不包含 `output` 字段：

```
1 {
2     "id": "openseek-5-xxxx",
3     "input": "tweet text here"
4 }
```

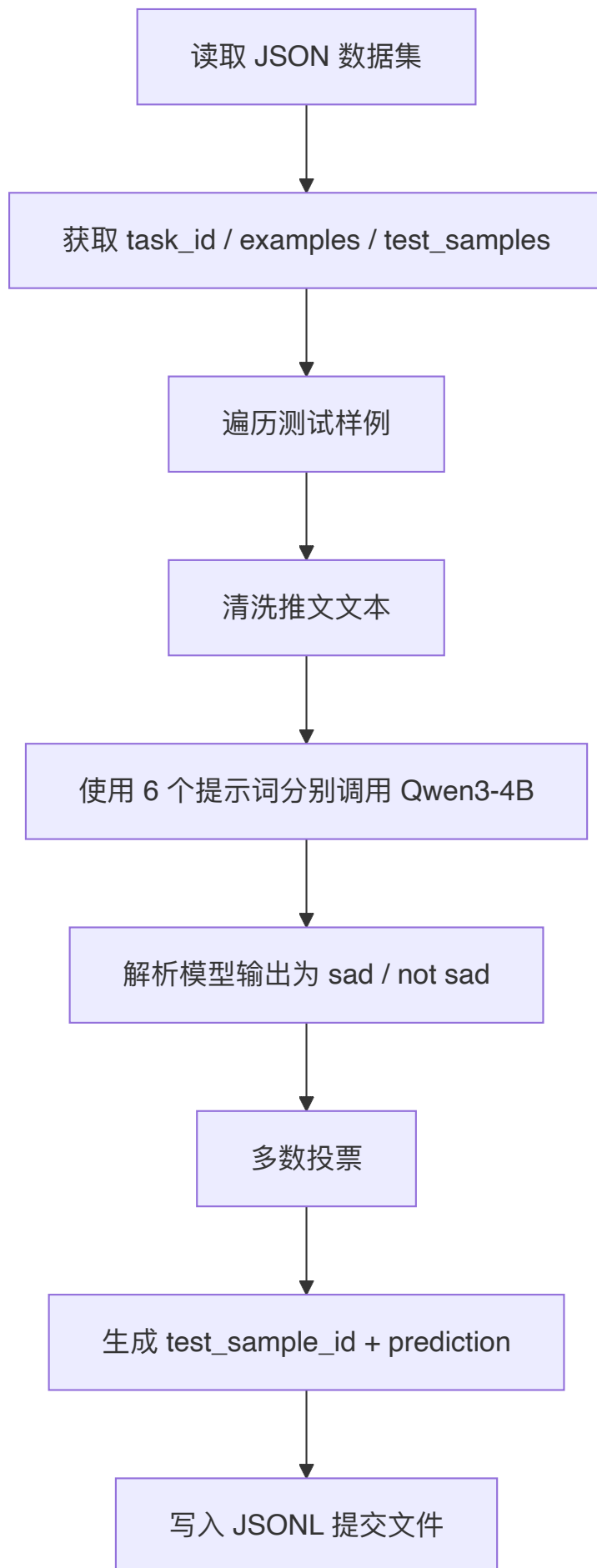
当前代码虽然加载了 `examples`，但主流程中实际只使用了 `test_samples` 进行预测，未对训练样例进行本地准确率验证。

## 4. 方法总体设计

### 4.1 总体技术路线

本任务采用的是 **Prompt Ensemble Classification**，即提示词集成分类方法。其本质是将一个二分类问题转化为多次大语言模型判断，再通过投票融合多个判断结果。

整体流程如下：



## 4.2 为什么使用多提示词投票

推文情感判断存在较多边界情况。例如：

- 同样出现哭泣表情，可能是真悲伤，也可能是激动或夸张；
- `sorry` 可能是真正道歉或悲伤，也可能只是礼貌用语；
- `rage`、`frustrated` 等词虽然不是 `sadness` 的狭义词，但在该任务中可能仍属于负面情绪；
- `lol`、`haha` 可能代表幽默，但也可能和真实负面情绪混合出现；
- `hashtag` 可能是情绪标签，也可能只是装饰性标签。

单一提示词容易对某类模式过度敏感，从而产生误报或漏报。因此代码设计了 6 个不同角度的提示词，包括精准规则版、few-shot 示例版、严格规则版、分步判断版、权重评分版和综合版。多个提示词的判断结果再通过投票融合，以降低单个提示词不稳定带来的风险。

## 5. 代码模块详细分析

### 5.1 依赖导入

代码导入的主要模块如下：

```
1 import json
2 import os
3 import time
4 from openai import OpenAI
5 from tqdm import tqdm
6 from concurrent.futures import ThreadPoolExecutor
7 from functools import partial
8 import re
```

各依赖作用如下：

| 依赖                              | 作用                        |
|---------------------------------|---------------------------|
| <code>json</code>               | 读取任务数据集，写入 JSONL 输出       |
| <code>os</code>                 | 创建输出目录                    |
| <code>time</code>               | API 重试等待时使用               |
| <code>OpenAI</code>             | 使用 OpenAI 兼容接口调用 Qwen3-4B |
| <code>tqdm</code>               | 显示测试集处理进度条                |
| <code>ThreadPoolExecutor</code> | 并发处理多条测试样例                |
| <code>partial</code>            | 固定函数部分参数，便于并发 map 调用      |
| <code>re</code>                 | 使用正则表达式清洗推文文本             |

## 5.2 数据加载模块

数据加载函数如下：

```
1 def task5_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数实现较简单，主要完成三项工作：

1. 使用 UTF-8 编码读取 JSON 文件；
2. 从 JSON 顶层对象中获取 `task_id`；
3. 获取训练样例 `examples` 和测试样例 `test_samples`。

使用 `data.get(..., [])` 可以避免字段缺失时直接报错。如果数据中没有 `examples` 或 `test_samples` 字段，函数会返回空列表，从而提高程序鲁棒性。

不过，该函数目前没有读取 `task_name` 和 `Definition` 字段。如果后续希望自动根据任务说明生成提示词，或者在报告中自动记录完整任务信息，可以扩展为：

```
1 return (
2     data.get("task_id", ""),
3     data.get("task_name", ""),
4     data.get("Definition", []),
5     data.get("examples", []),
6     data.get("test_samples", [])
7 )
```

## 5.3 模型接口配置

当前代码使用 OpenAI 兼容接口访问 Qwen3-4B：

```
1 client = OpenAI(
2     api_key="dummy",
3     base_url="https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1"
4 )
```

模型名称为：

```
1 /Qwen3-4B/Qwen/Qwen3-4B
```

该设置说明当前环境中的模型服务提供了类 OpenAI Chat Completions API，因此可以直接通过：

```
1 client.chat.completions.create(...)
```

进行调用。



需要注意的是，`api_key="dummy"` 表示接口可能不校验密钥，或者实际鉴权由平台环境完成。为了增强代码可移植性，建议后续改为环境变量方式：

```
1 client = OpenAI(  
2     api_key=os.getenv("OPENAI_API_KEY", "dummy"),  
3     base_url=os.getenv("OPENAI_BASE_URL")  
4 )
```

这样可以避免将服务地址、密钥等配置硬编码到代码中。

---

## 5.4 API 重试函数

代码中定义了 `qwen_api`：

```
1 def qwen_api(messages, model="/Qwen3-4B/Qwen/Qwen3-4B", retries=3):  
2     for attempt in range(retries):  
3         try:  
4             res = client.chat.completions.create(  
5                 model=model,  
6                 messages=messages,  
7                 temperature=0.0,  
8             )  
9             return res.choices[0].message.content  
10        except Exception as e:  
11            if attempt == retries - 1:  
12                print(f"\nAPI 调用失败: {e}")  
13                return ""  
14            time.sleep(2)
```

该函数具有以下特点：

1. 最多重试 3 次；
2. 每次失败后等待 2 秒；
3. 最后一次仍失败则返回空字符串；
4. 使用 `temperature=0.0` 保证输出尽可能稳定。

但是需要特别指出：当前主预测流程并没有实际调用 `qwen_api`。真正用于预测的是 `get_prediction()`，它直接调用了 `client.chat.completions.create()`，而没有复用这里的重试机制。因此，虽然代码中存在 API 重试函数，但实际测试集预测过程并没有获得该函数提供的重试保护。

这属于当前代码的一个重要可改进点。更合理的做法是让 `get_prediction()` 调用 `qwen_api()`，统一模型调用逻辑。

---

## 5.5 文本清洗模块

当前代码定义了如下清洗函数：

```

1 def clean_text(text):
2     cleaned_text = re.sub(r'@\w+', '', text)
3     cleaned_text = re.sub(r'\s+', ' ', cleaned_text).strip()
4     return cleaned_text

```

该函数主要完成两步：

1. 删除推文中的用户名引用，例如 @user、@abc123；
2. 将连续空格、换行符、制表符等规整为单个空格。

清洗前：

```

1 @user I am so upset today

```

清洗后：

```

1 I am so upset today

```

这样处理的原因是推文中的用户名通常不包含情感信息，保留它可能干扰模型判断。例如，模型可能错误关注用户名而忽略正文。

不过，当前清洗逻辑仍然较保守，没有删除 URL、HTML 实体、重复标点等内容。对于推文任务来说，可以考虑进一步扩展：

```

1 text = re.sub(r'https?://\S+|www\.\S+', '', text)
2 text = re.sub(r'&','&', text)
3 text = re.sub(r'\s+', ' ', text).strip()

```

但需要注意，不能盲目删除 hashtag 和 emoji，因为它们在本任务中可能包含重要情绪信号。

## 5.6 提示词集成模块

当前代码的核心是 PROMPTS 列表，其中包含 6 个提示词模板。每个模板都包含 {text} 占位符，用于插入当前推文：

```

1 content = prompt.format(text=text)

```

6 个提示词的设计目标并不完全相同，具体如下。

### 5.6.1 Prompt 1：精准规则版

Prompt 1 以规则为主，结构包括：

- 宽泛悲伤定义；
- 立即排除规则；
- 特别注意的 not sad 情况；

- 确认 sad 的条件；
- 特殊判断规则；
- 严格要求只输出 `sad` 或 `not sad`。

该提示词的优点是覆盖全面，对容易误判的情况进行了细粒度约束，例如礼貌用语、成就场景、宗教引用、纯标签、疲劳表达等。

## 5.6.2 Prompt 2: Few-shot 示例版

Prompt 2 在规则基础上加入了示例学习，列举了多个典型输入和对应判断。其作用是让模型通过具体样例理解边界情况，例如：

- 成就语境中的哭泣表情不一定是悲伤；
- 礼貌性的 `sorry` 不应直接判断为悲伤；
- 愤怒、失望、强烈厌恶可归入广义负面情绪；
- 纯幽默或夸张表达需要排除。

Few-shot 提示词通常适合处理语义边界模糊的问题，因为它不仅告诉模型规则，还提供了可模仿的判断模式。

## 5.6.3 Prompt 3: 严格规则版

Prompt 3 更强调“先排除，再确认”的判断方式。它把 `not sad` 的排除条件放在前面，要求模型优先识别纯积极、礼貌、幽默、宗教引用等非悲伤文本。

这种设计有利于降低误报，即避免模型因为看到某个单独负面词或表情符号就直接输出 `sad`。

## 5.6.4 Prompt 4: 分步判断版

Prompt 4 将判断过程拆分为 16 个步骤。它的逻辑类似人工标注流程：

1. 先判断是否纯积极；
2. 再判断是否成就场景；
3. 再判断是否礼貌用语；
4. 继续判断幽默、宗教、标签、疲劳等排除项；
5. 最后识别悲伤、愤怒、烦躁、失望、痛苦等负面情绪。

分步提示词的优点是显式引导模型进行层级判断，减少一步到位分类时的随机性。

## 5.6.5 Prompt 5: 权重评分版

Prompt 5 采用评分思想，把不同情绪信号赋予正负权重。例如：

- 强烈负面情绪给更高正分；
- 明显负面情绪给中等正分；
- 成就场景、礼貌用语、纯积极、宗教引用等给负分；
- 最后根据总分判断 `sad` 或 `not sad`。

虽然模型并不一定真的进行严格数学加减，但这种提示方式可以帮助模型理解不同证据的强弱关系。

## 5.6.6 Prompt 6：综合版

Prompt 6 是综合规则版，整合了前面几个提示词中的核心规则，强调先排除非悲伤语境，再识别真实负面情绪。

该提示词在投票中起到“稳定器”作用，避免过度依赖某一种提示词风格。

## 5.7 系统提示词设计

代码中定义了全局系统提示词：

```
1 SYSTEM_PROMPT = "你是一位情感分类专家，请根据提示词判断文本是否表达负面情绪。只输出 sad 或 not sad。"
```

该系统提示词主要起到两个作用：

1. 设定模型角色为情感分类专家；
2. 强制输出格式只能是 sad 或 not sad。

主流程中另外定义了一个局部变量：

```
1 system_prompt = "你是情感分析专家，能识别文字背后的真实情绪。特别注意讽刺、幽默、表情符号的语境含义。"
```

并通过 `partial` 传入 `process_single_sample()`。但是当前 `process_single_sample()` 函数内部并没有使用这个参数，`get_prediction()` 仍然使用全局 `SYSTEM_PROMPT`。因此，主流程中的 `system_prompt` 实际上没有生效。

这说明代码中存在一定冗余。如果希望该局部系统提示词生效，需要修改函数调用链，让 `get_prediction()` 接收并使用 `system_prompt` 参数。

## 5.8 单次预测函数

核心预测函数如下：

```
1 def get_prediction(text, prompt, temperature):
2     content = prompt.format(text=text)
3     messages = [
4         {"role": "system", "content": SYSTEM_PROMPT},
5         {"role": "user", "content": content}
6     ]
7
8     try:
9         res = client.chat.completions.create(
10             model="/Qwen3-4B/Qwen/Qwen3-4B",
11             messages=messages,
```

```

12         temperature=temperature
13     )
14     output = res.choices[0].message.content.lower().strip()
15     if 'sad' in output and 'not sad' not in output:
16         return 'sad'
17     return 'not sad'
18 except:
19     return None

```

该函数的流程如下：

1. 将清洗后的文本插入提示词模板；
2. 构建 Chat Completions API 所需的 messages；
3. 调用 Qwen3-4B 模型；
4. 将输出转为小写并去除首尾空格；
5. 如果输出包含 `sad` 且不包含 `not sad`，则判为 `sad`；
6. 否则统一判为 `not sad`；
7. 如果 API 调用异常，则返回 `None`。

这里的输出解析逻辑较简洁，能够处理模型输出中包含额外解释的情况。例如：

```
1 | sad
```

或：

```
1 | The answer is sad.
```

都会被解析为 `sad`。

但该逻辑也有一个特点：只要输出中含有 `not sad`，即使同时出现 `sad`，最终也会返回 `not sad`。这是为了避免 `not sad` 中包含子串 `sad` 导致误判。

## 5.9 单样例处理与投票机制

单条样例处理函数如下：

```

1  def process_single_sample(sample, system_prompt, task_id):
2      sample_id = sample.get("id", "")
3      input_text = clean_text(sample["input"])
4      votes = {'sad': 0, 'not sad': 0}
5      predictions = []
6      n_models = len(PROMPTS)
7      for i in range(min(n_models, len(PROMPTS))):
8          pred = get_prediction(input_text, PROMPTS[i], 0.0)
9          if pred:
10             votes[pred] += 1
11             predictions.append(pred)

```

```

12
13     if votes['sad'] > votes['not sad']:
14         prediction = 'sad'
15     else:
16         prediction = 'not sad'
17
18     return {
19         "task_id": task_id,
20         "sample_id": sample_id,
21         "input": input_text,
22         "model_output": prediction,
23     }

```

该函数对每条推文执行完整预测流程：

1. 读取样例 ID；
2. 清洗输入文本；
3. 初始化投票计数器；
4. 遍历 6 个提示词；
5. 每个提示词调用一次模型；
6. 记录 `sad` 和 `not sad` 的票数；
7. 如果 `sad` 票数严格大于 `not sad`，输出 `sad`；
8. 否则输出 `not sad`。

投票规则可以写成：

```

1 prediction = 'sad' if votes['sad'] > votes['not sad'] else 'not sad'

```

这意味着：

- 4:2、5:1、6:0 时输出 `sad`；
- 3:3 平票时输出 `not sad`；
- 如果部分 API 调用失败导致有效票数减少，也按当前有效票数判断；
- 如果所有调用都失败，`sad=0` 且 `not sad=0`，最终也会输出 `not sad`。

这种设计偏保守，可以减少误报，但也可能增加漏报。对于悲伤检测任务，如果平台更看重召回率，可以考虑把平票改为 `sad`，或者对 Prompt 2 / Prompt 4 等更可靠提示词设置更高权重。

## 5.10 并发推理模块

主流程使用 `ThreadPoolExecutor` 并发处理测试样例：

```

1 process_func = partial(process_single_sample, system_prompt=system_prompt,
2 task_id=task5_id)
3
4 max_workers = 200
5 results_list = []
6
7 with ThreadPoolExecutor(max_workers=max_workers) as executor:
8     for result in tqdm(executor.map(process_func, eval_data), total=total_cnt,
9                             desc="Evaluating"):
10         results_list.append(result)

```

这里使用 `executor.map()` 而不是 `as_completed()`，其主要好处是：

- 并发执行；
- 返回结果顺序与输入顺序一致；
- 方便后续直接生成提交文件；
- 不需要手动维护样例索引。

每条样例会调用 6 次模型，因此如果测试集数量为 `N`，总 API 调用次数约为：

```
1 6 × N
```

例如：

- 100 条测试样例：约 600 次模型调用；
- 1000 条测试样例：约 6000 次模型调用。

当前 `max_workers=200`，并发度非常高。它可以显著提高速度，但也可能带来以下风险：

1. 模型服务限流；
2. 网络连接过多导致请求失败；
3. 服务器压力过大；
4. 日志难以排查；
5. API 异常时大量样例回退为 `not sad`。

更稳妥的设置通常是先从 `16`、`32` 或 `64` 开始，根据模型服务吞吐能力逐步调高。

## 5.11 输出文件生成

模型推理完成后，代码将结果整理为提交格式：

```

1 output_data = [
2     {
3         "test_sample_id": item["sample_id"],
4         "prediction": item["model_output"]
5     }
6     for item in results_list
7 ]

```

最终写入 JSONL 文件：

```

1 os.makedirs(os.path.dirname(out_path), exist_ok=True)
2
3 with open(out_path, 'w', encoding='utf-8') as f:
4     for item in output_data:
5         line = json.dumps(item, ensure_ascii=False)
6         f.write(line + '\n')

```

输出路径为：

```

1 out_path = r"D:\Workspace\python\flag05赛题三\LongContext-ICL-
  Annotation\rgs_q5\experiment\openseek-5-v1.jsonl"

```

输出文件每一行是一个 JSON 对象，例如：

```

1 {"test_sample_id": "openseek-5-f296990fa391466eb3196f899d2a7b4e", "prediction": "not
  sad"}

```

该格式适合比赛平台逐行读取和评测。

## 6. 提示词策略分析

### 6.1 广义负面情绪定义

原任务名称是 Tweet Sadness Detection，看似只判断悲伤，但从当前代码设计可以看出，作者将 `sad` 扩展成广义负面情绪类别。这样做的原因是，任务样例中可能并不只包含直接悲伤表达，还可能包含：

- 愤怒；
- 失望；
- 痛苦；
- 烦躁；
- 厌恶；
- 抱怨；
- 心理低落；
- 明显负面情绪标签。



如果只识别狭义 sadness，可能漏掉大量真实标注为 `sad` 的样例。因此提示词中多次强调“宽泛悲伤定义”。

## 6.2 排除规则的重要性

---

推文数据的一大难点是表达方式非常非正式。例如，哭泣表情不一定代表悲伤，`sorry` 不一定代表负面情绪，`lol` 也不一定代表快乐。为了减少误报，提示词加入了大量排除规则，包括：

- 纯积极表达；
- 成就完成语境；
- 礼貌用语；
- 纯中性客观信息；
- 纯幽默文本；
- 祝福和感谢；
- 宗教引用；
- 讽刺或夸张表达；
- 纯标签；
- 单纯疲劳。

这些排除规则体现了本任务的核心难点：不能只依赖单个关键词，而要结合语境判断作者真实情绪。

## 6.3 确认规则的作用

---

在排除非悲伤语境后，提示词再通过确认规则判断 `sad`。确认规则主要覆盖以下信号：

- 明确悲伤词；
- 愤怒、厌恶、强烈负面评价；
- 烦躁和挫败；
- 失望和不满；
- 痛苦和受伤；
- 抱怨句式；
- 第一人称负面情绪表达；
- 明确情绪标签。

这种“先排除、再确认”的方式比单纯枚举 sad 词表更稳健。

## 6.4 多提示词互补性

---

6 个提示词并不是简单重复，而是从不同方向增强判断稳定性：

| 提示词      | 主要特点         | 作用           |
|----------|--------------|--------------|
| Prompt 1 | 精准规则版        | 覆盖完整规则，降低漏判  |
| Prompt 2 | Few-shot 示例版 | 用示例校准边界情况    |
| Prompt 3 | 严格规则版        | 强调先排除非悲伤语境   |
| Prompt 4 | 分步判断版        | 引导模型按流程推理    |
| Prompt 5 | 权重评分版        | 显式表达不同证据强弱   |
| Prompt 6 | 综合版          | 平衡规则覆盖与输出稳定性 |

通过投票融合后，单个提示词偶然误判对最终结果的影响会降低。

## 7. 实验流程说明

### 7.1 当前代码实际实验流程

当前主函数执行流程如下：

```
1 task5_id, examples, test_samples = task5_data_loader(file_path)
2 eval_data = test_samples
```

也就是说，代码将测试集作为预测对象，不使用训练集进行本地验证。

随后：

- 1. 使用 `partial()` 固定 `system_prompt` 和 `task_id`；
- 2. 使用 `ThreadPoolExecutor(max_workers=200)` 并发处理所有测试样例；
- 3. 每条测试样例调用 6 个提示词；
- 4. 对 6 个结果进行投票；
- 5. 将最终结果写为 JSONL。

### 7.2 当前代码没有执行训练集准确率评估

虽然数据加载函数返回了 `examples`，但当前主流程没有使用它们计算准确率。因此代码不能直接输出本地 `accuracy`。

如果需要在训练样例或开发样例上验证效果，可以加入如下逻辑：

```
1 eval_data = examples
2 correct = 0
3 for item in results_list:
4     # 需要保留原始 output 标签后再比较
5     pass
```

或者写一个单独的评估函数：

```
1 def evaluate_examples(examples, task_id):
2     correct = 0
3     for sample in examples:
4         result = process_single_sample(sample, system_prompt="", task_id=task_id)
5         gt = sample["output"][0].lower().strip()
6         pred = result["model_output"]
7         if pred == gt:
8             correct += 1
9     return correct / len(examples) if examples else 0.0
```

这样可以更清楚地比较不同提示词版本、不同投票策略和不同并发设置的实际效果。

## 7.3 输出结果

当前程序最终输出：

```
1 评测结果已成功保存为 JSONL 格式：D:\WorkSpace\python\flagOS赛题三\LongContext-ICL-
  Annotation\rgs_q5\experiment/openseek-5-v1.jsonl
```

输出文件格式示例：

```
1  {"test_sample_id": "openseek-5-f296990fa391466eb3196f899d2a7b4e", "prediction": "not
  sad"}
```

# OpenSeek-6 MNLI Same Genre Classification 详细技术报告

## 1. 项目概述

本报告面向 OpenSeek-6 任务：**MNLI Same Genre Classification**。该任务要求模型根据输入中的两条句子以及给定的 `Genre` 信息，判断两条句子是否可以视为来自同一语境、同一文体或同一类型的数据分布。最终输出为二分类标签：

- `Y`：两条句子属于相同或高度相关的语境 / 文体 / 类型；
- `N`：两条句子明显不属于同一语境 / 文体 / 类型，或者两者缺乏足够的实体、句法或语义联系。

现有代码采用 **提示词工程 + 大语言模型推理 + 并发批处理 + JSONL 提交文件生成** 的方式完成任务。与传统机器学习分类器不同，该方案没有训练专门的分类模型，而是通过人工设计规则化提示词，引导 Qwen3-4B 在每个样本上直接完成 `Y/N` 判断。

本任务的关键难点不只是判断两个句子的表面 `Genre` 是否一致，而是需要综合考虑以下因素：

- 两个句子是否共享相同实体、名词、专有名词或核心主题；
- 两个句子是否存在明显句法复用、否定改写或结构变体；
- 第二个句子是否是第一个句子的语义解释、示例化表达或近似改写；
- 两个句子是否只是宽泛主题相似，但缺少具体语义连接；
- 给定 `Genre` 是否能帮助判断句子风格，但又不能成为唯一判断依据。

因此，本代码的核心思想是：**优先判断句子间是否存在直接语义或结构联系；只有在缺乏实体重叠、句法重叠和语义重叠时，才进一步依据文体差异判断为 `N`。**

## 2. 任务定义与输入输出格式

### 2.1 任务名称

- 任务 ID：`openseek-6`
- 任务名称：`MNLI Same Genre Classification`

### 2.2 输入格式

每个测试样本的 `input` 字段通常包含三部分：

```
1 Sentence 1: ...
2 Sentence 2: ...
3 Genre: ...
```

其中：

- `Sentence 1`：第一条句子；

- `Sentence 2`：第二条句子；
- `Genre`：目标文体或语境类型。

代码中通过正则表达式对这三部分进行解析：

```
1 pattern = r"Sentence 1:\s*(.*?)\s*Sentence 2:\s*(.*?)\s*Genre:\s*(.*)"
```

该正则使用 `re.DOTALL`，因此可以处理句子中包含换行的情况。解析后得到：

```
1 sentence_1
2 sentence_2
3 genre
```

## 2.3 输出格式

模型最终需要为每个测试样本输出一个标签：

```
1 {
2     "test_sample_id": "openseek-6-xxxx",
3     "prediction": "Y"
4 }
```

或：

```
1 {
2     "test_sample_id": "openseek-6-xxxx",
3     "prediction": "N"
4 }
```

所有结果按 JSONL 格式写入输出文件，即每一行是一个独立 JSON 对象。该格式便于比赛平台逐行读取和评测。

## 3. 数据集结构说明

代码默认读取的数据文件路径为：

```
1 file_path = r"D:\Workspace\python\flagOS赛题三\LongContext-ICL-
  Annotation\data\openseek-6_mnli_same_genre_classification.json"
```

该 JSON 文件中主要包含：

```
1 {
2     "task_id": "...",
3     "examples": [...],
4     "test_samples": [...]
5 }
```

代码中的数据加载函数如下：

```
1 def task_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数返回三部分内容：

1. `task_id`：任务编号；
2. `examples`：带标签样例，理论上可用于开发和验证；
3. `test_samples`：测试样例，用于最终预测。

需要注意的是，当前代码虽然读取了 `examples`，但后续没有使用训练样例进行自动验证、错误分析或动态提示词构建。实际运行时使用的是 `test_samples`：

```
1 eval_data = test_samples
```

因此，当前方案属于 **固定提示词零样本 / 少样本推理方案**，而不是基于训练样例自动学习的方案。

---

## 4. 整体技术路线

当前代码整体流程如下：

```
1  读取任务 JSON 数据
2      ↓
3  获取 task_id、examples、test_samples
4      ↓
5  遍历 test_samples
6      ↓
7  解析 Sentence 1、Sentence 2、Genre
8      ↓
9  填充用户提示词模板
10     ↓
11  调用 Qwen3-4B 接口进行 Y/N 判断
12     ↓
13  清洗模型输出，只保留 Y 或 N
14     ↓
15  并发处理全部测试样本
16     ↓
17  按 JSONL 格式写入提交文件
```

该流程可以进一步拆成五个核心模块：

1. 数据加载模块；
2. 输入解析模块；
3. 提示词构建模块；

- 4. 模型调用与输出提取模块；
- 5. 并发预测与结果保存模块。

## 5. 代码模块详细分析

### 5.1 依赖库导入

代码首先导入所需模块：

```
1 import json
2 import os
3 import time
4 import re
5 from openai import OpenAI
6 from tqdm import tqdm
7 from concurrent.futures import ThreadPoolExecutor
8 from functools import partial
```

各依赖作用如下：

| 模块                 | 作用                                      |
|--------------------|-----------------------------------------|
| json               | 读取任务 JSON 文件，并写出 JSONL 预测结果             |
| os                 | 创建输出目录                                  |
| time               | API 调用失败后等待重试                           |
| re                 | 解析输入文本、清洗模型输出                           |
| OpenAI             | 通过 OpenAI-compatible 接口调用 Qwen3-4B      |
| tqdm               | 显示预测进度条                                 |
| ThreadPoolExecutor | 多线程并发处理测试样本                             |
| partial            | 固定部分函数参数，便于传入 <code>executor.map</code> |

### 5.2 数据加载模块

数据加载函数实现较为直接：

```
1 def task_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数使用 `data.get()`，如果字段不存在则返回默认值。这种写法可以避免因为某个字段缺失而直接抛出 `KeyError`，具有一定鲁棒性。

不过，如果 JSON 文件路径错误或文件内容不是合法 JSON，函数仍会抛出异常。当前代码没有对文件读取失败进行捕获，因此实际运行前需要确认：

1. `file_path` 路径正确；
2. 文件编码为 UTF-8；
3. 文件内容是合法 JSON；
4. 数据中至少包含 `test_samples` 字段。

## 5.3 模型客户端配置

代码通过 OpenAI-compatible API 方式创建模型客户端：

```
1 client = OpenAI(  
2     api_key="dummy",  
3     base_url="https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1"  
4 )
```

这里的 `api_key="dummy"` 表示当前平台接口可能不需要真实密钥，或者由平台网关完成身份控制。  
`base_url` 指向部署好的模型服务地址。

实际使用时需要注意：

1. 如果部署地址变化，需要同步修改 `base_url`；
2. 如果接口要求真实 API key，需要替换 `api_key`；
3. 如果模型名称变化，需要在 `qwen_api()` 中修改默认 `model` 参数；
4. 并发线程数过高时，接口可能触发限流或连接失败。

## 5.4 API 调用模块

模型调用函数如下：

```
1 def qwen_api(messages, model="/Qwen3-4B/Qwen/Qwen3-4B", retries=3):  
2     for attempt in range(retries):  
3         try:  
4             res = client.chat.completions.create(  
5                 model=model,  
6                 messages=messages,  
7                 temperature=0.0,  
8             )  
9             return res.choices[0].message.content  
10        except Exception as e:  
11            if attempt == retries - 1:  
12                print(f"\nAPI 调用失败: {e}")  
13                return ""  
14            time.sleep(2)
```

该函数具有以下特点：

1. 固定使用低温度推理



`temperature=0.0` 使模型输出更稳定，适合分类任务。

## 2. 支持失败重试

当接口请求失败时，会等待 2 秒后重试，最多重试 3 次。

## 3. 最终失败时返回空字符串

如果所有重试都失败，则返回 `""`。后续 `extract_prediction()` 会将空输出默认解析为 `N`。

## 4. 没有设置 `max_tokens`

当前提示词要求只输出 `Y` 或 `N`，理论输出很短，但建议显式设置较小的 `max_tokens`，例如 4 或 8，以减少无关输出和资源消耗。

## 5. 没有对异常类型进行区分

网络错误、限流错误、模型服务错误和响应格式异常都会进入同一分支。为了更便于排查，可以后续增加更详细的日志。

# 6. 提示词工程设计

## 6.1 系统提示词

系统提示词如下：

```
1 system_prompt = """You are an expert data annotator.  
2 Return exactly one label: Y or N.  
3 Do not provide any explanation."""
```

该系统提示词的目标是强约束模型输出格式：

1. 模型扮演专业数据标注员；
2. 只能返回 `Y` 或 `N`；
3. 不允许解释原因。

这种设计适合提交评测场景，因为评测平台通常只接受标准标签。如果模型输出解释文本，可能导致格式错误。

## 6.2 用户提示词整体结构

用户提示词模板 `user_prompt_template` 是整个方案的核心。它包含以下部分：

1. 任务说明；
2. 标签含义；
3. 关键快捷判断规则；
4. `N` 标签判断条件；
5. `Genre` 提示；
6. 5 个示例；
7. 当前待预测样本；
8. 输出格式要求。

整体结构如下：

```
1 Task:
2 You are given Sentence 1, Sentence 2, and a target Genre.
3 Decide whether to output Y or N.
4
5 Label meaning:
6 - Y: ...
7 - N: ...
8
9 CRITICAL SHORTCUTS:
10 ...
11
12 When to output N:
13 ...
14
15 Genre hints:
16 ...
17
18 Examples:
19 ...
20
21 Now solve this instance:
22 Sentence 1: ...
23 Sentence 2: ...
24 Genre: ...
25
26 Return only Y or N.
```

这种提示词属于 **规则驱动的 few-shot prompt**。它不是只让模型凭直觉分类，而是把任务中最容易出错的判断规则提前写明，并通过示例强化模型行为。

## 6.3 Y 标签判断规则

代码提示词中最强调的是 Y 的优先判断规则。提示词明确要求：

不要在 Sentence 2 明显由 Sentence 1 派生时，单独判断 Sentence 2 的 genre。

也就是说，若两个句子存在强关联，即使第二句表面风格有变化，也应优先输出 Y。

### 6.3.1 实体或名词重叠

如果两个句子包含相同的具体实体、专有名词或核心名词，应直接判断为 Y。

例如：

```
1 Sentence 1: Beyond is the Bhimsen Temple...
2 Sentence 2: The Bhimsen Temple was never built...
3 Genre: travel
4 Output: Y
```

即使第二句与第一句在事实上矛盾，只要两者围绕同一个具体实体展开，仍视为相关语境。

这一规则主要解决 MNLI 类任务中常见的“假设句与前提句事实冲突但语境相同”的问题。模型如果只关注真假关系，可能会误判为 **N**；但本任务关注的是同文体 / 同语境，而不是蕴含真假。

### 6.3.2 句法重叠或否定改写

如果第二句大量复用了第一句的句法结构，只是增加或删除否定词，也应判断为 **Y**。

例如：

```
1 Sentence 1: Therefore, this Statement requires that information...
2 Sentence 2: This statement doesn't require that any information...
3 Genre: government
4 Output: Y
```

虽然两句语义方向相反，但它们明显来自同一语境，是对同一 statement 的改写或否定。因此输出 **Y**。

### 6.3.3 语义转述或具体化示例

如果第二句是第一句中抽象概念的具体例子，也判断为 **Y**。

例如：

```
1 Sentence 1: ... politics and in the business sector
2 Sentence 2: Being the governor of state or being a manager at a company.
3 Genre: telephone
4 Output: Y
```

**governor** 对应 politics，**manager at a company** 对应 business sector，因此第二句可以看作第一句的语义具体化表达。

## 6.4 **N** 标签判断规则

提示词中对 **N** 的判断比较保守：

```
1 Output N ONLY IF there is NO specific entity overlap AND the sentences belong to
  clearly different domains.
```

也就是说，只有同时满足以下条件时才输出 **N**：

1. 两个句子之间没有具体实体、名词或主题重叠；
2. 两个句子属于明显不同的领域或叙述风格；
3. 第二句不能视为第一句的解释、改写、否定或例子。

例如：

```
1 Sentence 1: Such simulations can help policymakers assess the long-term consequences
  of fiscal policy...
2 Sentence 2: Nixon's decision to force Ho Chi Minh to withdraw saved 18,000 American
  lives.
3 Genre: government
4 Output: N
```

虽然两句都可能宽泛地与政治或政府相关，但第一句是正式政策分析，第二句是历史叙述，且没有共享具体实体，因此判断为 `N`。

## 6.5 Genre hints 的作用

提示词列出了常见 genre 的含义：

| Genre        | 含义              |
|--------------|-----------------|
| face-to-face | 面对面日常对话         |
| government   | 正式公共信息、政策语言     |
| letters      | 筹款、捐赠、信件风格      |
| 9/11         | 与 9/11 袭击相关的语料  |
| slate        | 文化、社会评论或杂志风格    |
| telephone    | 电话对话、口语、停顿、轮次交替 |
| travel       | 旅游指南式描述         |
| verbatim     | 简短语言学相关文本       |
| oup          | 非虚构、教育性说明文      |
| fiction      | 小说或文学叙事         |

这些 genre hints 可以帮助模型识别句子风格，但提示词也明确强调不能只依赖 genre。原因是本任务中两个句子是否相同 genre，通常需结合句子间关系，而不是孤立判断某一句是否像目标 genre。

## 7. 输入解析机制

### 7.1 正则解析函数

输入解析函数如下：

```

1 def parse_input_text(input_text: str):
2     pattern = r"Sentence 1:\s*(.*?)\s*Sentence 2:\s*(.*?)\s*Genre:\s*(.*)"
3     match = re.search(pattern, input_text.strip(), re.DOTALL)
4     if not match:
5         raise ValueError(f"输入格式无法匹配: {input_text}")
6
7     sentence_1 = match.group(1).strip()
8     sentence_2 = match.group(2).strip()
9     genre = match.group(3).strip()
10
11     return sentence_1, sentence_2, genre

```

该函数使用非贪婪匹配 `(.*?)` 提取句子内容：

1. 从 `Sentence 1:` 后开始匹配第一句；
2. 到 `Sentence 2:` 前结束；
3. 从 `Sentence 2:` 后开始匹配第二句；
4. 到 `Genre:` 前结束；
5. `Genre:` 后的所有内容作为 `genre`。

## 7.2 解析方案优点

该解析方案简洁有效，适合输入格式固定的比赛数据：

1. 对多行句子具有兼容性；
2. 可以保留句子内部标点和大小写；
3. 不需要复杂 JSON 嵌套解析；
4. 便于快速定位输入格式错误。

## 7.3 潜在风险

当前解析函数对输入格式依赖较强。如果测试数据出现以下情况，可能导致解析失败：

1. `Sentence 1:` 拼写变化，例如 `Sentence1:`；
2. `Sentence 2:` 拼写变化，例如 `Sentence Two:`；
3. `Genre:` 缺失；
4. 句子正文中意外包含 `Sentence 2:` 或 `Genre:` 字样；
5. 输入字段为空。

如果解析失败，函数会抛出 `ValueError`，而当前多线程流程没有对该异常进行单独捕获，因此可能导致整体程序中断。后续可以在 `process_single_sample()` 中增加异常保护。

# 8. 输出提取与格式控制

## 8.1 模型输出提取函数

由于大语言模型偶尔可能输出解释性文字，因此代码设计了输出清洗函数：

```
1 def extract_prediction(raw_output: str) -> str:
2     if not raw_output:
3         return "N"
4
5     text = raw_output.strip().upper()
6
7     if text == "Y":
8         return "Y"
9     if text == "N":
10        return "N"
11
12    match = re.search(r"\b([YN])\b", text)
13    if match:
14        return match.group(1)
15
16    return "N"
```

## 8.2 处理逻辑

该函数按以下顺序处理：

1. 如果模型输出为空，默认返回 **N**；
2. 如果清洗后的文本正好是 **Y**，返回 **Y**；
3. 如果清洗后的文本正好是 **N**，返回 **N**；
4. 如果文本中存在独立的 **Y** 或 **N**，提取第一个；
5. 如果无法解析，默认返回 **N**。

## 8.3 默认返回 **N** 的影响

默认返回 **N** 是一种保守策略，能够避免输出非法标签。但它也会带来潜在偏差：

1. API 失败会被记为 **N**；
2. 模型输出异常会被记为 **N**；
3. 解析失败但未显式报错的情况会偏向 **N**；
4. 如果真实标签中 **Y** 占比较高，可能降低准确率。

更稳妥的做法是将异常输出单独记录，例如：

```
1 {
2     "test_sample_id": "...",
3     "raw_output": "...",
4     "prediction": "N",
5     "status": "parse_failed"
6 }
```

这样可以在正式提交前检查异常样本。

## 9. 单样本处理流程

核心处理函数如下：

```
1  def process_single_sample(sample, task_id):
2      test_sample_id = sample.get("id", "")
3      input_text = sample.get("input", "").strip()
4
5      sentence_1, sentence_2, genre = parse_input_text(input_text)
6
7      user_prompt = user_prompt_template.format(
8          sentence_1=sentence_1,
9          sentence_2=sentence_2,
10         genre=genre
11     )
12
13     messages = [
14         {"role": "system", "content": system_prompt},
15         {"role": "user", "content": user_prompt}
16     ]
17
18     raw_output = qwen_api(messages)
19     prediction = extract_prediction(raw_output)
20
21     return {
22         "test_sample_id": test_sample_id,
23         "prediction": prediction
24     }
```

该函数完成了从原始样本到预测结果的完整转换。

### 9.1 获取样本 ID

```
1  test_sample_id = sample.get("id", "")
```

样本 ID 用于最终提交。如果 ID 缺失，则使用空字符串。建议实际比赛数据中不要出现空 ID，否则提交结果可能无法正确对应样本。

### 9.2 获取输入文本

```
1  input_text = sample.get("input", "").strip()
```

`strip()` 可以去除前后空白字符，减少解析失败概率。

### 9.3 解析句子和 genre

```
1  sentence_1, sentence_2, genre = parse_input_text(input_text)
```

解析成功后，后续提示词构建不再使用完整原始输入，而是使用结构化字段。

## 9.4 构建模型消息

```
1 messages = [  
2     {"role": "system", "content": system_prompt},  
3     {"role": "user", "content": user_prompt}  
4 ]
```

这种消息结构符合 OpenAI Chat Completions 接口格式。

## 9.5 返回预测结果

函数最终只返回提交必需字段：

```
1 {  
2     "test_sample_id": test_sample_id,  
3     "prediction": prediction  
4 }
```

这使输出文件简洁，符合提交要求。

---

# 10. 并发预测设计

## 10.1 多线程执行

主流程中使用 `ThreadPoolExecutor` 并发处理测试样本：

```
1 max_workers = 50  
2 results_list = []  
3  
4 with ThreadPoolExecutor(max_workers=max_workers) as executor:  
5     for result in tqdm(executor.map(process_func, eval_data), total=total_cnt,  
6                             desc="Predicting"):  
7         results_list.append(result)
```

这里设置 `max_workers=50`，表示最多同时发起 50 个样本的预测请求。

## 10.2 为什么使用多线程

本任务的主要耗时来自 API 请求，而不是本地 CPU 计算。对于这种 I/O 密集型任务，多线程可以显著提升吞吐量：

- ```
1 单线程：一个样本请求完成后再处理下一个  
2 多线程：多个样本请求同时进行
```

因此，在接口服务能承受并发压力的前提下，多线程可以大幅缩短整体推理时间。



## 10.3 使用 `executor.map` 的优势

代码使用的是：

```
1 executor.map(process_func, eval_data)
```

而不是 `as_completed()`。其优点是：

1. 返回结果顺序与输入顺序一致；
2. 不需要额外排序；
3. 输出文件顺序更稳定；
4. 逻辑简单，便于维护。

虽然 JSONL 提交通常依赖 `test_sample_id` 而不是顺序，但保持顺序仍有利于调试和人工核查。

## 10.4 并发风险

并发数设置为 50 在很多环境下是合理的，但仍有以下风险：

1. 模型服务限流；
2. 网络连接数过多；
3. 单个请求失败率上升；
4. 返回空字符串后被默认解析为 `N`；
5. 本地线程调度开销增加。

如果运行中出现大量 API 失败，可以将 `max_workers` 降低到 10、20 或 30，并增加错误日志统计。

---

## 11. 输出文件生成

### 11.1 输出路径

代码默认输出路径为：

```
1 out_path = r"D:\Workspace\python\flagOS赛题三\LongContext-ICL-Annotation\rqs_q6\experiment/openseek-6-v1.jsonl"
```

该文件是最终提交文件。

### 11.2 创建输出目录

写文件前，代码会自动创建目录：

```
1 os.makedirs(os.path.dirname(out_path), exist_ok=True)
```

这样即使输出目录不存在，也不会因为路径缺失而写入失败。

## 11.3 写入 JSONL

写入逻辑如下：

```
1 with open(out_path, 'w', encoding='utf-8') as f:
2     for item in results_list:
3         f.write(json.dumps(item, ensure_ascii=False) + "\n")
```

每个 `item` 是一个字典，写入后每行格式如下：

```
1 {"test_sample_id": "openseek-6-8c412f5feb914ef7ab4cd468fb6396d6", "prediction": "N"}
```

## 11.4 输出完成提示

程序最后打印：

```
1 print(f"\n预测完成！总数：{total_cnt}")
2 print(f"结果已成功保存至：{out_path}")
```

这有助于确认预测样本数量和结果文件位置。

# OpenSeek-7 Jeopardy Answer Generation 技术报告

## 1. 任务概述

本代码面向 OpenSeek-7 Jeopardy 答案生成任务，目标是根据输入中的 `Category` 与 `Clue` 自动生成一个简短、规范的 Jeopardy-style 答案。与直接让模型回答不同，该代码采用“两阶段提示词推理流程”：

### 1. 第一阶段：路由分类

使用 Qwen 模型判断题目属于哪一种解题类型，即输出一级 `route` 和二级 `subroute`。

### 2. 第二阶段：答案生成

将第一阶段得到的 `route/subroute` 作为强制约束，重新构造答题 Prompt，让 Qwen 在指定解题视角下生成最终答案。

该设计的核心思想是：先把题目划分到更细的解题模式，再利用该模式约束答案生成，从而降低模型在 Jeopardy 题目中出现跑题、过度解释或答案不规范的问题。

## 2. 输入数据与输出格式

### 2.1 数据来源

代码通过如下路径读取 OpenSeek-7 数据集：

```
1 file_path = r"D:\Workspace\python\flagOS赛题三\LongContext-ICL-Annotation\data\openseek-7_jeopardy_answer_generation_all.json"
```

数据加载函数如下：

```
1 def task5_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数返回三类内容：

- `task_id`：任务编号；
- `examples`：带标准答案的训练/验证样例；
- `test_samples`：需要生成答案的测试样例。

### 2.2 单条输入格式

代码默认每条样例的 `input` 字段包含类似如下结构：

```
1 Category: SOME CATEGORY
2 Clue: SOME JEOPARDY CLUE
```

其中：

- `Category` 表示 Jeopardy 题目类别；
- `Clue` 表示需要回答的题干。

代码通过 `parse_input_text()` 解析这两个字段。

## 2.3 输出格式

当 `eval_flag=True` 时，代码处理测试集并输出 JSONL 文件，每行格式如下：

```
1 {"test_sample_id": "xxx", "prediction": "answer"}
```

该格式符合一般提交文件要求，即每条测试样例对应一个 `test_sample_id` 和一个模型预测答案 `prediction`。

## 3. 整体流程设计

代码主流程可以概括为：

```
1 读取 few-shot 路由样例
2      ↓
3 构造 ROUTE_PROMPT
4      ↓
5 读取 OpenSeek-7 数据集
6      ↓
7 对每条样例执行两阶段推理
8      ↓
9 第一阶段：Qwen 输出 route/subroute
10     ↓
11 解析 route/subroute JSON
12     ↓
13 第二阶段：构造 forced route/subroute 答题 Prompt
14     ↓
15 Qwen 输出 final_answer
16     ↓
17 本地验证或生成提交 JSONL
```

该流程中最重要的设计是：路由分类结果不会直接作为答案，而是作为第二阶段答题 **Prompt** 的约束条件。这样可以让模型先判断题目属于客观事实题、文字游戏题、补全题、拼写题还是多对象题，再根据对应模式生成答案。

## 4. Few-shot 路由样例读取

### 4.1 外部 few-shot 文件

代码首先读取外部 few-shot 文件：

```
1 FEW_SHOT_PATH = r"D:\Workspace\python\flagOS赛题三\LongContext-ICL-
  Annotation\rgs_q7\experiment\filtered_routes_output.txt"
2
3 with open(FEW_SHOT_PATH, "r", encoding="utf-8") as f:
4     few_shot_examples = f.read().strip()
```

该文件的作用是为路由分类阶段提供示例。由于 Jeopardy 题目大多数是客观知识问答，少部分才是文字游戏、补全、拼写等特殊类型，因此 few-shot 样例主要用于帮助模型稳定地区分不同路由类别。

## 4.2 few-shot 文件的必要性

如果只依靠短规则，模型容易把一些普通事实题误判成：

- `wordplay`
- `completion`
- `spelling`
- `multi`

而实际任务中大多数题目应优先归入 `factual`。因此，引入外部 few-shot 文件有三个作用：

### 1. 提供分类边界样例

通过人工筛选或模型生成的示例，告诉 Qwen 哪类题属于 `factual`，哪类题才属于 `wordplay/completion/spelling`。

### 2. 增强长上下文使用

将大量 few-shot 样例拼入 `ROUTE_PROMPT` 后，可以形成较长上下文，满足任务对长 Prompt 使用的要求。

### 3. 降低误路由概率

第一阶段路由越稳定，第二阶段答案生成越容易被约束到正确方向。

## 5. 第一阶段：路由分类 Prompt

### 5.1 ROUTE\_PROMPT 的构造

第一阶段使用 `ROUTE_PROMPT` 作为 system prompt，其核心要求是：

```
1 你需要对 Jeopardy trivia 题目进行双层级路由分类：一级路由 route、二级子路由 subroute。
```

输出固定为：

```
1  {"route": "xxx", "subroute": "xxx"}
```

并且要求：

- 只输出纯 JSON；
- 不输出解释；
- 不输出思考过程；

- 默认优先判断为 `factual`；
- 只有在非常明确时才使用 `wordplay`、`completion`、`spelling`、`multi`。

## 5.2 一级路由设计

代码中定义了五类一级路由：

| 一级路由                    | 含义       | 使用场景                     |
|-------------------------|----------|--------------------------|
| <code>factual</code>    | 客观事实题    | 默认首选，适合绝大多数 Jeopardy 题   |
| <code>wordplay</code>   | 文字游戏题    | 谐音、押韵、变位词、before-after 等 |
| <code>completion</code> | 补全题      | 谚语、名言、歌词、标题补全            |
| <code>spelling</code>   | 拼写/形式约束题 | 首字母、尾字母、长度、字母模式          |
| <code>multi</code>      | 多对象/集合题  | 多个实体、集合选择、组合类题           |

其中，`factual` 又细分为：

- `factual_person`
- `factual_place`
- `factual_title`
- `factual_number`
- `factual_definition`
- `factual_role_identity`
- `factual_bridge`
- `factual_list_or_set_selection`

这种二级分类使后续答题 Prompt 能更明确地引导模型，例如人物题偏向回答人名，地点题偏向回答国家/城市/地名，作品题偏向回答书名、电影名或歌曲名。

# 6. 第二阶段：答案生成 Prompt

## 6.1 PROBLEM\_SOLVE\_PROMPT

第二阶段使用 `PROBLEM_SOLVE_PROMPT`，其角色定义是：

```
1 | You are an expert Jeopardy-style trivia solver.
```

该 Prompt 明确要求模型输入：

- `category`
- `clue`
- `forced route`

- forced subroute
- instruction

并输出 JSON：

```
1 {
2   "reasoning_brief": "...",
3   "candidates": ["...", "...", "..."],
4   "final_answer": "..."
5 }
```

其中真正用于提交的是 `final_answer` 字段。

## 6.2 build\_route\_instruction()

为了让第二阶段更准确地使用路由结果，代码通过 `build_route_instruction(route, subroute)` 为不同路由动态生成说明。例如：

- 如果 `route=factual` 且 `subroute=factual_person`，则提示模型偏向人物识别；
- 如果 `route=factual` 且 `subroute=factual_place`，则提示模型偏向地点识别；
- 如果 `route=wordplay` 且 `subroute=homophone`，则提示模型偏向谐音推理；
- 如果 `route=completion`，则提示模型偏向短语、标题或引语补全；
- 如果 `route=spelling`，则提示模型关注首字母、尾字母、长度等形式约束。

该函数将分类结果转化为自然语言指令，使第二阶段不是简单地“知道分类”，而是能够按照分类采取不同解题策略。

## 7. JSON 解析与容错机制

### 7.1 路由结果解析

第一阶段模型理论上应输出严格 JSON：

```
1 {"route": "factual", "subroute": "factual_definition"}
```

但实际模型可能输出多余文本，因此代码先用正则提取 JSON 片段：

```
1 json_match = re.search(r'\{.*?\}', raw_str, re.DOTALL)
```

如果仍然失败，则用正则分别提取 `"route"` 和 `"subroute"`：

```
1 route_match = re.search(r'"route"\s*:\s*"([^"]+)"', raw_str)
2 subroute_match = re.search(r'"subroute"\s*:\s*"([^"]+)"', raw_str)
```

这种设计可以提升模型输出格式不稳定时的鲁棒性。

## 7.2 答案结果解析

第二阶段模型输出也可能不是严格 JSON，因此代码定义了：

```
1 def extract_json_from_text(raw_text: str) -> Dict[str, Any]:
```

该函数先尝试直接 `json.loads()`，如果失败，再用正则匹配 `{...}` JSON 块。最终通过：

```
1 final_answer_raw = data.get("final_answer", raw_str).lower()
```

提取最终答案。如果 JSON 解析失败，则直接使用原始模型输出作为答案。

## 8. 单样例处理流程

### 8.1 测试样例处理：process\_single\_sample()

`process_single_sample()` 用于正式生成测试集答案，其流程为：

1. 读取样例 ID 和输入文本；
2. 使用 `ROUTE_PROMPT` 调用 Qwen 进行路由分类；
3. 解析 `route/subroute`；
4. 解析 `Category/Clue`；
5. 构造第二阶段 payload；
6. 使用 `PROBLEM_SOLVE_PROMPT` 再次调用 Qwen；
7. 解析 `final_answer`；
8. 返回提交所需字段。

返回结构如下：

```
1 return {
2     "task_id": task_id,
3     "sample_id": sample_id,
4     "input": input_text,
5     "model_output": final_answer_raw,
6 }
```

### 8.2 训练样例验证：process\_single\_example()

`process_single_example()` 与 `process_single_sample()` 基本一致，但额外读取标准答案：

```
1 gt = sample["output"][0]
```

并返回：



```
1 | "predict_right": gt == final_answer_raw
```

该函数用于在本地 examples 上估计模型方案效果。代码主程序在 `eval_flag=False` 时默认运行该路径，并统计前 1000 条样例的正确率。

## 9. 并发处理设计

代码使用 `ThreadPoolExecutor` 进行并发调用：

```
1 | max_workers = 200
2 | with ThreadPoolExecutor(max_workers=max_workers) as executor:
3 |     for result in tqdm(executor.map(process_func, data), total=total_cnt,
4 |         desc="Evaluating"):
5 |         results_list.append(result)
```

这里使用的是 `executor.map()`，而不是 `as_completed()`。它的好处是：

- 并发执行；
- 返回结果顺序与输入顺序一致；
- 便于后续按原始样例顺序写入 JSONL。

不过，`max_workers=200` 较大，如果 API 服务端限流或本地网络不稳定，可能导致大量请求失败。因此实际运行时可以根据服务器吞吐量改为 16、32 或 64。

## 10. 本地验证与正式提交

### 10.1 eval\_flag=False：本地验证模式

默认代码中：

```
1 | eval_flag = False
```

此时程序运行：

```
1 | data = examples[:1000]
```

并计算：

```
1 | correct_rate = float(correct_cnt) / float(total_cnt)
```

该模式用于在训练/验证样例上检查 Prompt 设计是否有效。

### 10.2 eval\_flag=True：正式生成模式

当设置：

```
1 | eval_flag = True
```

程序会处理 `test_samples`，并写出：

```
1 | out_path = r"...\\openseek-7-v1.jsonl"
```

输出 JSONL 格式：

```
1 | {"test_sample_id": "xxx", "prediction": "xxx"}
```

# OpenSeek-8 技术报告

## 1. 任务概述

本任务面向 OpenSeek-8 kernel generation 场景，目标是根据每条测试样例中的自然语言功能描述、Wrapper Entry Information、参数说明和数学定义，自动生成一段可执行的 Python 代码作为预测结果。与普通算法题不同，该任务的输出不是一个数值、字符串或分类标签，而是完整的 Python 函数实现。最终提交文件采用 JSONL 格式，每一行对应一道测试题，字段中保存测试样例 ID 和生成的代码字符串。

本方案的核心思想是：不要让模型在每道题上都从零开始理解题目、推断函数名、选择算子、处理参数并编写代码，而是先离线构建一组中间知识文件，再在最终生成阶段把这些知识文件作为 Prompt-KB 提供给 Qwen 模型。这样可以把任务拆成两个阶段：

1. **中间知识构建阶段**：根据训练样例、测试题描述和高频算子模式，生成算子手册与逐题分析文件；
2. **最终代码生成阶段**：主脚本读取中间文件，针对每个测试样例构造短而精准的 Prompt，调用 Qwen 生成代码，并进行清洗、校验、修复和规则回退。

本报告对应的主代码文件为：

```
1 | task8_prompt_kb_generator copy.py
```

额外构建的中间文件生成脚本为：

```
1 | build_task8_kb_files_no_autoscore.py
```

最终使用的三个中间知识文件为：

```
1 | task8_operator_manual.md
2 | task8_per_question_analysis.jsonl
3 | task8_per_question_analysis.md
```

其中，`task8_operator_manual.md` 和 `task8_per_question_analysis.jsonl` 会被主预测脚本直接读取；`task8_per_question_analysis.md` 是逐题分析的 Markdown 可读版本，主要用于人工检查、报告展示和版本复核，不直接参与主脚本推理。

## 2. 文件组成与作用

### 2.1 主预测脚本

主预测脚本是整个方案的最终执行入口，负责读取原始任务数据、读取中间知识文件、调用 Qwen 生成代码、验证代码格式，并输出 JSONL 提交文件。

```
1 | task8_prompt_kb_generator copy.py
```

脚本中的核心路径配置如下：

```
1 FILE_PATH = "/Users/ks/Desktop/LongContext-ICL-Annotation/data/openssl-  
8_kernel_generation.json"  
2 OUT_PATH = "/Users/ks/Desktop/LongContext-ICL-  
Annotation/outputs/experiment/task8_prompt_kb_fixed_v4_submit.jsonl"  
3  
4 OPERATOR_MANUAL_PATH = "/Users/ks/Desktop/LongContext-ICL-  
Annotation/task8_operator_manual.md"  
5 PER_QUESTION_ANALYSIS_PATH = "/Users/ks/Desktop/LongContext-ICL-  
Annotation/task8_per_question_analysis.jsonl"  
6  
7 MODEL_NAME = "/Qwen3-4B/Qwen/Qwen3-4B"  
8 MAX_WORKERS = 8  
9  
10 USE_MODEL = True  
11 ENABLE_REPAIR = True  
12 ENABLE_RULE_FALLBACK = True
```

上述配置说明：

- `FILE_PATH` 是原始任务数据文件路径；
- `OUT_PATH` 是最终提交 JSONL 输出路径；
- `OPERATOR_MANUAL_PATH` 指向全局算子手册；
- `PER_QUESTION_ANALYSIS_PATH` 指向逐题结构化分析 JSONL；
- `MODEL_NAME` 指定 Qwen 模型；
- `MAX_WORKERS` 用于多线程并发生成；
- `USE_MODEL` 控制是否调用模型；
- `ENABLE_REPAIR` 控制模型输出失败后是否尝试修复；
- `ENABLE_RULE_FALLBACK` 控制是否启用规则回退代码生成。

## 2.2 中间文件一： `task8_operator_manual.md`

`task8_operator_manual.md` 是任务级算子使用手册。它根据题目输入和参考答案中出现的 PyTorch/F API 进行整理，每个高频算子通常包含以下信息：

- 算子名称；
- 用途说明；
- 典型使用场景；
- 简短 demo 代码。

例如文件中包含如下类型的内容：

```

1  ## `F.linear`
2
3  - **用途**: 线性层  $y = xA^T + b$ 。
4  - **典型场景**: linear+activation 复合算子。
5  - **Demo**:
6
7  ```python
8  import torch
9  import torch.nn.functional as F
10 y = F.linear(input, weight, bias)

```

1  
2 该文件的作用是为最终生成脚本提供全局算子知识。主脚本不会每次都把完整手册塞进 Prompt，而是根据当前题目的 family 选择相关片段。例如：

```

3
4  - `conv_norm_pool` 类题目优先选择 `F.conv2d`、`F.batch_norm`、pooling 等内容；
5  - `matmul_linear` 类题目优先选择 `F.linear`、`torch.bmm`、`torch.matmul`、`torch.mm`、
  `torch.mv` 等内容；
6  - `linalg` 类题目优先选择 `torch.linalg.solve`、`torch.linalg.svd`、`torch.linalg.qr`、
  `torch.linalg.cholesky` 等内容；
7  - `activation` 和 `elementwise_math` 类题目优先选择 `F.relu`、`F.gelu`、`torch.sqrt`、
  `torch.exp`、`torch.log` 等内容。

```

8  
9 这样做可以降低 Prompt 长度，同时让模型在生成当前题代码时看到最相关的算子规则。

10  
11 `### 2.3 中间文件二: `task8_per_question_analysis.jsonl``  
12  
13 ``task8_per_question_analysis.jsonl`` 是逐题结构化分析文件，每行对应一道测试样例。该文件记录每道题的函数名、任务类型、功能描述、数学定义、算子链、实现策略和公开简化解题步骤。

14  
15 典型字段设计如下：

```

16
17  ```json
18  {
19    "id": "openseek-8-501f776ba20444458ac14dd7292cc913",
20    "function": "fused_bmm_rmsnorm_gelu_dropout_sub",
21    "family": "matmul_linear",
22    "description": "Performs a fused operation combining batch matrix multiplication,
  RMS normalization, GELU activation, dropout, and subtraction.",
23    "wrapper_signature": "fused_bmm_rmsnorm_gelu_dropout_sub(input1, input2, other,
  normalized_shape, dropout_p=0.5, training=True, approximate='none', eps=1e-5, *,
  out=None) -> Tensor",
24    "math": "Z=bmm(X,Y); Z_norm=RMSNorm(Z); G=GELU(Z_norm); D=Dropout(G); output=D-O",
25    "other": "other must be broadcastable to output shape",
26    "detected_ops": ["torch.bmm", "custom_rms_norm", "F.gelu", "F.dropout"],
27    "answer_apis": ["torch.bmm", "torch.rsqrt", "torch.mean", "F.gelu", "F.dropout"],
28    "answer_summary": "Use PyTorch fallback implementation with _write_out support.",
29    "thinking_steps": [
30      "先识别 wrapper 函数名并保持同名定义。",
31      "根据功能描述将任务拆成 bmm、RMSNorm、GELU、Dropout、Sub 五个步骤。",
32      "优先使用 PyTorch API 保证语义正确，避免复杂 Triton。",

```

```
33     "处理 out= 参数并返回写回后的结果。"
34 ]
35 }
```

这里的 `thinking_steps` 不是隐藏思维链，也不是要求模型输出完整内部推理过程，而是公开可审计的简化步骤摘要。它的作用类似工程执行清单，用于提醒最终生成模型按照哪些步骤实现代码。

该 JSONL 文件会被主脚本通过 `sample id` 检索。最终生成 Prompt 时，每道题只会插入当前题对应的分析内容，而不是把 166 道题全部塞入上下文。

## 2.4 中间文件三： `task8_per_question_analysis.md`

`task8_per_question_analysis.md` 是 JSONL 文件的可读版本。它不是最终预测脚本必须读取的输入文件，但在工程上有重要意义：

1. 方便人工检查每道题的 wrapper、family、算子链是否合理；
2. 方便技术报告展示逐题拆解过程；
3. 方便在不同候选版本之间比较分析质量；
4. 方便发现 JSONL 中字段缺失、题目顺序异常或 family 分类错误。

该文件通常包含共性总结，例如：

```
1 总题数：166。
2 任务共同模式：自然语言功能描述 + Wrapper Entry Information + 参数/数学定义 → 生成同名
  Python/Triton wrapper。
3 高稳策略：先保证函数名、import、参数兼容、out/inplace 支持，再用 PyTorch API 实现语义；复杂
  Triton 仅在必要且简单时使用。
4 family 分布：linalg=58, matmul_linear=45, reduction=27, conv_norm_pool=24,
  attention_softmax_loss=6, indexing=4, activation=2
```

该分布说明，Task8 中线性代数、矩阵乘、归约和卷积归一化类任务占比较高。如果强行生成复杂 Triton kernel，容易出现维度、stride、mask、block size 或参数签名错误。因此本方案把“优先 PyTorch fallback”作为核心策略。

## 3. 总体流程设计

整体流程可以分为两个阶段：

```
1 阶段一：中间知识文件构建
2 原始任务 JSON
3   ↓
4 构造一次大于 20k 字符的长 Prompt
5   ↓
6 调用 Qwen 生成/辅助生成 task8_operator_manual.md
7   ↓
8 分批调用 Qwen 生成 task8_per_question_analysis.jsonl
9   ↓
10 将 JSONL 渲染为 task8_per_question_analysis.md
```

```
11      ↓
12 人工检查不同 candidate_tag 版本，线上提交评分后选择最终版本
13
14 阶段二：最终代码生成
15 原始任务 JSON
16      ↓
17 读取 task8_operator_manual.md
18      ↓
19 读取 task8_per_question_analysis.jsonl
20      ↓
21 逐题抽取函数名、任务类型、算子链和对应分析
22      ↓
23 构造短 Prompt 调用 Qwen 生成代码
24      ↓
25 清洗模型输出
26      ↓
27 语法与结构校验
28      ↓
29 失败时修复或规则 fallback
30      ↓
31 写入最终 JSONL 提交文件
```

## 4. 中间文件生成过程

### 4.1 为什么要额外构建中间文件

如果直接让 Qwen 对每道测试题生成代码，模型在一次请求中需要同时完成以下任务：

1. 从自然语言题面中识别功能；
2. 从 Wrapper Entry 中抽取函数名；
3. 判断参数顺序、默认值和关键字参数；
4. 判断属于哪类算子任务；
5. 选择 PyTorch API 或 Triton 实现；
6. 处理 `out=`、`inplace`、`dim`、`keepdim`、`dtype`、`eps`、`alpha`、`beta` 等参数；
7. 写出完整 Python 代码；
8. 避免输出 markdown、JSON 或解释文字；
9. 避免生成无法解析的语法；
10. 在复杂任务中避免错误 Triton kernel。

这些要求全部集中到一次生成中，会明显增加失败概率。中间文件的作用是把任务拆开：

- 第一阶段做“理解和总结”；
- 第二阶段做“代码实现”。

这样最终模型在生成代码时，不需要重新长篇分析，只需要按照已经总结好的函数名、family、算子链和实现策略写代码。

## 4.2 build\_task8\_kb\_files\_no\_autoscore.py 的设计原则

中间知识生成脚本的设计原则如下：

1. 至少构造并使用一次长度大于 20k 字符的 Prompt。
2. 使用 Qwen/OpenAI-compatible API 生成 operator manual 和逐题结构化分析。
3. 不把 hidden chain-of-thought 写入文件。
4. thinking\_steps 只保存公开可展示的简化解题步骤摘要。
5. 不直接生成最终提交答案，只生成中间知识文件。

脚本开头的注释已经明确说明：

```
1 """
2 用途：
3     为 OpenSeek-8 kernel generation 任务生成三个中间知识文件：
4     1) task8_operator_manual.md
5     2) task8_per_question_analysis.jsonl
6     3) task8_per_question_analysis.md
7
8 设计原则：
9     - 至少构造并使用一次长度大于 20k 字符的 Prompt，以满足题目对长上下文使用的要求。
10    - 使用 Qwen/OpenAI-compatible API 生成 operator manual 和逐题结构化分析。
11    - 不进行“多版本自动评分”。线上平台分数无法由本地脚本获得，因此不同版本仅按 candidate_tag 保
    存，
12    最终版本由人工提交到线上平台后根据真实分数选择。
13    - thinking_steps 字段只保存可公开展示的简化解题步骤摘要，不要求模型输出隐藏思维链。
14 """
```

## 4.3 构造一次大于 20k 字符的长 Prompt

题目要求中需要体现一次大于 20k 的 Prompt 使用。因此，中间文件生成脚本专门提供 build\_long\_kb\_prompt 函数，用于构造长上下文知识构建 Prompt。

核心代码如下：

```
1 def build_long_kb_prompt(
2     task_id: str,
3     definitions: List[str],
4     examples: List[Dict[str, Any]],
5     test_samples: List[Dict[str, Any]],
6     min_chars: int = 20000,
7 ) -> str:
8     family_counter = Counter(detect_task_family(get_sample_input(s)) for s in
    test_samples)
9     header = f"""
10 你是 OpenSeek-8 kernel generation 任务的中间知识构建助手。
11
12 任务 ID: {task_id}
13
14 目标：
```



```

15 1. 从训练样例答案与测试题目描述中总结常见 PyTorch/F API 使用模式。
16 2. 形成一个算子使用手册 task8_operator_manual.md。
17 3. 为每道测试题抽取 wrapper、任务类型、算子链、实现策略和公开的简化解题步骤。
18 4. 输出内容必须偏工程化、可复现、可直接被后续代码生成脚本检索使用。
19 5. thinking_steps 只写可公开展示的简要解题步骤，不输出隐藏思维链。
20
21 任务定义：
22 {json.dumps(definitions, ensure_ascii=False, indent=2)}
23
24 测试集 family 初步统计：
25 {json.dumps(family_counter, ensure_ascii=False, indent=2)}
26
27 生成要求：
28 - 优先 PyTorch fallback，不盲目生成 Triton。
29 - 对 conv/matmul/attention/linalg/indexing 等复杂任务，先保证语义正确与参数兼容。
30 - 记录 out=、inplace、dim、keepdim、dtype、eps、alpha、beta、training、p 等常见参数。
31 - 对每题输出 compact JSON 字段：
    id/function/family/description/wrapper_signature/math/other/detected_ops/answer_apis/
    answer_summary/thinking_steps。
32 """.strip()

```

该 Prompt 的内容不是无意义填充，而是由以下部分组成：

1. 任务 ID；
2. 任务定义；
3. 测试集 family 初步统计；
4. 训练样例输入和参考答案代码摘要；
5. 测试样例输入片段；
6. 全局生成策略；
7. 参数处理要求；
8. 输出字段要求；
9. PyTorch fallback 策略；
10. 避免复杂 Triton 的限制。

为了保证长度满足要求，脚本会不断追加训练样例和测试样例摘要；如果数据本身不足，还会追加重复但有用的工程规则：

```

1 filler = "\n".join([
2     "请继续保持：函数名必须完全一致；优先 PyTorch fallback；避免复杂 Triton；输出字段稳定；
  thinking_steps 为公开简化步骤。"
3     for _ in range(200)
4 ])
5 while len(prompt) < min_chars:
6     prompt += "\n" + filler
7
8 assert len(prompt) >= min_chars, f"long prompt is too short: {len(prompt)} <
  {min_chars}"

```

随后脚本会把长 Prompt 写入文件：

```
1 long_prompt_path = out_dir / f"task8_long_prompt_{args.candidate_tag}.txt"
2 long_prompt_path.write_text(long_prompt, encoding="utf-8")
3 print(f"long_prompt_chars={len(long_prompt)}")
```

该文件用于复查生成过程确实使用过一次大于 20k 的 Prompt。

## 4.4 使用长 Prompt 生成 `task8_operator_manual.md`

生成算子手册时，脚本把前面构造好的长 Prompt 作为上下文，再追加手册格式要求：

```
1 def generate_operator_manual_md(client: OpenAI, model: str, long_prompt: str,
2   out_path: Path) -> str:
3     user_prompt = f"""
4     {long_prompt}
5     请基于以上长上下文，生成 task8_operator_manual.md。
6
7     格式要求：
8     # Task8 答案中常用算子使用手册
9
10    本手册根据题目输入与答案代码中出现的 PyTorch/F API 总结，每个算子给出用途、场景和 demo。
11
12    ## 高频算子统计
13
14    然后按照算子逐个说明，每个算子包含：
15    - 用途
16    - 典型场景
17    - Demo 代码块
18
19    只输出 Markdown，不要输出 JSON，不要解释你如何思考。
20    """.strip()
21    print(f"[INFO] operator manual prompt chars = {len(user_prompt)}")
22    assert len(user_prompt) > 20000, "At least one Qwen prompt must be >20k chars."
```

这里的关键点有三个：

1. `operator manual` 的生成 Prompt 明确大于 20k；
2. 输出格式限定为 Markdown；
3. 不要求模型解释内部思考，只要求输出工程文档。

生成后，如果模型输出没有以 Markdown 标题开头，脚本会自动补充标题：

```
1 if not content.startswith("#"):
2     content = "# Task8 答案中常用算子使用手册\n\n" + content
3 out_path.write_text(content, encoding="utf-8")
```

最终得到：

该文件在最终预测脚本中被读取并按 family 选择片段。

## 4.5 分批生成 task8\_per\_question\_analysis.jsonl

逐题分析文件数量较多，如果一次性让模型输出 166 行 JSONL，容易出现截断、格式错误或漏题。因此脚本采用 batch 方式生成。

批次 Prompt 构造函数如下：

```

1  def build_analysis_batch_prompt(
2      definitions: List[str],
3      examples: List[Dict[str, Any]],
4      batch: List[Dict[str, Any]],
5      operator_manual_md: str,
6      batch_start: int,
7  ) -> str:
8      example_blocks = []
9      for i, ex in enumerate(examples[:6], 1):
10         example_blocks.append(compact_example_block(ex, i, max_input_chars=1000,
11             max_output_chars=1200))
12
13     sample_blocks = []
14     for offset, sample in enumerate(batch):
15         sample_blocks.append(compact_test_block(sample, batch_start + offset,
16             max_input_chars=1800))
17
18     prompt = f"""
19     请为下面这一批 OpenSeek-8 测试题生成结构化分析 JSONL。
20
21     任务定义：
22     {json.dumps(definitions, ensure_ascii=False, indent=2)}
23
24     算子手册摘要：
25     {operator_manual_md[:5000]}
26
27     参考训练样例：
28     {"\n\n".join(example_blocks)}
29
30     待分析测试题：
31     {"\n\n".join(sample_blocks)}
32
33     输出要求：
34     - 只输出 JSONL，每个测试题一行。
35     - 每行必须是合法 JSON 对象。
36     - 字段必须包含：
37       id, function, family, description, wrapper_signature, math, other,
38       detected_ops, answer_apis, answer_summary, thinking_steps
39     - detected_ops 和 answer_apis 用字符串数组。
40     - thinking_steps 用字符串数组，写 3 到 5 条可公开展示的简化解题步骤，不输出隐藏思维链。

```

```

39 - family 从以下集合中选择: conv_norm_pool, matmul_linear, attention_softmax_loss,
    linalg, indexing, activation, reduction, quantization, elementwise_math, generic。
40 """.strip()
41     return prompt

```

该 Prompt 明确要求：

- 每行一个 JSON 对象；
- 字段必须完整；
- `detected_ops` 和 `answer_apis` 必须是数组；
- `thinking_steps` 是公开简化步骤，不是隐藏思维链；
- `family` 必须从固定集合中选择。

## 4.6 JSONL 输出解析与补全

模型输出 JSONL 时，可能出现单行 JSON 不合法、字段缺失或 ID 不一致等问题。因此脚本需要进行解析和补全。核心原则是：

1. 保留原始 sample id，因为最终线上提交依赖这个 ID；
2. 如果 Qwen 输出缺字段，则用规则抽取补齐；
3. 如果 family 缺失，则用本地规则 `detect_task_family` 重新判断；
4. 如果 function 缺失，则用 `extract_function_name` 从题目输入中抽取；
5. 如果 detected\_ops 缺失，则用 `extract_ops` 补充；
6. 如果 thinking\_steps 缺失，则用模板步骤补充。

这保证了 `task8_per_question_analysis.jsonl` 即使在模型输出不稳定时，也能保持基本可用。

## 4.7 渲染生成 `task8_per_question_analysis.md`

当 JSONL 生成完成后，脚本会调用 `render_analysis_md` 生成 Markdown 可读版本。Markdown 文件不是主脚本直接读取的输入，但方便人工检查。例如每道题会展示：

```

1  ## 1. openseek-8-xxx - `fused_bmm_rmsnorm_gelu_dropout_sub`
2
3  - **任务类型**: matmul_linear
4  - **Wrapper**: ...
5  - **功能描述**: ...
6  - **数学定义**: ...
7  - **题目算子链**: ...
8  - **答案中显式 API**: ...
9  - **答案实现风格**: ...
10 - **拆解思路**:
11   1. 先识别 wrapper 函数名。
12   2. 映射到 PyTorch 算子链。
13   3. 优先使用 PyTorch fallback。
14   4. 处理 out= 和参数兼容。

```

该文件的价值在于：它可以让人工快速发现异常。例如：

- wrapper 函数名是否错误；
- family 是否明显不合理；
- detected\_ops 是否与题面不符；
- thinking\_steps 是否过空；
- 是否漏掉某些题目。

## 4.8 版本记录与人工线上评分

脚本会生成 `manifest.json`，记录当前候选版本信息。关键字段如下：

```
1 manifest = {
2     "candidate_tag": candidate_tag,
3     "created_at": datetime.now().isoformat(timespec="seconds"),
4     "model": model,
5     "task_id": task_id,
6     "long_prompt_chars": long_prompt_chars,
7     "long_prompt_requirement": "satisfied" if long_prompt_chars > 20000 else
    "not_satisfied",
8     "analysis_items": len(rows),
9     "family_distribution": dict(family_counter),
10    "files": files,
11    "scoring": {
12        "auto_score": False,
13        "reason": "线上评测分数只能通过提交平台获得，本脚本不做多版本自动评分。",
14        "selection_method": "人工将不同 candidate_tag 版本接入主生成脚本并提交线上评测，根据
    真实分数选择最终版本。",
15    },
16 }
```

## 4.9 中间文件构建命令示例

实际运行中间文件构建脚本时，可以使用如下命令：

```
1 export OPENAI_API_KEY=dummy
2 export OPENAI_BASE_URL=http://127.0.0.1:8000/v1
3
4 python build_task8_kb_files_no_autoscore.py \
5     --data /Users/ks/Desktop/LongContext-ICL-Annotation/data/opensseek-
    8_kernel_generation.json \
6     --out-dir /Users/ks/Desktop/LongContext-ICL-Annotation/kb_candidates/v4 \
7     --candidate-tag v4 \
8     --model /Qwen3-4B/Qwen/Qwen3-4B \
9     --batch-size 8
```

输出目录中会包含：

```
1 task8_long_prompt_v4.txt
2 task8_operator_manual.md
3 task8_per_question_analysis.jsonl
4 task8_per_question_analysis.md
5 manifest.json
```

其中：

- `task8_long_prompt_v4.txt` 用于记录大于 20k 的长 Prompt；
- `task8_operator_manual.md` 是生成的算子手册；
- `task8_per_question_analysis.jsonl` 是主脚本需要读取的逐题分析；
- `task8_per_question_analysis.md` 是人工检查版本；
- `manifest.json` 记录版本信息和人工线上评分说明。

## 5. 主预测脚本实现细节

### 5.1 数据加载

主脚本通过 `task_data_loader` 加载原始任务数据：

```
1 def task_data_loader(file_path: str) -> Tuple[str, List[Dict[str, Any]],
2   List[Dict[str, Any]], List[str]]:
3     with open(file_path, "r", encoding="utf-8") as f:
4         data = json.load(f)
5
6     return (
7         data.get("task_id", ""),
8         data.get("examples", []),
9         data.get("test_samples", []),
10        data.get("Definition", []),
11    )
```

该函数返回四类信息：

1. `task_id`：任务 ID；
2. `examples`：训练样例；
3. `test_samples`：待预测样例；
4. `definitions`：任务定义。

其中，训练样例主要用于清洗和潜在上下文支持；测试样例是最终需要生成代码的对象。

### 5.2 中间文件读取

主脚本读取 `task8_operator_manual.md`：

```

1 def load_text_file(path: str, default: str = "") -> str:
2     if not path or not os.path.exists(path):
3         print(f"[WARN] text file not found: {path}")
4         return default
5     with open(path, "r", encoding="utf-8") as f:
6         return f.read().strip()

```

读取逐题分析 JSONL:

```

1 def load_question_analysis(path: str) -> Dict[str, Dict[str, Any]]:
2     results: Dict[str, Dict[str, Any]] = {}
3
4     if not path or not os.path.exists(path):
5         print(f"[WARN] question analysis file not found: {path}")
6         return results
7
8     with open(path, "r", encoding="utf-8") as f:
9         for line_no, line in enumerate(f, 1):
10            line = line.strip()
11            if not line:
12                continue
13            try:
14                obj = json.loads(line)
15            except json.JSONDecodeError:
16                print(f"[WARN] skip invalid analysis JSONL line {line_no}")
17                continue
18
19            sid = obj.get("id") or obj.get("test_sample_id") or obj.get("sample_id")
20            if sid:
21                results[str(sid)] = obj
22
23     return results

```

这里将 JSONL 转换为字典，key 是 sample id，value 是该题的分析对象。这样最终生成每道题时可以 O(1) 查到当前样例对应的分析。

## 5.3 断点续跑机制

代码生成任务容易耗时较长。如果中途失败，不应该重新生成所有样例。因此脚本实现了断点续跑：

```

1 def load_existing_predictions(out_path: str) -> Dict[str, Dict[str, Any]]:
2     existing: Dict[str, Dict[str, Any]] = {}
3
4     if not os.path.exists(out_path):
5         return existing
6
7     with open(out_path, "r", encoding="utf-8") as f:
8         for line_no, line in enumerate(f, 1):
9             line = line.strip()
10            if not line:

```

```

11         continue
12
13     try:
14         obj = json.loads(line)
15     except json.JSONDecodeError:
16         print(f"[WARN] skip invalid JSONL output line {line_no}")
17         continue
18
19     sid = obj.get("test_sample_id") or obj.get("id")
20     pred = obj.get("prediction", "")
21
22     has_prediction = False
23     if isinstance(pred, str):
24         has_prediction = bool(pred.strip())
25     elif isinstance(pred, list):
26         has_prediction = bool(pred and str(pred[0]).strip())
27
28     if sid and has_prediction:
29         existing[str(sid)] = obj
30
31     return existing

```

该函数不仅兼容 `prediction` 为字符串的情况，也兼容旧版本中 `prediction` 为 list 的情况。这样可以避免因为格式变化导致已完成样例被重复生成。

## 5.4 输出 JSONL 写入

最终每道题的结果会通过 `safe_jsonl_row` 统一封装：

```

1 def safe_jsonl_row(test_sample_id: str, code: str) -> Dict[str, Any]:
2     return {"test_sample_id": test_sample_id, "prediction": code}

```

再通过 `append_jsonl` 追加到输出文件：

```

1 def append_jsonl(out_path: str, row: Dict[str, Any]) -> None:
2     out_dir = os.path.dirname(out_path)
3     if out_dir:
4         os.makedirs(out_dir, exist_ok=True)
5
6     with open(out_path, "a", encoding="utf-8") as f:
7         f.write(json.dumps(row, ensure_ascii=False) + "\n")

```

这种写法支持边生成边保存，避免全部样例生成完后才写文件造成中途结果丢失。

## 6. 题目信息抽取与任务分类

### 6.1 文本标准化

主脚本首先对输入文本进行标准化：



```

1 def normalize_text(text: Optional[str]) -> str:
2     if text is None:
3         return ""
4
5     text = str(text)
6     text = text.replace("\r\n", "\n").replace("\r", "\n")
7     text = re.sub(r"[ \t]+", " ", text)
8     text = re.sub(r"\n{3,}", "\n\n", text)
9     return text.strip()

```

该步骤可以统一换行符、压缩多余空格和空行，减少正则抽取时的不稳定性。

## 6.2 抽取 Wrapper Entry

题目中通常包含 `Wrapper Entry Information`，主脚本通过正则提取其中的函数签名：

```

1 def extract_wrapper_entry(input_text: str) -> str:
2     text = normalize_text(input_text)
3
4     m = re.search(
5         r"Wrapper Entry Information:\s*(.*?)(?:\n\s*Args:|\n\s*Keyword
6         args:|\n\s*Returns:|\n\s*Math:|\Z)",
7         text,
8         flags=re.DOTALL | re.IGNORECASE,
9     )
10    return m.group(1).strip() if m else ""

```

该函数的作用是从原始题面中截取 wrapper 信息，为后续函数名抽取和 Prompt 构造提供依据。

## 6.3 抽取函数名

函数名是否正确是线上评测的关键。如果生成代码没有定义题目要求的同名函数，即使内部逻辑正确，也无法通过调用。因此主脚本专门提供 `extract_function_name`：

```

1 def extract_function_name(input_text: str) -> str:
2     text = normalize_text(input_text)
3     entry = extract_wrapper_entry(text)
4
5     m = re.search(r"(?:def\s+)?([A-Za-z_]\w*)\s*\(", entry)
6     if m:
7         return m.group(1)
8
9     m = re.search(
10        r"(?:function|wrapper|entry)\s+['`\""]?([A-Za-z_]\w*)['`\""]?",
11        text,
12        flags=re.IGNORECASE,
13    )
14    if m:
15        return m.group(1)
16

```

```

17     lower = text.lower()
18     if "mean value" in lower or "computes the mean" in lower:
19         return "mean"
20     if "square system of linear equations" in lower:
21         return "solve"
22     if "conv2d" in lower and "add" in lower:
23         return "conv2d_add"
24
25     return "generated_function"

```

其逻辑为：

1. 优先从 Wrapper Entry 中抽取 `xxx(...)` 形式；
2. 如果失败，则从正文中寻找 function/wrapper/entry 后面的名称；
3. 对少数常见描述进行规则兜底；
4. 最后返回 `generated_function` 作为保底名称。

## 6.4 抽取 wrapper 签名

为了在 Prompt 中保留原始函数签名，主脚本使用：

```

1 def extract_wrapper_signature(input_text: str) -> str:
2     entry = extract_wrapper_entry(input_text)
3     if entry:
4         return entry.splitlines()[0].strip()
5     return ""

```

原始签名能提醒模型参数顺序和默认值，尤其对 `out=None`、`training=True`、`dim=-1` 等参数非常重要。

## 6.5 任务 family 分类

Task8 题目类型复杂，主脚本通过关键词将题目分为多个 family：

```

1 def detect_task_family(input_text: str) -> str:
2     lower = input_text.lower()
3
4     if any(x in lower for x in [
5         "conv2d", "conv1d", "conv3d", "pool2d", "batch_norm",
6         "instance_norm", "layer_norm", "group_norm", "pixel_shuffle",
7         "adaptive_avg_pool2d", "max_pool2d", "avg_pool2d",
8     ]):
9         return "conv_norm_pool"
10
11     if any(x in lower for x in [
12         "linear", "matmul", "matrix multiplication", "mm(", " bmm",
13         "torch.bmm", "mv", "addmm", "einsum", "matrix-vector", "matrix vector",
14     ]):
15         return "matmul_linear"
16
17     if any(x in lower for x in [

```

```

18         "attention", "softmax", "log_softmax", "cross_entropy",
19         "dropout", "transformer", "scaled dot-product",
20     ]):
21         return "attention_softmax_loss"
22
23     if any(x in lower for x in [
24         "svd", "qr", "lu", "cholesky", "solve", "inverse", "invert",
25         "determinant", "det(", "eigen", "eig", "pinv", "lstsq",
26         "least squares", "matrix_power", "matrix power",
27     ]):
28         return "linalg"

```

分类结果用于两件事：

1. 从 operator manual 中选择相关片段；
2. 决定生成策略，例如复杂 family 优先使用 PyTorch fallback。

## 6.6 抽取算子链

主脚本通过 `extract_ops` 将题目关键词映射为 PyTorch API：

```

1  def extract_ops(input_text: str) -> List[str]:
2      lower = input_text.lower()
3      ops: List[str] = []
4
5      candidates = [
6          ("conv2d", "F.conv2d"),
7          ("linear", "F.linear"),
8          ("bmm", "torch.bmm"),
9          ("matmul", "torch.matmul"),
10         ("batch_norm", "F.batch_norm"),
11         ("softmax", "F.softmax"),
12         ("dropout", "F.dropout"),
13         ("relu", "F.relu"),
14         ("gelu", "F.gelu"),
15         ("sigmoid", "torch.sigmoid"),
16         ("sqrt", "torch.sqrt"),
17         ("mean", "torch.mean"),
18         ("sum", "torch.sum"),
19         ("gather", "torch.gather"),
20         ("svd", "torch.linalg.svd"),
21         ("qr", "torch.linalg.qr"),
22         ("solve", "torch.linalg.solve"),
23     ]

```

实际代码中的候选表更长，覆盖 elementwise、activation、reduction、indexing、linalg、quantization 等多类算子。抽取结果会写入 Prompt 的 `Operation chain` 部分，引导模型按顺序组合 API。

## 6.7 构建生成策略

主脚本通过 family 决定策略：

```

1 def build_strategy(family: str, ops: List[str]) -> str:
2     if family in {
3         "conv_norm_pool", "matmul_linear", "attention_softmax_loss",
4         "linalg", "indexing", "activation", "reduction",
5     }:
6         return (
7             "Use PyTorch fallback only. Do not use complex Triton. "
8             "Compose the detected operations step by step in the listed order."
9         )
10
11     if family == "quantization":
12         return (
13             "Prefer PyTorch arithmetic implementation. Avoid complex Triton unless
14             the operation is simple row-wise dequantization."
15         )
16
17     if family == "elementwise_math":
18         return (
19             "Use PyTorch elementwise operations. Triton is allowed only for a very
20             simple elementwise kernel, but PyTorch fallback is preferred."
21         )
22
23     return "Use the safest PyTorch fallback implementation. Do not generate complex
24     Triton."

```

这里的核心是“先正确，再性能”。对复杂题目而言，PyTorch fallback 通常更容易保证语义和参数兼容，避免模型生成错误 Triton。

## 7. Prompt-KB 选择与最终 Prompt 构造

### 7.1 截断中间文本

主脚本并不会把完整 operator manual 和完整 per-question analysis 全部放进 Prompt。为了避免 Prompt 过长导致代码生成被截断，脚本设置：

```

1 MAX_OPERATOR_MANUAL_CHARS = 2500
2 MAX_QUESTION_ANALYSIS_CHARS = 1200

```

并通过 `truncate_text` 截断文本：

```

1 def truncate_text(text: str, max_chars: int) -> str:
2     text = normalize_text(text)
3     if len(text) <= max_chars:
4         return text
5     return text[:max_chars] + "\n\n...[TRUNCATED]..."

```

这说明：

- `>20k Prompt` 只用于中间文件构建阶段；
- 最终逐题代码生成阶段使用的是短 Prompt；
- 这样可以降低模型输出代码被截断的风险。

## 7.2 按 family 选择 operator manual 片段

主脚本根据当前题目的 family，从算子手册中筛选相关行：

```

1  def select_operator_manual_section(operator_manual: str, family: str, max_chars: int
    = MAX_OPERATOR_MANUAL_CHARS) -> str:
2      manual = normalize_text(operator_manual)
3      if not manual:
4          return ""
5
6      keywords_by_family = {
7          "activation": ["ReLU", "Sqrt", "Exp", "Log", "Sigmoid", "Tanh", "GELU",
"Softplus"],
8          "elementwise_math": ["Sqrt", "Exp", "Log", "Sigmoid", "Tanh", "zeta",
"chebyshev", "rsqrt"],
9          "reduction": ["Reduction", "max", "logsumexp", "rsqrt", "sum_std", "std",
"mean", "norm"],
10         "indexing": ["Index", "Index Fill", "Index Select", "gather", "masked", "赋
值"],
11         "linalg": ["solve", "svd", "qr", "cholesky", "det", "inverse", "linalg"],
12         "matmul_linear": ["linear", "bmm", "matmul", "mv", "matrix",
"softplus_linear", "fused_mv"],
13         "conv_norm_pool": ["conv", "conv2d", "batch_norm", "pool", "normalization"],
14         "attention_softmax_loss": ["attention", "softmax", "cross_entropy",
"dropout"],
15         "quantization": ["quantize", "dequantize", "int8", "fp8"],
16     }

```

该函数的优点是：

1. 当前题只读取相关算子知识；
2. 避免 Prompt 过长；
3. 减少无关 API 干扰；
4. 保持不同 family 的生成策略差异。

## 7.3 选择逐题分析

主脚本通过 sample id 获取当前题分析：

```

1 def select_question_analysis(sample_id: str, question_analysis: Dict[str, Dict[str,
  Any]]) -> str:
2     obj = question_analysis.get(sample_id, {})
3     if not obj:
4         return "No precomputed analysis for this sample."
5     text = json.dumps(obj, ensure_ascii=False, indent=2)
6     return truncate_text(text, MAX_QUESTION_ANALYSIS_CHARS)

```

如果某道题在 JSONL 中没有找到分析，也不会直接崩溃，而是返回提示文本。最终 Prompt 仍然可以依赖本地抽取规则生成。

## 7.4 固定 helper 代码

为了统一处理 `out=` 写回，Prompt 中固定包含 `_write_out`：

```

1 FIXED_HELPER_CODE = """def _write_out(value, out=None):
2     if out is None:
3         return value
4     if isinstance(value, tuple):
5         if isinstance(out, tuple):
6             for dst, src in zip(out, value):
7                 dst.copy_(src)
8             return out
9         return value
10    out.copy_(value)
11    return out
12 """

```

该 helper 解决三类问题：

1. `out is None` 时直接返回结果；
2. 如果输出是 tuple 且 out 也是 tuple，则逐项写回；
3. 如果输出是普通 Tensor，则执行 `out.copy_(value)`。

这对 PyTorch API 的兼容性非常重要。

## 7.5 System Prompt

主脚本的 system prompt 明确要求模型输出完整可执行 Python 代码，并把正确性放在最高优先级：

```

1 You are a code-generation model for OpenSeek-8 kernel generation.
2
3 Generate one complete executable Python code answer.
4
5 Priority:
6 1. Correctness
7 2. Importability / executable code
8 3. Exact wrapper function name
9 4. Robust positional and keyword argument handling
10 5. PyTorch semantic equivalence

```

```

11 6. Performance
12
13 Core strategy:
14 - Prefer PyTorch fallback over complex Triton.
15 - Do not blindly write Triton.
16 - Use Triton only for very simple elementwise kernels.
17 - For conv, matmul/bmm, attention, dropout, normalization, indexing, linalg, and
    complex fused blocks, use PyTorch.

```

这部分约束了模型的整体行为，避免其为了性能盲目写 Triton。

## 7.6 User Prompt 模板

最终每道题的 user prompt 包含四部分：

```

1 A. Relevant operator manual
2 B. Extracted rules for current sample
3 C. Current original test sample
4 D. Required output

```

模板中的关键字段包括：

```

1 USER_PROMPT_TEMPLATE = """
2 Solve this OpenSeek-8 test sample.
3
4 A. Relevant operator manual:
5 {operator_manual_for_sample}
6
7 B. Extracted rules for current sample:
8 Sample ID: {sample_id}
9 Wrapper function name: {function_name}
10 Original wrapper signature: {wrapper_signature}
11 Task family: {family}
12 Operation chain:
13 {ops_text}
14 Strategy: {strategy}
15
16 Precomputed analysis:
17 {analysis_text}
18
19 C. Current original test sample:
20 {current_input}
21
22 D. Required output:
23 Generate ONLY executable Python code.
24
25 The code MUST:
26 - start with import torch
27 - include import torch.nn.functional as F when useful
28 - include this exact helper code:
29

```

```

30 {fixed_helper_code}
31
32 - define exactly:
33   def {function_name}(*args, **kwargs):
34 - support positional args and kwargs
35 - support out= if present
36 - compose PyTorch operations step by step
37 - avoid complex Triton
38 - not be a minimal one-line wrapper
39 - not output explanation or markdown
40 - never write assignment expressions inside function call arguments
41 - use separate assignment lines before function calls
42 - never write code like: func(arg = value = other)
43
44 Final answer:
45 """.strip()

```

这说明最终 Prompt 并不是简单地把题目丢给模型，而是把以下信息全部显式提供给模型：

- 当前题 ID；
- 目标函数名；
- 原始 wrapper 签名；
- 任务 family；
- 算子链；
- 实现策略；
- 预生成逐题分析；
- 原始题面；
- 输出格式要求；
- 必须包含的 helper 代码。

## 7.7 构造 messages

最终由 `build_messages` 汇总上述信息：

```

1  def build_messages(
2      sample: Dict[str, Any],
3      definitions: List[str],
4      examples: List[Dict[str, Any]],
5      operator_manual: str,
6      question_analysis: Dict[str, Dict[str, Any]],
7  ) -> List[Dict[str, str]]:
8      current_input = normalize_text(sample.get("input", ""))
9      sample_id = str(sample.get("id", ""))
10
11     function_name = extract_function_name(current_input)
12     wrapper_signature = extract_wrapper_signature(current_input)
13     family = detect_task_family(current_input)

```



```

14 ops = extract_ops(current_input)
15 strategy = build_strategy(family, ops)
16 ops_text = build_ops_text(ops)
17
18 operator_manual_for_sample = select_operator_manual_section(operator_manual,
family)
19 analysis_text = select_question_analysis(sample_id, question_analysis)

```

该函数体现了 Prompt-KB 的核心：

```

1 当前题输入
2   ↓
3 本地规则抽取 function/family/ops
4   ↓
5 从 operator manual 选相关知识
6   ↓
7 从 JSONL 取当前题分析
8   ↓
9 拼成最终 messages

```

## 8. 模型调用、清洗与验证

### 8.1 Qwen API 调用

主脚本通过 OpenAI-compatible 接口调用 Qwen：

```

1 client = OpenAI(
2     api_key=os.getenv("OPENAI_API_KEY", "dummy"),
3     base_url=os.getenv(
4         "OPENAI_BASE_URL",
5         "https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1",
6     ),
7 )

```

调用函数为：

```

1 def qwen_api(
2     messages: List[Dict[str, str]],
3     model: str = MODEL_NAME,
4     retries: int = 3,
5     sleep_base: float = 6.0,
6 ) -> str:
7     for attempt in range(retries):
8         try:
9             res = client.chat.completions.create(
10                 model=model,
11                 messages=messages,
12                 temperature=0.0,
13             )

```

```

14         return res.choices[0].message.content or ""
15     except Exception as e:
16         if attempt == retries - 1:
17             print(f"\n[ERROR] API call failed after {retries} attempts: {e}")
18             return ""
19         wait = sleep_base * (2 ** attempt) + random.random()
20         time.sleep(wait)
21     return ""

```

这里使用 `temperature=0.0`，目的是降低随机性，让输出更稳定。重试机制可以缓解网络或服务临时错误。

## 8.2 清洗模型输出

模型有时会输出 markdown 代码块、解释文字或额外格式。主脚本通过 `clean_code_response` 提取纯代码：

```

1  def clean_code_response(text: str) -> str:
2      if not text:
3          return ""
4
5      text = str(text).strip()
6
7      fence = re.search(
8          r"```(?:python|py)?\s*(.*?)```",
9          text,
10         flags=re.DOTALL | re.IGNORECASE,
11     )
12     if fence:
13         text = fence.group(1).strip()
14
15     start_markers = ["import ", "from ", "@triton", "@torch", "def ", "class "]
16     positions = [text.find(m) for m in start_markers if text.find(m) != -1]
17     if positions:
18         text = text[min(positions):].strip()
19
20     return text.replace("```", "").strip()

```

该函数处理三类问题：

1. 移除 ```python 代码块；
2. 如果前面有解释文字，则从第一个 import/def/class 等位置开始截取；
3. 删除残留的反引号。

## 8.3 基础代码校验

清洗后的代码会进入 `validate_code_basic`：

```

1  def validate_code_basic(code: str, expected_func_name: str, family: str) ->
    Tuple[bool, str]:
2      if not code or not code.strip():

```

```

3         return False, "empty output"
4
5     if "`" in code:
6         return False, "markdown fence remains"
7
8     if re.search(r"\bTODO\b|\.\.\.", code):
9         return False, "placeholder detected"
10
11    if re.search(r"def\s+\w+\s*\([^)]*\):\s*\n\s*pass\s*(?:\n|$)", code):
12        return False, "pass-only function detected"
13
14    try:
15        tree = ast.parse(code)
16    except SyntaxError as e:
17        return False, f"syntax error: {e}"

```

校验项目包括：

- 输出是否为空；
- 是否残留 markdown；
- 是否包含 TODO 或省略号；
- 是否是 pass-only 函数；
- 是否能被 Python AST 解析。

随后检查函数名是否存在：

```

1 func_names = {
2     node.name for node in ast.walk(tree)
3     if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef))
4 }
5 class_names = {
6     node.name for node in ast.walk(tree)
7     if isinstance(node, ast.ClassDef)
8 }
9
10 if expected_func_name not in func_names and expected_func_name not in class_names:
11     return False, f"expected wrapper `{expected_func_name}` not defined"

```

这可以防止模型定义错函数名。

## 8.4 检测可疑 Triton

复杂 Triton 是本任务常见失败来源。主脚本通过 `has_complex_or_suspicious_triton` 拦截部分高风险输出：

```

1 def has_complex_or_suspicious_triton(code: str, family: str) -> bool:
2     if "@triton.jit" not in code and "triton.language" not in code:
3         return False
4
5     complex_families = {
6         "conv_norm_pool",
7         "matmul_linear",
8         "attention_softmax_loss",
9         "linalg",
10        "indexing",
11    }
12    if family in complex_families:
13        return True

```

对于 `conv_norm_pool`、`matmul_linear`、`attention_softmax_loss`、`linalg`、`indexing` 等复杂 family，如果模型生成 Triton，直接视为高风险。这样做的原因是这些任务通常涉及复杂维度、广播、stride 或矩阵分解，Triton 容易产生不可运行代码。

## 8.5 长度检查

对于复杂 family，代码过短通常意味着模型没有真正实现题目逻辑：

```

1 length_sensitive_families = {
2     "conv_norm_pool",
3     "matmul_linear",
4     "attention_softmax_loss",
5     "indexing",
6 }
7
8 if family in length_sensitive_families and len(code) < 500:
9     return False, f"code too short for complex family `{family}`"

```

这可以拦截“一行调用但没有处理参数”的弱实现。

## 9. 修复机制与规则回退

### 9.1 fatal syntax 直接 fallback

主脚本认为语法错误是高风险错误：

```

1 def is_fatal_truncated_syntax(reason: str) -> bool:
2     """
3     Treat every SyntaxError from the model as fatal and use rule fallback directly.
4     For Qwen-4B, repairing syntax errors often wastes time and may produce another
5     malformed answer.
6     """
7     return "syntax error" in reason.lower()

```

如果模型输出存在语法错误，脚本不一定继续修复，而是优先使用规则 fallback。这是因为语法错误往往来自代码截断、括号未闭合或模型生成格式混乱，继续让模型修复可能浪费时间且仍然失败。

## 9.2 repair prompt

如果不是 fatal syntax，且 `ENABLE_REPAIR=True`，脚本会构造修复 Prompt：

```
1  repair_prompt = f"""The previous code was invalid.
2
3  Validation error:
4  {reason}
5
6  Expected wrapper function:
7  {func_name}
8
9  Detected family:
10 {family}
11
12 Detected operations:
13 {build_ops_text(ops)}
14
15 Previous code:
16 {bad_code[:3000]}
17
18 Regenerate corrected code.
19
20 Rules:
21 - Output ONLY Python code.
22 - Define def {func_name}(*args, **kwargs):
23 - Include import torch and import torch.nn.functional as F.
24 - Include the exact _write_out helper from the prompt.
25 - Use PyTorch fallback style.
26 - Do not write complex Triton.
27 - Never write assignment expressions inside function call arguments.
28 - Use separate assignment lines before function calls.
29 - No markdown.
30 """
```

该 Prompt 会告诉模型：

- 上一次错误原因是什么；
- 正确函数名是什么；
- 当前 family 和算子链是什么；
- 上一次代码片段是什么；
- 修复后的输出仍必须是纯 Python 代码。

## 9.3 规则 fallback 模板

当模型生成失败或修复失败时，脚本使用规则模板生成兜底实现：

```

1  FALLBACK_TEMPLATE = """
2  import torch
3  import torch.nn.functional as F
4
5  def _write_out(value, out=None):
6      if out is None:
7          return value
8      if isinstance(value, tuple):
9          if isinstance(out, tuple):
10             for dst, src in zip(out, value):
11                 dst.copy_(src)
12             return out
13         return value
14     out.copy_(value)
15     return out
16
17 def _rms_norm(x, normalized_shape=None, eps=1e-5, weight=None):
18     if normalized_shape is None:
19         dims = (-1,)
20     elif isinstance(normalized_shape, int):
21         dims = (-1,)
22     else:
23         dims = tuple(range(x.dim() - len(tuple(normalized_shape)), x.dim()))
24     y = x * torch.rsqrt(torch.mean(x * x, dim=dims, keepdim=True) + eps)
25     if weight is not None:
26         y = y * weight
27     return y
28
29 def {func_name}(*args, **kwargs):
30     out = kwargs.pop("out", None)
31     name = "{func_name}"

```

fallback 的基本逻辑是：

1. 如果函数名正好是 `torch`、`torch.linalg`、`torch.special` 或 `F` 中已有 API，则直接分发调用；
2. 如果是常见复合函数名，则按规则组合 PyTorch API；
3. 支持 `out=` 写回；
4. 对 RMSNorm、conv2d 组合、linear 激活组合、reduction 组合、solve/cholesky solve 等常见模式提供兜底实现。

例如 conv2d 类 fallback：

```

1  if "conv2d" in name:
2      input = kwargs.get("input", args[0] if len(args) > 0 else None)
3      weight = kwargs.get("weight", args[1] if len(args) > 1 else None)
4      bias = kwargs.get("bias", args[2] if len(args) > 2 else None)
5      stride = kwargs.get("stride", 1)
6      padding = kwargs.get("padding", 0)
7      dilation = kwargs.get("dilation", 1)
8      groups = kwargs.get("groups", 1)

```

```

9     y = F.conv2d(input, weight, bias, stride, padding, dilation, groups)
10    if "add" in name:
11        other = kwargs.get("other", args[3] if len(args) > 3 else None)
12        if other is not None:
13            y = y + kwargs.get("alpha", 1) * other
14    if "batch_norm" in name:
15        running_mean = kwargs.get("running_mean", None)
16        running_var = kwargs.get("running_var", None)
17        bn_weight = kwargs.get("bn_weight", kwargs.get("weight_bn", None))
18        bn_bias = kwargs.get("bn_bias", kwargs.get("bias_bn", None))
19        training = kwargs.get("training", True)
20        momentum = kwargs.get("momentum", 0.1)
21        eps = kwargs.get("eps", 1e-5)
22        y = F.batch_norm(y, running_mean, running_var, bn_weight, bn_bias, training,
momentum, eps)

```

该规则不能保证覆盖全部题目，但能显著降低空输出、语法错误或完全不可运行代码的比例。

## 9.4 单题预测流程

主函数 `predict_one` 将模型生成、校验、修复和 fallback 串联起来：

```

1  def predict_one(...):
2      sid = str(sample.get("id", ""))
3      expected_func_name = extract_function_name(sample.get("input", ""))
4      family = detect_task_family(sample.get("input", ""))
5
6      code = ""
7
8      if use_model:
9          messages = build_messages(...)
10         raw = qwen_api(messages, model=model)
11         code = clean_code_response(raw)
12
13         ok, reason = validate_code_basic(code, expected_func_name, family)
14
15         if not ok and is_fatal_truncated_syntax(reason):
16             if enable_rule_fallback:
17                 return safe_jsonl_row(sid, make_rule_fallback_code(sample))
18
19         if not ok and enable_repair:
20             repaired = repair_code_with_prompt_kb(...)
21             repaired_ok, repaired_reason = validate_code_basic(repaired,
expected_func_name, family)
22             if repaired_ok:
23                 code = repaired
24                 ok, reason = True, "ok after repair"
25             else:
26                 code = repaired
27                 reason = repaired_reason
28

```

```
29         if ok:
30             return safe_jsonl_row(sid, code)
31
32     if enable_rule_fallback:
33         return safe_jsonl_row(sid, make_rule_fallback_code(sample))
34
35     return safe_jsonl_row(sid, code)
```

这一流程保证：

1. 优先使用模型生成结果；
2. 模型结果合法则直接采用；
3. 模型结果非法但可修复则尝试修复；
4. 修复失败则使用规则 fallback；
5. 最终始终尽量输出一个 JSONL row。

---

## 10. 主程序执行流程

主函数 `main` 的执行步骤如下。

### 10.1 加载数据与中间文件

```
1 def main() -> None:
2     task_id, examples, test_samples, definitions = task_data_loader(FILE_PATH)
3
4     operator_manual = load_text_file(OPERATOR_MANUAL_PATH)
5     question_analysis = load_question_analysis(PER_QUESTION_ANALYSIS_PATH)
6
7     operator_manual = truncate_text(operator_manual, 15000)
```

这里说明主脚本实际读取两个中间文件：

- `task8_operator_manual.md`；
- `task8_per_question_analysis.jsonl`。

`task8_per_question_analysis.md` 不在此处读取，它是人工查看用文件。

### 10.2 清洗训练样例与测试样例



```

1  for ex in examples:
2      ex["input"] = normalize_text(ex.get("input", ""))
3      out = ex.get("output", "")
4      if isinstance(out, list):
5          ex["output"] = [clean_code_response(str(x)) for x in out]
6      else:
7          ex["output"] = [clean_code_response(str(out))]
8
9  for sample in test_samples:
10     sample["input"] = normalize_text(sample.get("input", ""))

```

这一步统一输入格式，并清洗训练样例中的参考代码。

## 10.3 打印运行信息

```

1  print(f"task_id={task_id}")
2  print(f"examples={len(examples)}, test_samples={len(test_samples)}")
3  print(f"system_prompt_chars={len(SYSTEM_PROMPT)}")
4  print(f"operator_manual_chars={len(operator_manual)}")
5  print(f"question_analysis_items={len(question_analysis)}")
6  print(f"model={MODEL_NAME}")
7  print(f"use_model={USE_MODEL}, max_workers={MAX_WORKERS}")
8  print(f"out_path={OUT_PATH}")

```

这些日志可以确认：

- 原始数据是否正确读取；
- 中间文件是否成功加载；
- 模型名称是否正确；
- 输出路径是否正确；
- 当前有多少条逐题分析。

## 10.4 断点续跑和待处理样例

```

1  existing = load_existing_predictions(OUT_PATH)
2  pending_samples = [s for s in test_samples if str(s.get("id", "")) not in existing]
3  print(f"existing predictions={len(existing)}, pending={len(pending_samples)}")
4
5  if not pending_samples:
6      print("All test_samples already generated.")
7      return

```

这样可以避免重复调用模型，节省时间和 API 成本。

## 10.5 多线程生成

```

1  worker = partial(

```

```

2     predict_one,
3     definitions=definitions,
4     examples=examples,
5     operator_manual=operator_manual,
6     question_analysis=question_analysis,
7     model=MODEL_NAME,
8     use_model=USE_MODEL,
9     enable_repair=ENABLE_REPAIR,
10    enable_rule_fallback=ENABLE_RULE_FALLBACK,
11 )
12
13 with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
14     futures = {
15         executor.submit(worker, sample): sample
16         for sample in pending_samples
17     }

```

每个线程处理一道样例。完成后立即写入 JSONL：

```

1  for future in tqdm(as_completed(futures), total=len(futures), desc="Generating"):
2      sample = futures[future]
3      try:
4          result = future.result()
5      except Exception as e:
6          print(f"\n[ERROR] Generation failed: {sample.get('id', '')} | {e}")
7          try:
8              result = safe_jsonl_row(str(sample.get("id", "")),
make_rule_fallback_code(sample))
9          except Exception as fallback_error:
10             print(f"\n[ERROR] Emergency fallback failed: {sample.get('id', '')} |
{fallback_error}")
11             result = safe_jsonl_row(str(sample.get("id", "")), "")
12
13     append_jsonl(OUT_PATH, result)

```

即使某个 future 抛出异常，脚本也会尝试 emergency fallback，避免整个任务中断。

## 11. 三个中间文件的必要性总结

### 11.1 task8\_operator\_manual.md 的必要性

该文件解决的是“全局算子知识复用”问题。Task8 中大量题目都在重复使用相似算子，例如：

- `F.relu`、`F.gelu`、`torch.sigmoid`、`torch.tanh`；
- `torch.sqrt`、`torch.exp`、`torch.log`、`torch.rsqrt`；
- `F.linear`、`torch.bmm`、`torch.matmul`、`torch.mm`、`torch.mv`；
- `F.conv2d`、`F.batch_norm`、`F.layer_norm`、`F.group_norm`；
- `F.softmax`、`F.log_softmax`、`F.dropout`；

- `torch.sum`、`torch.mean`、`torch.std`、`torch.min`、`torch.max`；
- `torch.linalg.solve`、`torch.linalg.svd`、`torch.linalg.qr`、`torch.linalg.cholesky`。

如果每道题都让模型重新回忆这些 API 的使用方式，容易出现参数遗漏。算子手册将这些 API 固化成可检索知识，能提升生成稳定性。

## 11.2 task8\_per\_question\_analysis.jsonl 的必要性

该文件解决的是“单题理解前置化”问题。每道题在最终生成代码前，已经有对应的：

- 函数名；
- family；
- wrapper 签名；
- 功能描述；
- 数学定义；
- 算子链；
- API 候选；
- 实现策略；
- 公开简化步骤。

最终模型不再需要从零分析，而是按照这些字段完成实现。

## 11.3 task8\_per\_question\_analysis.md 的必要性

该文件解决的是“人工可读审查”问题。JSONL 适合程序读取，但不适合人工逐题检查。Markdown 版本可以快速检查 family 分布、题目拆解和实现风格，对发现错误非常有用。

尤其是在没有本地自动线上评分的情况下，人工检查中间文件质量是必要步骤。Markdown 文件可以辅助人工决定哪个 candidate\_tag 更适合提交线上评分。

---

## 12. 版本选择与线上评分说明

由于线上平台分数只能通过提交系统获得，本地脚本不做自动评分。正确流程如下：

1. 用 `build_task8_kb_files_no_autoscore.py` 生成 `candidate_tag=v1` 的中间文件。
2. 将 `v1` 的 `task8_operator_manual.md` 和 `task8_per_question_analysis.jsonl` 接入主脚本。
3. 运行主脚本生成提交 JSONL。
4. 上传线上平台获取真实分数。
5. 修改 Prompt 或中间文件生成策略，生成 `candidate_tag=v2/v3/v4`。
6. 重复提交。
7. 根据线上真实分数选择最终版本。

---

## 13. 稳定性设计

### 13.1 优先保证函数名正确

线上评测通常会导入生成代码并调用指定函数。如果函数名错误，代码逻辑再正确也无法通过。因此主脚本多处强调：

```
1 Define the exact wrapper function name.
2 def {function_name}(*args, **kwargs):
```

同时本地验证会检查 AST 中是否存在目标函数名。

## 13.2 优先 PyTorch fallback

本任务虽然是 kernel generation，但不是所有题都适合写 Triton。对于矩阵分解、线性代数、归一化、attention、indexing 等任务，PyTorch API 更稳定。

因此 system prompt 中明确写道：

```
1 Prefer PyTorch fallback over complex Triton.
2 Do not blindly write Triton.
3 Use Triton only for very simple elementwise kernels.
```

## 13.3 参数兼容

Prompt 中明确要求支持：

```
1 out=, inplace, dim, keepdim, dtype, eps, alpha, beta, threshold, p, dropout_p,
   training, stride, padding, dilation, groups
```

这些参数是 PyTorch wrapper 中最常见的兼容点。漏掉这些参数会导致评测调用失败。

## 13.4 输出格式控制

模型必须只输出 Python 代码：

```
1 Output only Python code.
2 No markdown.
3 No JSON.
4 No explanations.
5 Start with import torch.
```

同时清洗函数会移除 markdown code fence，验证函数会检查是否残留 ````。

## 13.5 修复与 fallback

本方案不是完全相信模型输出，而是形成了多层防线：

```
1 Qwen 生成
2   ↓
3 clean_code_response 清洗
4   ↓
5 validate_code_basic 校验
6   ↓
7 repair_code_with_prompt_kb 修复
8   ↓
9 make_rule_fallback_code 规则回退
```

这样即使模型输出不稳定，也能尽量保证最终 JSONL 有可执行代码。

## 14 本方案最终形成了一条稳定的 Prompt-KB 代码生成流程：

```
1 1. 读取 openseek-8_kernel_generation.json。
2 2. 使用 build_task8_kb_files_no_autoscore.py 构造一次 >20k 长 Prompt。
3 3. 调用 Qwen 生成 task8_operator_manual.md。
4 4. 分批调用 Qwen 生成 task8_per_question_analysis.jsonl。
5 5. 将 JSONL 渲染为 task8_per_question_analysis.md。
6 6. 按 candidate_tag 保存不同候选版本。
7 7. 人工将不同版本接入主脚本并进行线上评分。
8 8. 选择线上表现最好的中间文件版本。
9 9. 运行 task8_prompt_kb_generator copy.py 生成最终提交 JSONL。
10 10. 主脚本逐题读取 operator manual 片段和对应 analysis。
11 11. Qwen 生成代码。
12 12. 本地清洗、校验、修复和 fallback。
13 13. 追加写入最终 JSONL。
```

该流程的关键优势是：

- 将复杂代码生成任务拆成“知识构建”和“代码生成”两个阶段；
- 使用中间文件降低每道题即时分析压力；
- 使用 >20k Prompt 满足长上下文要求；
- 使用人工线上评分保证版本选择真实可信；
- 使用 AST 校验、repair 和 fallback 提升最终输出稳定性；